


DBMS Complete Exam Notes

Chapters 1 & 2: Introduction & Architecture

Subject: Database Management Systems

Covers: Fundamentals, Data Models, Three-Schema
Architecture, DBMS Classification

Includes: Diagrams, Definitions, Q&A for Exam Preparation

EXAM EDITION 2026

Table of Contents

Chapter 1: Introduction to Database Management Systems

- 1.1 Introduction
- 1.2 An Example: The UNIVERSITY Database
- 1.3 Characteristics of the Database Approach
- 1.4 Actors on the Scene
- 1.5 Workers behind the Scene
- 1.6 Advantages of Using the DBMS Approach
- 1.8 When Not to Use a DBMS

Chapter 2: Database System Concepts and Architecture

- 2.1 Data Models, Schemas, and Instances
- 2.2 Three-Schema Architecture and Data Independence
- 2.3 DBMS Languages
- 2.4 Database System Environment
- 2.6 Classification of DBMS

Exam Q&A Section

Chapter 1: Introduction to Database Management Systems (DBMS)

1.1 Introduction

I. Impact and Scope of Databases

Databases and database technology are foundational to modern computer usage across virtually every domain:

- **Business:** Customer records, sales transactions, inventory management
- **E-commerce:** Product catalogs, order processing, payment systems
- **Social Media:** User profiles, relationships, content feeds
- **Engineering:** Design specifications, simulation data, CAD libraries
- **Medicine & Genetics:** Patient records, genomic sequences, clinical trials
- **Law & Education:** Case databases, student records, research archives
- **Library Science:** Cataloging systems, circulation management

II. Fundamental Definitions

Data

Known facts that can be recorded and possess implicit meaning.

Example: Names, telephone numbers, and addresses stored in a mobile phone, address book, MS Access, or Excel.

Database

A collection of **related data** with an implicit meaning. A random assortment of data does NOT constitute a database.

Database Management System (DBMS)

A computerized, general-purpose software system that enables users to create and maintain a database.

Database System

The comprehensive system comprised of the **Database** itself plus the **DBMS software**.

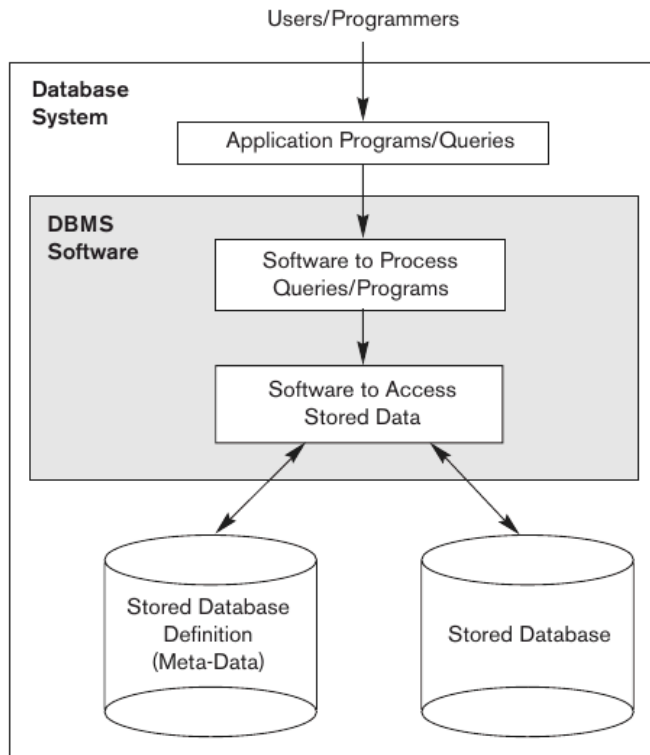


Figure 1.1
A simplified database
system environment.

Figure 1.1: A simplified database system environment showing *Users/Programmers* interacting with *Application Programs/Queries*, which pass through the *DBMS Software* to access both the *Stored Database* and the *Stored Database Definition (Meta-Data)*.

III. Implicit Properties of a Database

A true database possesses three implicit properties that distinguish it from random data collection:

Table 1.1: *The Three Implicit Properties of a Database*

Property	Description	Key Point
1. Representation of the Real World	Represents a specific aspect of the real world (miniworld or Universe of Discourse - UoD)	Changes in the UoD must be reflected in the database ASAP to maintain accuracy
2. Logical Coherence	A logically coherent collection of data with inherent, non-random meaning	Data is organized purposefully, not haphazardly
3. Specific Purpose and Audience	Designed, built, and populated for a specific purpose with intended users and applications	Features active interaction: users perform transactions that alter the database

IV. Size, Complexity, and Evolution

Databases range vastly in size and complexity:

- **Small/Simple:** A list of names and addresses (few hundred records, simple structure)
- **Medium:** Computerized catalog of a large library (half a million entries, organized by author, subject, title)
- **Massive/Highly Complex:**
 - **Social Media (e.g., Facebook):** Maintains over a billion users, tracks complex relationships (friends), posts, privacy/permissions, requiring a large number of databases to handle constantly changing information
 - **Commercial E-commerce (e.g., Amazon):** Over 60 million active users and millions of inventory items; occupies >42 Terabytes distributed across hundreds of servers; millions of daily visitors with continuous updates

Exam Key Point

Databases can be **manual** (e.g., physical library card catalog) or **computerized**. University study focuses *strictly* on computerized databases.

V. Core Processes Facilitated by a DBMS

A general-purpose DBMS facilitates four primary processes:

**Table 1.2:** *Four Core DBMS Processes*

Process	Description	Details
1. Defining	Specifying data types, structures, and constraints	Meta-data (database definition) stored in the DBMS catalog/dictionary
2. Constructing	Storing data on a storage medium controlled by the DBMS	Actual process of populating the database with data
3. Manipulating	Querying, updating, and reporting	Retrieving specific data; changing data to reflect UoD changes; generating reports
4. Sharing	Allowing multiple users and applications simultaneous access	Requires concurrency control mechanisms

VI. Additional DBMS Functions and Application Interaction

- **Query:** Typically causes data to be retrieved from the database
- **Transaction:** May cause data to be both read and written/modified
- **System Protection:** Safeguarding against hardware or software crashes/malfunctions
- **Security Protection:** Safeguarding against unauthorized or malicious access
- **Long-term Maintenance:** Databases have life cycles of many years; the DBMS must allow the system to evolve as requirements change

Special-Purpose vs. General-Purpose DBMS

While general-purpose DBMS software is standard, organizations can write custom programs to act as a **special-purpose DBMS** (e.g., legacy airline reservation systems). Both approaches require highly complex software deployment.

1.2 An Example: The UNIVERSITY Database

To illustrate the concepts of defining, constructing, and manipulating a database, we examine a simple UNIVERSITY database environment maintaining data on students,

courses, and grades.

I. Database Structure and Files

The database is logically organized into five distinct "files" (or tables), each storing data records of the same type:

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

Figure 1.2
A database that stores student and course information.

Figure 1.2: A database that stores student and course information, organized into five tables: *STUDENT*, *COURSE*, *SECTION*, *GRADE_REPORT*, and *PREREQUISITE*.

II. Defining the Database

Defining involves specifying structures, data elements, data types, and coding schemes.

Data Types Example

- **Name** (STUDENT) = String of alphabetic characters
- **Student_number** (STUDENT) = Integer
- **Grade** (GRADE_REPORT) = Single character from a constrained set: {'A', 'B', 'C', 'D', 'F', 'T'}

Coding Schemes

Used to represent values efficiently:

- **Class variable in STUDENT:** 1 = freshman, 2 = sophomore, 3 = junior, 4 = senior, 5 = graduate student

III. Constructing the Database

Constructing involves storing the actual data to represent entities and their relationships.

- **Relationships:** Records across different files are interconnected.
 - Example 1: The record for student "Smith" in the STUDENT file is related to two separate records in the GRADE_REPORT file, indicating grades for two different sections.
 - Example 2: The PREREQUISITE file relates two COURSE records together (the target course and its required prerequisite).

Key Point

Medium to large databases contain many types of records and highly complex webs of relationships.

IV. Manipulating the Database

Manipulation involves queries (retrievals) and updates (modifications). These informal requests must be precisely translated into the formal query language of the DBMS to be processed.

Table 1.4: *Examples of Database Manipulation*

Type	Example Operation
Queries (Retrieval)	Retrieve the transcript (list of all courses and grades) for 'Smith'
	List the names and grades of students who took the 'Database' course section in Fall 2008
	List the prerequisites for the 'Database' course
Updates (Modification)	Change the class status of 'Smith' to sophomore
	Create a new section for the 'Database' course for the current semester
	Enter a grade of 'A' for 'Smith' in last semester's 'Database' section

V. Database Design within an Information System

A database is typically part of a broader Information System designed and maintained by an organization's IT department. The creation follows a distinct lifecycle:

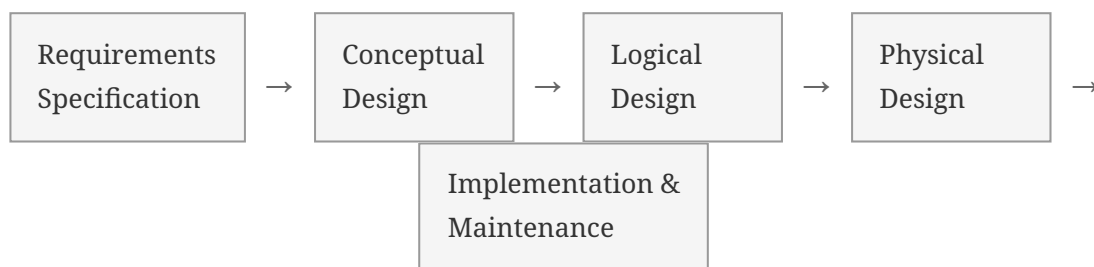


Table 1.5: Database Design Lifecycle

Phase	Description	Key Output
1. Requirements Specification	Document requirements in detail	Requirements document
2. Conceptual Design	Transform requirements into a conceptual design using tools like the ER model	ER Diagram
3. Logical Design	Translate conceptual design into a data model the DBMS can implement (e.g., Relational Model)	Relational schema
4. Physical Design	Define how data will be stored physically and accessed efficiently	Storage structures, indexes
5. Implementation & Maintenance	Build, populate, and continuously maintain the database	Operational database

1.3 Characteristics of the Database Approach

The database approach fundamentally differs from the older traditional file-processing approach:

Table 1.6: File Processing vs. Database Approach

Aspect	Traditional File Processing	Database Approach
Data Storage	Each user/department defines and implements their own files	Single, centralized repository; data defined once and accessed by various users
Redundancy	High duplication of data across files	Controlled redundancy through normalization
Maintenance	Redundant maintenance efforts	Centralized maintenance
Example	Grade reporting office keeps student grade files; accounting office keeps student fee files separately	One integrated system serving all departments

Characteristic 1: Self-Describing Nature of a Database System

A database system contains both the data itself AND a complete definition/description of the database structure and constraints.

The DBMS Catalog

Where the definition is stored. Contains **meta-data** (data about data), such as file structures, data types, storage formats, and constraints.

RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

Figure 1.3

An example of a database catalog for the database in Figure 1.2.

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....		
....		
....		
Prerequisite_number	XXXXNNNN	PREREQUISITE

Note: Major_type is defined as an enumerated type with all known majors.

XXXXNNNN is used to define a type with four alphabetic characters followed by four numeric digits.

Figure 1.3: An example of a database catalog for the UNIVERSITY database. The RELATIONS table stores table names and column counts; the COLUMNS table stores column names, data types, and their parent relations.

- **General-Purpose Flexibility:** Because the DBMS refers to the catalog to understand the structure, the same DBMS software can work with a university database, a banking database, or a company database.
- **Contrast with File Processing:** In file processing, data definition is hardcoded into the application programs (e.g., struct or class declarations in C++). Thus, a file-processing program can only access one specific database structure.

Characteristic 2: Insulation between Programs and Data, and Data Abstraction

Program-Data Independence

Changes to the structure of a data file (e.g., adding a Birth_date field) do not require changing the application programs that access the file. The DBMS simply updates the catalog, and the programs continue to work seamlessly.

Data Item Name	Starting Position in Record	Length in Characters (bytes)
Name	1	30
Student_number	31	4
Class	35	1
Major	36	4

Figure 1.4
Internal storage format for a STUDENT record, based on the database catalog in Figure 1.3.

Figure 1.4: Internal storage format for a STUDENT record based on the database catalog. Name starts at position 1 (30 bytes); Student_number at position 31 (4 bytes); Class at position 35 (1 byte); Major at position 36 (4 bytes). Users are insulated from these details.

Data Abstraction

The DBMS provides users with a **conceptual representation** of data, hiding low-level storage and implementation details. A **Data Model** is a type of data abstraction using logical concepts (objects, properties, relationships) that users easily understand.

Program-Operation Independence

Found in object-oriented (OO) and object-relational systems. Users define operations in two parts:

1. **Interface (Signature):** Operation name and data types of its arguments
2. **Implementation (Method):** The actual code. This can be changed without altering the interface or affecting the application programs that call the operation.

Characteristic 3: Support of Multiple Views of the Data

A database has diverse users, and a multiuser DBMS provides facilities for defining multiple views.

View

A customized perspective of the database. It can be a subset of the explicit database, or it can contain **virtual data** (data derived from files but not explicitly stored). Users do not need to know if data is stored or derived.

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

(a)

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

(b)

Figure 1.5

Two views derived from the database in Figure 1.2. (a) The TRANSCRIPT view.

(b) The COURSE_PREREQUISITES view.

Figure 1.5: Two views derived from the database: (a) The TRANSCRIPT view combines information from STUDENT, COURSE, SECTION, and GRADE_REPORT. (b) The COURSE_PREREQUISITES view shows course names, numbers, and prerequisites for advisors.

Characteristic 4: Sharing of Data and Multiuser Transaction Processing

Concurrency Control

Software within the DBMS that ensures multiple users trying to update the same data do so in a controlled manner, guaranteeing correct results.

Example: Airline seat reservation -- the DBMS ensures only one agent can assign a specific seat at a time.

Online Transaction Processing (OLTP)

Applications that rely heavily on concurrent transactions.

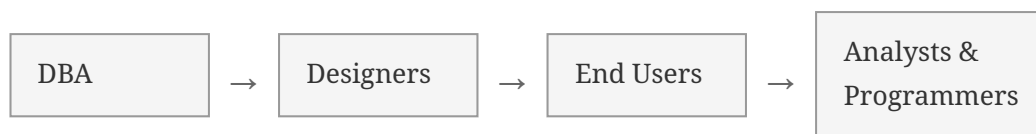
Transaction

An executing program or process that includes one or more database accesses (read/update). The DBMS enforces strict transaction properties:

- **Isolation Property:** Each transaction appears to execute entirely in isolation, even if hundreds of others are executing concurrently.
- **Atomicity Property:** The "all-or-nothing" rule. Either all database operations within a transaction are completed successfully, or none of them are.

1.4 Actors on the Scene

In a small personal database, a single person typically handles all tasks. In large organizations with hundreds or thousands of users, tasks are distributed among specialized roles.



1.4.1 Database Administrators (DBA)

The chief administrator responsible for overseeing primary resources (the database) and secondary resources (the DBMS and related software).

Table 1.7: *DBA Key Responsibilities*

Responsibility	Description
Authorizing Access	Granting permissions to users
Coordinating Use	Monitoring and coordinating database usage
Resource Acquisition	Acquiring necessary software and hardware resources
Accountability	Strictly accountable for system issues, including security breaches and poor system response times

1.4.2 Database Designers

Responsible for determining the optimal structure of the database before it is implemented. Often members of the DBA's staff.

- Identifying the data to be stored
- Choosing appropriate structures to represent and store this data
- Communicating with prospective database users to understand their requirements
- Developing customized views for each user group
- Analyzing and integrating all views to create a final design capable of supporting all user groups

1.4.3 End Users

The people whose jobs require access to the database. They are categorized into four types:

Table 1.8: *Types of End Users*

Type	Characteristics	Examples
1. Casual End Users	Occasionally access; often need different information each time; use sophisticated query interfaces	Middle/high-level managers, occasional browsers
2. Naive/Parametric Users	Large portion of users; constantly query/update using pre-programmed canned transactions ; minimal DBMS knowledge needed	Bank tellers, airline reservation agents, shipping clerks, social media users
3. Sophisticated End Users	Thoroughly familiar with DBMS facilities; implement their own custom applications	Engineers, scientists, business analysts
4. Standalone Users	Maintain personal databases using ready-made, easy-to-use software (menu/graphics-based)	Users managing personal data with financial software

1.4.4 System Analysts and Application Programmers (Software Engineers)

- **System Analysts:** Determine the requirements of end users and develop specifications for the necessary canned transactions.
- **Application Programmers:** Take specifications from analysts and implement them as programs. They test, debug, document, and maintain these canned transactions.

1.5 Workers behind the Scene

These people are associated with the design, development, and operation of the DBMS *software and system environment*. They are **not** interested in the database content itself, but rather the functionality of the system.

Table 1.9: *Workers Behind the Scene*

Role	Responsibilities
DBMS System Designers and Implementers	Design and build the complex software modules that make up the DBMS: catalog, query language processing, interface processing, data accessing and buffering, concurrency control, data recovery, and security handling. Ensure the DBMS interfaces correctly with other system software (OS, compilers).
Tool Developers	Independent vendors or internal teams who design and implement optional tools: database modeling and design, performance monitoring, natural language/graphical interfaces, prototyping, simulation, and test data generation.
Operators and Maintenance Personnel (System Administration Personnel)	Responsible for the actual day-to-day running and maintenance of the hardware and software environment that hosts the database system.

1.6 Advantages of Using the DBMS Approach

1.6.1 Controlling Redundancy

Traditional file processing forces each department to maintain separate files (e.g., registration vs. accounting).

Problems with Uncontrolled Redundancy

1. **Duplication of Effort:** Same data must be entered multiple times across different files
2. **Wasted Storage:** Repeated data consumes valuable space
3. **Inconsistencies:** Updates applied to one file but not another (or typos) lead to contradictory data

Data Normalization

The ideal database design approach where logical data items are stored in **only one place** to ensure consistency and save space.

Denormalization (Controlled Redundancy)

Occasionally used in practice to improve query performance. The DBMS must control this to prevent errors via integrity checks.

Figure 1.6

Redundant storage of Student_name and Course_name in GRADE_REPORT.
(a) Consistent data.
(b) Inconsistent record.

GRADE_REPORT				
Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

(a)

GRADE_REPORT				
Student_number	Student_name	Section_identifier	Course_number	Grade
17	Brown	112	MATH2410	B

(b)

Figure 1.6: Redundant storage of Student_name and Course_number in GRADE_REPORT. (a) Consistent data. (b) Inconsistent record -- Student_number 17 is incorrectly listed with name "Brown" instead of "Smith". Controlled redundancy checks prevent this.

1.6.2 Restricting Unauthorized Access

A multiuser DBMS must control who can access data and how they can manipulate it (read vs. write access).

- **Security & Authorization Subsystem:** Used by the DBA to create accounts, set passwords, and enforce strict access restrictions automatically.

- Protections apply to the data (e.g., hiding confidential salaries) as well as the DBMS software (e.g., ensuring parametric users only use predefined "canned transactions").

1.6.3 Providing Persistent Storage for Program Objects

Impedance Mismatch Problem

Traditional DBMSs and programming languages have incompatible data structures. A developer usually has to write complex code to convert a C++ or Java object into a format suitable for flat file storage, and convert it back upon retrieval.

Object-Oriented (OO) DBMSs

Resolve the impedance mismatch by offering data structure compatibility with languages like C++ and Java. **Persistent Objects** -- complex program objects (like C++ class objects) can be directly stored by the OO-DBMS. They survive the termination of program execution and can be retrieved instantly by other programs.

1.6.4 Storage Structures and Search Techniques for Query Processing

- **Indexes:** Auxiliary files (usually Tree or Hash data structures) that drastically speed up disk searches.
- **Buffering/Caching:** The DBMS maintains parts of the database in main memory buffers. The DBMS typically runs its own buffering module specifically optimized for database performance.
- **Query Processing & Optimization:** The DBMS evaluates available storage structures and indexes to calculate and execute the most efficient query execution plan.

1.6.5 Providing Backup and Recovery

- **Backup and Recovery Subsystem:** Protects against hardware or software failures.
- If a system crashes mid-transaction, the DBMS automatically restores the database to its precise state prior to the transaction starting.
- Also handles full disk backups for catastrophic hardware failures.

1.6.6 Providing Multiple User Interfaces

A DBMS caters to varying technical skill levels by providing diverse interfaces:

- Query languages for casual users

- Programming language interfaces for software developers
- Apps, forms, and command codes for parametric users
- Graphical User Interfaces (GUIs), menu-driven, natural language, and Web-enabled interfaces for standalone users

1.6.7 Representing Complex Relationships Among Data

Data is inherently interrelated (e.g., a student is linked to multiple grades, a section is linked to one course and many students). A DBMS must easily define, retrieve, and efficiently update complex webs of relationships.

1.6.8 Enforcing Integrity Constraints

A DBMS provides capabilities to define and enforce business rules and constraints that data must obey:

Table 1.10: *Types of Integrity Constraints*

Constraint Type	Description	Example
Data Type Constraints	Restricts the type of data that can be stored	Class must be an integer; Name cannot exceed 30 characters
Referential Integrity Constraints	A record in one file must validly relate to a record in another	A SECTION record cannot exist without pointing to a valid COURSE
Key/Uniqueness Constraints	Ensures uniqueness of identifier values	No two courses can have the same Course_number
Inherent Rules	Broad rules mandated by the specific data model	In the ER model, a relationship must involve at least two entities

Important Note on Constraint Limits

A DBMS can enforce *logical* constraints (rejecting 'Z' as a grade) but cannot automatically catch *human-error validity* if the constraint is met (e.g., typing 'C' instead of 'A' when 'C' is a valid option).

1.6.9 Permitting Inferencing and Actions Using Rules and Triggers

Deductive Database Systems

Allow the definition of declarative deduction rules to infer new information from stored facts (e.g., automatically inferring a student's academic probation status based on complex GPA rules, rather than writing explicit procedural code).

Triggers

A rule activated by a table update that subsequently performs additional operations or sends messages.

Active Database Systems

Systems equipped with active rules that automatically initiate actions based on predefined events and conditions.

Stored Procedures

Involved procedures to enforce rules invoked under specific conditions.

1.6.10 Additional Implications of the Database Approach

Table 1.11: *Additional Advantages of the DBMS Approach*

Advantage	Description
Enforcing Standards	The DBA can enforce standardized naming, data formats, and reporting structures centrally across the entire organization.
Reduced Development Time	Implementing a new application on an existing DBMS takes an estimated 1/6th to 1/4th of the time compared to building it from scratch on a file system.
Flexibility	The database structure can organically evolve (e.g., adding a new file or extending an element) without breaking existing data or application programs.
Up-to-Date Information	Through concurrency control, an update by one user is immediately visible to all other users system-wide (critical for banking or reservations).
Economies of Scale	Consolidating data reduces organizational overlap. Instead of individual departments buying lower-performance hardware, the organization can pool funds to invest in a highly powerful, centralized server/network.

1.8 When Not to Use a DBMS

Despite its massive advantages, there are specific scenarios where employing a general-purpose DBMS introduces unnecessary overhead costs.

I. Overhead Costs of a DBMS

1. High initial investment required for hardware, software, and specialized staff training.
2. The massive generality the DBMS provides for defining/processing data (which may be unneeded).
3. The computational overhead of constantly running security, concurrency control, recovery, and integrity subsystems.

II. Circumstances Favoring Customized File Processing

It may be more desirable to bypass a DBMS and use customized, specialized file applications if:

- The database application is **simple, well-defined, and expected to never change**.
- The application has **stringent, real-time requirements** that the inherent computational overhead of a DBMS would fail to meet.
- The system is an **embedded system** with highly limited physical storage capacity where a general-purpose DBMS simply wouldn't fit.
- The system **strictly requires no multiple-user access** to the data.

III. Industry Examples Avoiding General-Purpose DBMSs

Table 1.12: *Industries Using Custom File Management*

Industry	Reason for Custom Solution
CAD Tools (Mechanical/Civil Engineering)	Proprietary software optimized specifically for the internal manipulation of dense 3D objects and drawings
Communication & Switching Systems (e.g., AT&T)	Custom hierarchical structures geared solely for lightning-fast access and routing of phone calls
Geographic Information Systems (GIS)	Customized data organizations uniquely built to efficiently process and render maps, physical contours, lines, and polygons

Chapter 2: Database System Concepts and Architecture

2.1 Data Models, Schemas, and Instances

A primary goal of the database approach is to provide **data abstraction**, which suppresses details regarding how data is internally organized and stored, thereby highlighting essential features for better understanding.

Data Model

Provides the necessary means to achieve data abstraction. It is a collection of concepts used to describe:

- **The Structure of a Database:** Data types, relationships, and constraints
- **Basic Operations:** Generic capabilities to specify retrievals and updates (e.g., insert, delete, modify, retrieve)
- **Dynamic Behavior (User-Defined Operations):** Modern data models allow designers to specify valid user-defined operations on database objects (e.g., COMPUTE_GPA on a STUDENT object)

2.1.1 Categories of Data Models

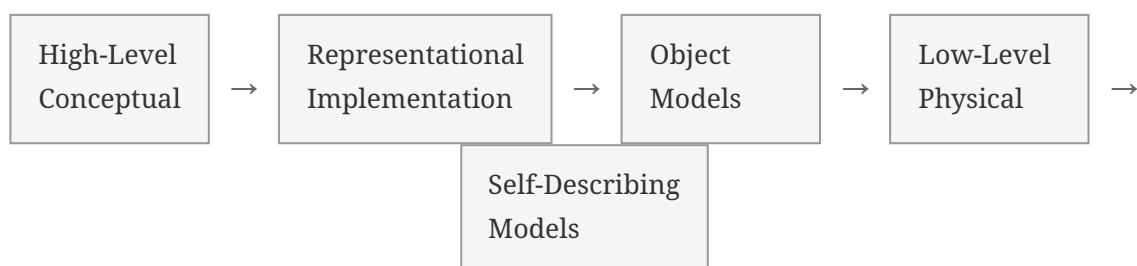


Table 2.1: *Categories of Data Models*

Category	Description	Key Concepts / Examples
1. High-Level or Conceptual	Concepts very close to how human users perceive data	Entity (real-world object), Attribute (property), Relationship (association). Example: ER model
2. Representational (Implementation)	Sit between conceptual and physical; hide disk storage details but can be implemented directly; concepts easily understood by end-users	Relational model (widely used), Network model, Hierarchical model. Called "record-based" data models.
3. Object Data Models	Higher-level implementation models closer to conceptual models; often used in software engineering	ODMG object model (Object Data Management Group standard)
4. Low-Level or Physical	Describe precise details of how data is physically stored as files on storage media. Meant exclusively for computer specialists.	Record formats, record orderings, access paths (index or hash search structures)
5. Self-Describing Data Models	Combine the description and the data values within one structure (unlike traditional DBMSs where schema is separate)	XML, Key-value stores, NOSQL systems for big data

2.1.2 Schemas, Instances, and Database State

It is crucial to distinguish between the abstract description of a database and the actual data it contains.

Database Schema (The Intension)

The formal description of a database. Specified during database design and is not expected to change frequently.

- **Schema Construct:** Each individual object defined in the schema (e.g., the STUDENT or COURSE tables)
- **Schema Diagram:** A visual representation of a schema

Figure 2.1

Schema diagram for the database in Figure 1.2.

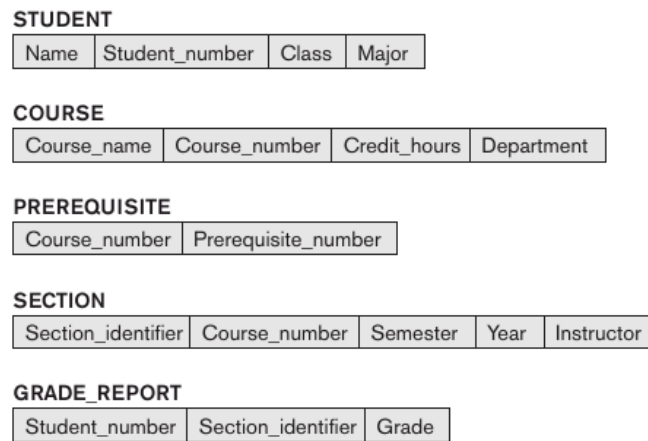


Figure 2.1: Schema diagram for the *UNIVERSITY* database from Figure 1.2, showing the structure of *STUDENT*, *COURSE*, *PREREQUISITE*, *SECTION*, and *GRADE_REPORT* relations with their attributes.

Database State (The Extension / Snapshot / Instances)

The actual data populated within the database at any particular moment in time. The data changes frequently -- every insert, delete, or modification transitions the database from one state to another.

Valid State

A state that strictly satisfies all the structures and constraints specified in the schema. The DBMS enforces this validity by storing the schema as meta-data and checking updates against it.

Schema Evolution

Applying changes to a live schema (e.g., needing to suddenly track a student's Date_of_birth). Most modern DBMSs allow this while the database remains operational.

Table 2.2: Lifecycle of Database States

State	Description
1. Empty State	When the schema is initially defined to the DBMS, but no data exists yet
2. Initial State	Achieved when the database is first populated or loaded with initial data
3. Current State	The ongoing valid state of the database at any given time

2.2 Three-Schema Architecture and Data Independence

The three-schema architecture was proposed to achieve and visualize three fundamental database characteristics: the catalog (self-describing nature), data insulation (program-data/program-operation independence), and multiple user views.

2.2.1 The Three-Schema Architecture

The primary goal is to strictly separate user applications from the physical database. Schemas are defined at three distinct levels:

Figure 2.2
The three-schema architecture.

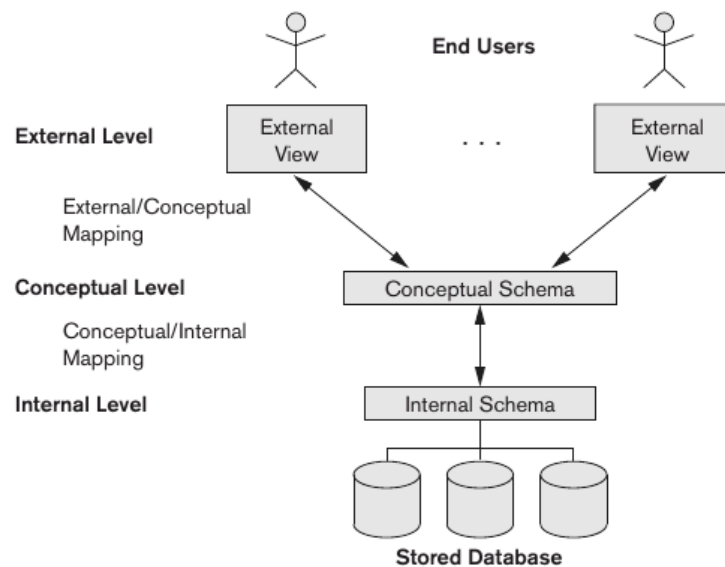


Figure 2.2: The three-schema architecture showing: External Level (External Views for End Users) mapped via External/Conceptual Mapping to the Conceptual Level (Conceptual Schema), which is mapped via Conceptual/Internal Mapping to the Internal Level (Internal Schema) that accesses the Stored Database.

Table 2.3: *The Three Levels of Schema Architecture*

Level	Name	Description	Data Model
1	Internal Level (Internal Schema)	Describes the physical storage structure of the entire database, including complete details of storage formatting and access paths (indexes, hashing)	Physical data model
2	Conceptual Level (Conceptual Schema)	Describes the logical structure of the whole database for a community of users. Completely hides physical storage details. Focuses on entities, data types, relationships, constraints, and user operations.	Representational data model (e.g., relational)
3	External/View Level (External Schemas)	Includes a number of external schemas (user views). Each describes only the specific part of the database that a particular user group is interested in, hiding the rest.	Same representational model as conceptual

Mappings

Remember, schemas are just descriptions; the actual data resides solely at the physical/internal level.

- The DBMS must transform a request from an **external schema** into a request on the **conceptual schema**, and finally into a request on the **internal schema**.
- If data is retrieved, it must be reformatted back up the chain to match the user's view.
- These transformation processes are called **mappings**.

Real-World Note

Most DBMSs do **not** separate the three levels completely and explicitly. However, the three-level ANSI architecture remains highly applicable in database design because it clearly visualizes the separation of user views, overall logic, and physical storage.

2.2.2 Data Independence

Data Independence

The capacity to change the schema at one level of a database system without having to change the schema at the next higher level. The DBMS catalog facilitates this by storing the mapping information; when a lower level changes, only the mapping needs updating.

Table 2.4: *Types of Data Independence*

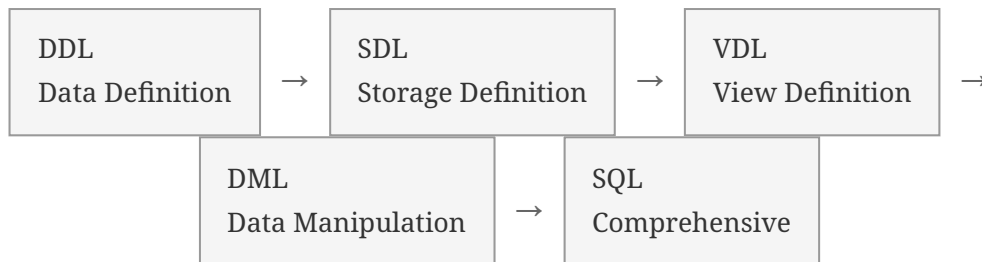
Type	Definition	Why Change?	Effect
Logical Data Independence	Capacity to change the conceptual schema without having to change external schemas or application programs	Expanding/reducing the database, adding new record types or data items, altering logical constraints	Only view definitions and mappings need to change. External schemas not referring to removed items remain unaffected.
Physical Data Independence	Capacity to change the internal schema without having to change the conceptual schema (and consequently, the external schemas)	Reorganizing physical files to improve retrieval/update performance (e.g., adding an index, changing hardware placement, compressing records)	The conceptual schema remains completely unaware. A query to "list all sections" runs exactly the same, just faster.

Exam Key Point

Logical data independence is generally considered *harder to achieve* because structural and constraint changes must not break the logic of existing application programs. **Physical data independence** exists in most modern databases and file environments, as physical details are broadly hidden from the user.

2.3 DBMS Languages

To define, construct, and manipulate the schemas within the three-schema architecture, a DBMS relies on specific languages.



1. Schema Definition Languages

Table 2.5: *Schema Definition Languages*

Language	Purpose	Modern Context
DDL (Data Definition Language)	Used by DBA and designers to define schemas. A DDL compiler processes statements and stores schema description (meta-data) in the DBMS catalog.	If levels are not strictly separated, DDL defines both conceptual and internal schemas.
SDL (Storage Definition Language)	Used specifically to define the internal schema in systems where strict separation is maintained.	In most relational DBMSs today, SDL does not exist as a separate language. Internal schema details are handled via functions, parameters, and storage mapping specifications controlled by the DBA staff.
VDL (View Definition Language)	Required in a true three-schema architecture to specify external schemas (user views) and their mappings to the conceptual schema.	Most DBMSs use DDL to define both conceptual and external schemas. In relational databases, SQL fulfills the role of VDL by allowing the creation of views as results of predefined queries.

2. Data Manipulation Language (DML)

Table 2.6: *Types of DML*

Type	Characteristics	Example
High-Level or Nonprocedural DML	Can be used interactively or embedded in a programming language; set-at-a-time ; declarative (specifies <i>what</i> data, not <i>how</i> to retrieve it)	SQL
Low-Level or Procedural DML	Must be embedded within a general-purpose programming language; record-at-a-time ; requires looping constructs to process sets	Embedded database commands in C++, Java

3. Comprehensive Integrated Languages & Contexts

- **Comprehensive Languages:** Modern systems combine DDL, VDL, DML, constraint specifications, and schema evolution into one integrated language (e.g., SQL).
- **Host Language vs. Data Sublanguage:** When DML commands are embedded in a general-purpose programming language (like Java or C++), the programming language is the **host language**, and the DML is the **data sublanguage**.
- **Query Language:** Refers to a high-level DML used in a standalone, interactive manner. Though "query" implies retrieval, these languages generally handle both retrieval and interactive updates.

2.4 Database System Environment

A DBMS is a complex software system that must seamlessly integrate with the computer's Operating System (OS) and other system software. The environment can be broadly divided into user/interface modules and internal storage/execution modules.

2.4.1 DBMS Component Modules

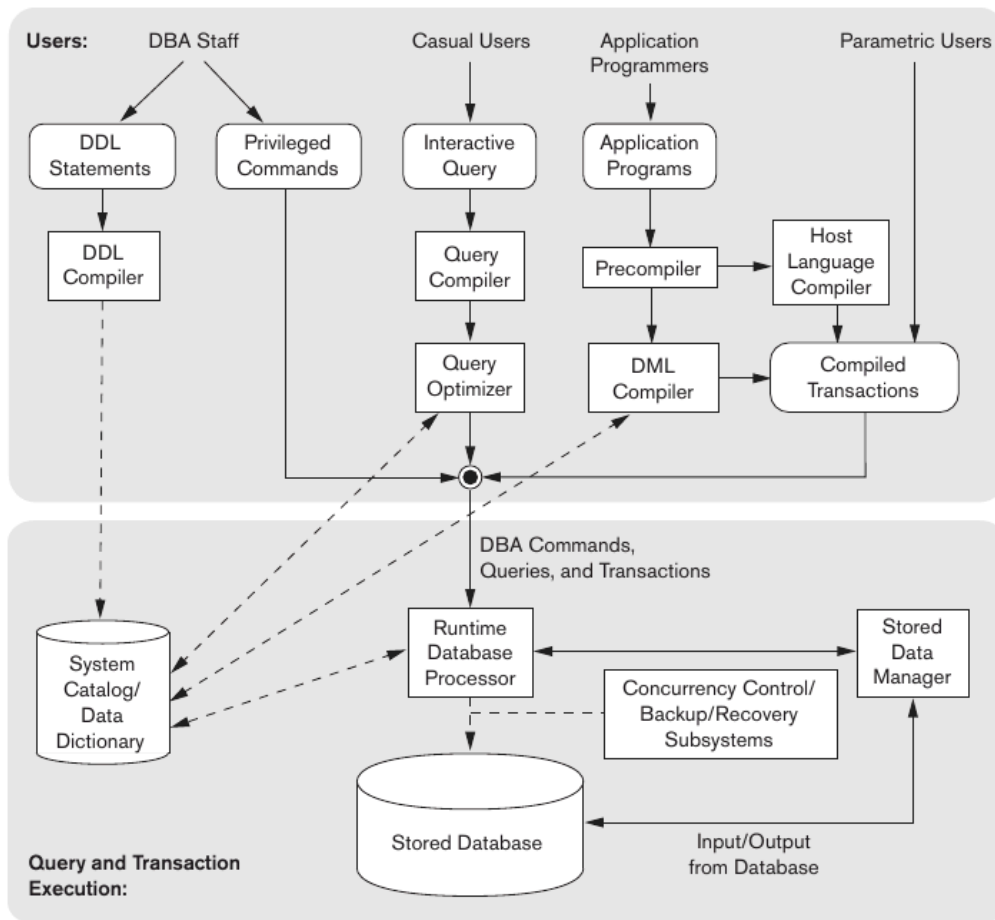


Figure 2.3
Component modules of a DBMS and their interactions.

Figure 2.3: Component modules of a DBMS and their interactions. Top half: User interfaces and compilers for DBA Staff, Casual Users, Application Programmers, and Parametric Users. Bottom half: Execution and transaction processing including Runtime Database Processor, System Catalog, Stored Data Manager, and Concurrency Control/Backup/Recovery Subsystems.

I. Storage and Buffer Management

- **OS Interaction:** The OS primarily controls disk access by scheduling read/write operations.
- **Buffer Management Module:** Many DBMSs bypass standard OS buffering and utilize their own specialized buffer management module to schedule disk read/writes optimally in main memory.
- **Stored Data Manager:** A high-level DBMS module that controls all access to DBMS information stored on the disk (both the user database and the meta-data catalog).

II. User Interfaces and Compilers (Top Half)

Table 2.7: DBMS User Types and Their Interfaces

User Type	Interface	Processing Path
DBA Staff	DDL Statements, Privileged Commands	DDL Compiler processes schema definitions and stores meta-data in the DBMS Catalog
Casual Users	Interactive query interface (menus, forms, mobile apps)	Query Compiler parses and validates syntax, checks names against catalog; Query Optimizer rearranges operations and selects efficient search algorithms
Application Programmers	Host languages (Java, C, C++, PHP, Python) with embedded DML	Precompiler extracts DML commands; DML Compiler compiles to object code; Host Language Compiler compiles the rest; linked together as Compiled Transactions
Parametric Users	PCs or mobile apps for day-to-day data entry	Execute Canned Transactions created by programmers, supplying runtime parameters (e.g., account number, amount)

III. Execution and Transaction Processing (Bottom Half)

- **Runtime Database Processor:** The core execution module. Executes privileged commands from DBA, executable query plans from Query Optimizer, and canned transactions with runtime parameters from Parametric Users.
- **Transaction Management Modules:** Integrated into the runtime processor:
 - **Concurrency Control Subsystem**
 - **Backup/Recovery Subsystem**

2.4.2 Distributed / Client-Server Implementations

- **Client Computer:** Runs the user-facing client software, interfaces, and apps.
- **Application Server (Middle Tier):** Often sits between the client and the database, processing application logic.
- **Database Server:** The dedicated machine where the database, catalog, and core DBMS processing engines reside.

2.6 Classification of Database Management Systems

DBMSs can be classified based on several different criteria. The most fundamental criterion is the **Data Model** on which the DBMS is based.

I. Classification by Data Model (Primary Criterion)

Table 2.8: *DBMS Classification by Data Model*

Data Model	Description	Notes
1. Relational (SQL Systems)	Database as a collection of tables	Main model used in most current commercial DBMSs; uses SQL
2. Object Data Model	Database in terms of objects, properties, and operations (methods)	Objects with same structure/behavior belong to a class; lacks widespread general use
3. Object-Relational (Extended Relational)	Hybrid from relational DBMSs evolving to incorporate object concepts	E.g., adding behavior to tables
4. Big Data (NOSQL)	For massively distributed data environments; four common types: Key-Value, Document (JSON), Graph, Column-Based	Key-Value: unique key with each value for fast access. Document: JSON-based. Graph: nodes and directed edges. Column-Based: columns clustered on disk for fast access
5. XML Model	Standard for exchanging data over the Web; hierarchical tree structure using elements and tags	Native XML DBMSs are experimental; many relational DBMSs include XML interfaces
6. Legacy Models	Older, historically important models	Hierarchical: Tree structures; IMS system (1965-1985) used DL/1 language. Network: Record types with 1:N "set type" relationships using pointers; CODASYL DBTG model (1971) with COBOL

II. Classification by Number of Users

- **Single-user Systems:** Support only one user at a time (most commonly used with personal computers)
- **Multiuser Systems:** Support concurrent multiple users; the vast majority of modern DBMSs

III. Classification by Number of Sites (Distribution)

Table 2.9: *DBMS Classification by Distribution*

Type	Description
Centralized DBMS	DBMS software and database reside completely at a single computer site (can still support multiple users)
Distributed DBMS (DDBMS)	Database and DBMS software distributed over many distinct sites connected by a computer network
Massively Distributed (Big Data)	Spans hundreds of sites; data frequently replicated across multiple sites for high availability
Homogeneous DDBMS	Uses the exact same DBMS software at all sites
Heterogeneous DDBMS	Uses different DBMS software at various sites
Federated DBMS (Multidatabase System)	A type of heterogeneous DDBMS where middleware software accesses autonomous, preexisting databases; loosely coupled with high local autonomy

IV. Classification by Cost and Licensing

- **Open Source (Free):** e.g., MySQL, PostgreSQL
- **Personal/Trial Versions:** Desktop single-user versions (like MS Access) or 30-day trials
- **Enterprise/Giant Systems:** Sold in highly modular format
 - **Site Licenses:** Unlimited use at a specific customer site
 - **Seat Licenses:** Limit access to specified number of concurrent users
 - **Add-ons:** Data warehousing and data mining incur extra costs

V. Classification by Access Path Options

DBMSs can be classified by the specific physical storage structures they rely on (e.g., a family of DBMSs based heavily on inverted file structures).

VI. Classification by Purpose

- **General Purpose:** Standard DBMSs designed to meet a wide variety of organizational needs
- **Special Purpose:** Designed exclusively for a very specific application where extreme performance is required (e.g., legacy airline reservation systems or telephone directory routing). Usually act as OLTP systems. Cannot be repurposed without major architectural changes.

Exam Q&A Section

This section contains practice questions and answers to help you prepare for your DBMS examination. Review these after studying the notes above.

Chapter 1 Questions

Q1: What are the three implicit properties of a database?

Answer: (1) **Representation of the Real World** -- represents a specific aspect of the real world (miniworld/UoD) and changes must be reflected promptly; (2) **Logical Coherence** -- a logically coherent collection with inherent, non-random meaning; (3) **Specific Purpose and Audience** -- designed for a specific purpose with intended users and applications.

Q2: Define Data, Database, DBMS, and Database System.

Answer: **Data** = known facts that can be recorded with implicit meaning. **Database** = a collection of related data with implicit meaning. **DBMS** = a computerized, general-purpose software system enabling users to create and maintain a database. **Database System** = the comprehensive system of the Database plus the DBMS software.

Q3: What are the four primary processes facilitated by a DBMS?

Answer: (1) **Defining** -- specifying data types, structures, and constraints (stored as meta-data in the catalog); (2) **Constructing** -- storing data on a storage medium; (3) **Manipulating** -- querying, updating, and reporting; (4) **Sharing** -- allowing multiple users and applications simultaneous access.

Q4: What is the difference between a query and a transaction?

Answer: A **query** typically causes data to be retrieved only. A **transaction** is an executing program that includes one or more database accesses and may cause data to be both read and written/modified.

Q5: Name and describe the five tables in the UNIVERSITY database example.

Answer: (1) **STUDENT** -- Name, Student_number, Class, Major; (2) **COURSE** -- Course_name, Course_number, Credit_hours, Department; (3) **SECTION** -- Section_identifier, Course_number, Semester, Year, Instructor; (4) **GRADE_REPORT** -- Student_number, Section_identifier, Grade; (5) **PREREQUISITE** -- Course_number, Prerequisite_number.

Q6: What are the four main characteristics that distinguish the database approach from file processing?

Answer: (1) **Self-Describing Nature** -- contains both data and complete definition/description in the DBMS catalog; (2) **Insulation between Programs and Data / Data Abstraction** -- program-data independence and hiding of low-level details; (3) **Support of Multiple Views** -- customized perspectives for different user groups; (4) **Sharing of Data and Multiuser Transaction Processing** -- concurrency control and transaction properties.

Q7: What is the DBMS Catalog and what does it store?

Answer: The **DBMS Catalog** is where the database definition is stored. It contains **meta-data** (data about data), including file structures, data types, storage formats, and constraints. It enables the DBMS to be general-purpose -- the same software can work with different databases by referring to the catalog.

Q8: Explain Program-Data Independence.

Answer: Changes to the structure of a data file (e.g., adding a Birth_date field) do not require changing the application programs that access the file. The DBMS updates the catalog, and programs continue to work seamlessly. In traditional file processing, file structures are embedded within programs, so changing the file means rewriting all related programs.

Q9: What is a "view" in the context of databases?

Answer: A **view** is a customized perspective of the database. It can be a subset of the explicit database, or it can contain **virtual data** (data derived from files but not explicitly stored). Users do not need to know if data is stored or derived.

Q10: Define Concurrency Control and give an example.

Answer: Concurrency Control is software within the DBMS that ensures multiple users trying to update the same data do so in a controlled manner, guaranteeing correct results. Example: Airline seat reservation -- the DBMS ensures only one agent can assign a specific seat at a time.

Q11: Explain the Isolation and Atomicity properties of transactions.

Answer: Isolation: Each transaction appears to execute entirely in isolation, even if hundreds of others are executing concurrently. **Atomicity:** The "all-or-nothing" rule -- either all database operations within a transaction are completed successfully, or none of them are.

Q12: Who are the "Actors on the Scene" and what are their roles?

Answer: (1) **DBA** -- chief administrator, authorizes access, coordinates use, acquires resources, accountable for issues; (2) **Database Designers** -- identify data, choose structures, develop views for user groups; (3) **End Users** -- casual, naive/parametric, sophisticated, and standalone users; (4) **System Analysts & Programmers** -- determine requirements, implement canned transactions.

Q13: Describe the four types of End Users.

Answer: (1) **Casual** -- occasional access, different info each time, use sophisticated interfaces; (2) **Naive/Parametric** -- large portion, use canned transactions via apps, minimal DBMS knowledge (bank tellers, reservation agents); (3) **Sophisticated** -- thoroughly familiar with DBMS, implement custom applications (engineers, analysts); (4) **Standalone** -- maintain personal databases with ready-made software.

Q14: Who are the "Workers behind the Scene"?

Answer: (1) **DBMS System Designers and Implementers** -- build the DBMS software modules; (2) **Tool Developers** -- create optional enhancement tools (modeling, monitoring, etc.); (3) **Operators and Maintenance Personnel** -- day-to-day running of hardware and software environment. They are NOT interested in the database content itself.

Q15: What is controlled redundancy and how does it differ from uncontrolled redundancy?

Answer: **Uncontrolled redundancy** (file processing) leads to duplication of effort, wasted storage, and inconsistencies. **Controlled redundancy** (denormalization) is intentionally used in databases to improve query performance, but the DBMS controls it via integrity checks to prevent errors (e.g., verifying that Name/Student_number combos in a redundant grade report match the master STUDENT file).

Q16: What is the Impedance Mismatch Problem and how is it solved?

Answer: The **Impedance Mismatch Problem** occurs because traditional DBMSs and programming languages have incompatible data structures -- developers must write complex code to convert objects to flat file formats and back. **Object-Oriented (OO) DBMSs** solve this by offering data structure compatibility with languages like C++ and Java, allowing complex program objects to be directly stored as **persistent objects**.

Q17: List five advantages of using the DBMS approach.

Answer: (1) Controlling redundancy; (2) Restricting unauthorized access; (3) Persistent storage for program objects; (4) Storage structures and search techniques for query processing; (5) Backup and recovery; (6) Multiple user interfaces; (7) Representing complex relationships; (8) Enforcing integrity constraints; (9) Inferencing and actions using rules/triggers; (10) Reduced application development time (1/6th to 1/4th of file system time).

Q18: What are the circumstances where a DBMS should NOT be used?

Answer: (1) Application is simple, well-defined, and expected to never change; (2) Stringent real-time requirements that DBMS overhead would fail to meet; (3) Embedded system with highly limited storage; (4) System strictly requires no multiple-user access. Examples: CAD tools, communication/switching systems (AT&T), GIS.

Chapter 2 Questions

Q19: What is a Data Model and what does it describe?

Answer: A **Data Model** is a collection of concepts used to describe: (1) the structure of a database (data types, relationships, constraints), (2) basic operations (insert, delete, modify, retrieve), and (3) dynamic behavior (user-defined operations like COMPUTE_GPA on a STUDENT object).

Q20: Name and describe the five categories of data models.

Answer: (1) **High-Level/Conceptual** -- close to human perception (entities, attributes, relationships); (2) **Representational/Implementation** -- between conceptual and physical, directly implementable (relational, network, hierarchical); (3) **Object** -- higher-level implementation closer to conceptual (ODMG standard); (4) **Low-Level/Physical** -- precise storage details for specialists; (5) **Self-Describing** -- description and data in one structure (XML, NOSQL).

Q21: What is the difference between a Database Schema and a Database State?

Answer: The **Database Schema (Intension)** is the formal description of the database specified during design; it doesn't change frequently. The **Database State (Extension/Snapshot)** is the actual data populated at any particular moment; it changes frequently with every insert, delete, or update.

Q22: What is a Schema Diagram and what are its limitations?

Answer: A **Schema Diagram** is a visual representation of a schema. It displays record names, data items, and some constraints, but rarely shows detailed data types, complex relationships, or complex business logic (e.g., "Computer Science majors must take CS1310 by sophomore year").

Q23: Describe the Three-Schema Architecture and its three levels.

Answer: The architecture separates user applications from the physical database: (1) **Internal Level** -- physical storage structure, access paths, storage formatting; (2) **Conceptual Level** -- logical structure for all users, hides physical details; (3) **External/View Level** -- user-specific views showing only relevant parts of the database.

Q24: What are mappings in the three-schema architecture?

Answer: **Mappings** are the transformation processes: a request from an external schema is transformed into a request on the conceptual schema, and finally into a request on the internal schema. If data is retrieved, it is reformatted back up the chain to match the user's view. Schemas are just descriptions; actual data resides only at the physical/internal level.

Q25: Define Logical and Physical Data Independence. Which is harder to achieve?

Answer: **Logical Data Independence:** Change the conceptual schema without changing external schemas or application programs (needed when expanding/reducing the database). **Physical Data Independence:** Change the internal schema without changing the conceptual schema (needed when reorganizing physical files, adding indexes). **Logical data independence is harder to achieve** because structural changes must not break existing application logic.

Q26: What are DDL, SDL, and VDL? How are they used in modern systems?

Answer: **DDL** (Data Definition Language) defines schemas; a DDL compiler stores meta-data in the catalog. **SDL** (Storage Definition Language) defines internal schemas in systems with strict separation -- rarely exists separately in modern relational DBMSs. **VDL** (View Definition Language) specifies external schemas -- in relational systems, SQL fulfills this role by allowing view creation. Most modern systems use SQL as a comprehensive language combining all three.

Q27: Differentiate between High-Level and Low-Level DML.

Answer: **High-Level (Nonprocedural):** Set-at-a-time, declarative (specifies what data, not how), can be used interactively or embedded, e.g., SQL. **Low-Level (Procedural):** Record-at-a-time, must be embedded in a host language, requires looping constructs to process sets, instructs the system how to retrieve data.

Q28: What is the difference between a Host Language and a Data Sublanguage?

Answer: When DML commands are embedded in a general-purpose programming language (like Java or C++), the programming language is the **host language**, and the DML is the **data sublanguage**.

Q29: Describe the DBMS Component Modules and their interactions.

Answer: The architecture has two halves: **Top (User Interfaces & Compilers):** DBA Staff (DDL Compiler), Casual Users (Query Compiler + Optimizer), Application Programmers (Precompiler + DML Compiler + Host Language Compiler), Parametric Users (Compiled Transactions). **Bottom (Execution & Transaction Processing):** Runtime Database Processor executes commands, works with System Catalog and Stored Data Manager, integrated with Concurrency Control and Backup/Recovery subsystems.

Q30: What is the role of the Query Optimizer?

Answer: The **Query Optimizer** takes the internal query from the Query Compiler and optimizes it for execution. It rearranges operations, eliminates redundancies, and selects efficient search algorithms by consulting statistical and physical information from the system catalog. It outputs executable code.

Q31: How are DBMSs classified by data model?

Answer: (1) Relational (SQL), (2) Object, (3) Object-Relational, (4) Big Data/NOSQL (Key-Value, Document, Graph, Column-Based), (5) XML, (6) Legacy (Hierarchical -- IMS/DL/1, Network -- CODASYL DBTG/COBOL).

Q32: What is the difference between a Centralized DBMS and a Distributed DBMS (DDBMS)?

Answer: A **Centralized DBMS** has both the DBMS software and the database at a single computer site. A **Distributed DBMS (DDBMS)** distributes the database and DBMS software over many distinct sites connected by a network. DDBMS can be homogeneous (same DBMS at all sites) or heterogeneous (different DBMSs).

Q33: What is a Federated DBMS?

Answer: A **Federated DBMS (Multidatabase System)** is a type of heterogeneous DDBMS where middleware software accesses autonomous, preexisting databases. The participating systems are loosely coupled and maintain a high degree of local autonomy.

Q34: What is the difference between a Site License and a Seat License?

Answer: A **Site License** allows unlimited use of the database system with any number of copies running at a specific customer site. A **Seat License** limits access to a specified number of concurrent users or "seats."

Q35: Define: Entity, Attribute, Relationship in the context of conceptual data models.

Answer:Entity: A real-world object or concept (e.g., an employee, a project).

Attribute: A property of interest describing an entity (e.g., name, salary).

Relationship: An association among two or more entities (e.g., a "works-on" relationship).

Q36: What are the NOSQL data models and their characteristics?

Answer: (1) **Key-Value:** Associates a unique key with each value for extremely fast access; (2) **Document:** Based on JSON, stores data as documents resembling complex objects; (3) **Graph:** Stores objects as nodes and relationships as directed edges; (4) **Column-Based:** Stores columns of rows clustered on disk for fast access, allows multiple versions.

Q37: What is schema evolution and why is it important?

Answer:Schema evolution is applying changes to a live schema (e.g., adding a Date_of_birth field to STUDENT). It is important because application requirements shift over time. Most modern DBMSs allow these operations while the database remains operational.

Q38: Explain the Stored Data Manager and Buffer Management Module.

Answer: The **Stored Data Manager** is a high-level DBMS module controlling all access to information stored on disk (user database and catalog). The **Buffer Management Module** schedules disk read/writes optimally in main memory; many DBMSs bypass standard OS buffering to run their own module optimized for database performance.

DATABASE MANAGEMENT SYSTEMS

Unit 2

High-Level Conceptual Data Models & Database Design

Topics Covered:

Database Design Process | ER Model | Entities & Attributes
Relationships & Constraints | Weak Entities
ER Diagrams | Ternary vs Binary Relationships

Comprehensive Exam Notes

with Diagrams & Q/A Section

Table of Contents

1. The Database Design Process

- 1.1 Phase 1: Requirements Collection and Analysis
- 1.2 Phase 2: Conceptual Design
- 1.3 Phase 3: Logical Design (Data Model Mapping)
- 1.4 Phase 4: Physical Design & Implementation

2. Sample Database Application: The COMPANY Database

3. Core ER Model Concepts: Entities and Attributes

- 3.1 Entities and Attributes
- 3.2 Entity Types, Entity Sets, Keys, and Value Sets
- 3.3 Initial Conceptual Design of the COMPANY Database
- 3.4 Relationship Types, Relationship Sets, Roles, and Constraints
- 3.5 Weak Entity Types
- 3.6 Refining the ER Design for the COMPANY Database
- 3.7 ER Diagrams, Naming Conventions, and Design Issues
- 3.8 Choosing Between Binary and Ternary Relationships

4. Visual Summary: ER Diagram Notation & Complete Diagram

5. Exam Questions & Answers

- 5.1 Short Answer Questions
- 5.2 Long Answer Questions
- 5.3 MCQ-style Quick Review
- 5.4 Key Formulas & Definitions Summary

1. The Database Design Process

The creation of a database involves a systematic design process that translates real-world requirements into a physical, functioning database. This process is divided into **two parallel pathways**: one focusing on **data** and the other on **functionality** (transactions/operations).

Key Insight

The database design process has two parallel tracks: the **data track** (Requirements → Conceptual Design → Logical Design → Physical Design) and the **functionality track** (Functional Analysis → Application Program Design → Transaction Implementation).

- **Outputs:**

- **Data Requirements:** A concisely written set of rules and data needs.
- **Functional Requirements:** User-defined operations (retrievals and updates) that will be applied to the database. These are often specified using software engineering techniques like data flow diagrams, sequence diagrams, and scenarios.

1.2 Phase 2: Conceptual Design

- **Objective:** Create a conceptual schema using a high-level conceptual data model (like the ER model).
- **Characteristics:**
 - Provides a concise description of data requirements (entity types, relationships, constraints).
 - **DBMS-Independent:** It does not include implementation details, making it accessible for non-technical users to review and confirm requirements.
 - Serves as a reference to ensure requirements do not conflict.
- **Verification:** High-level user queries identified in the functional analysis are tested against this schema to ensure all data needed for those transactions is present.

1.3 Phase 3: Logical Design (Data Model Mapping)

- **Objective:** Transform the high-level conceptual schema into an implementation data model (e.g., the relational/SQL model).
- **Characteristics:**
 - **DBMS-Specific:** The design is tailored to the specific type of commercial DBMS being used.
 - Results in a **Logical (Conceptual) Schema**.
 - Often automated or semi-automated by database design tools.

1.4 Phase 4: Physical Design & Implementation

- **Physical Design:** Specifies internal storage structures, file organizations, indexes, access paths, and physical parameters. Results in the **Internal Schema**.
- **Application Program Design:** Parallels physical design. It transforms high-level transaction specifications into application programs and implements database transactions.

EXAM TIP

Remember the **four phases in order**: Requirements → Conceptual → Logical → Physical. Also remember that Conceptual design is **DBMS-independent** while Logical design is **DBMS-specific**.

Table 1: Comparison of Database Design Phases

Phase	Output	DBMS Dependence	Key Activity
Requirements	Data + Functional Requirements	Independent	Interview users
Conceptual Design	Conceptual Schema (ER Model)	Independent	Create ER diagram
Logical Design	Logical Schema (Relational)	Specific	Map ER to tables
Physical Design	Internal Schema	Specific	Storage structures

2. Sample Database Application: The COMPANY Database

To illustrate database modeling, consider a sample miniworld representing a company. The requirements collected dictate the following rules:

1. Departments:

- Organizes the company.
- **Attributes:** Unique Name, Unique Number.
- **Management:** A specific employee manages it; must track the `Start_date` of management.
- **Locations:** Can have several physical locations.

2. Projects:

- Controlled by a department.
- **Attributes:** Unique Name, Unique Number, Single Location.

3. Employees:

- **Attributes:** Name, Social Security Number (SSN), Address, Salary, Sex (Gender), Birth Date.
- **Assignments:** Assigned to one primary department but can work on multiple projects (across different departments).
- **Tracking:** Must record hours per week worked on each project.

- **Supervision:** Must track the direct supervisor of each employee (who is also an employee).

4. Dependents:

- Tracked for insurance purposes.
- **Attributes:** First Name, Sex, Birth Date, Relationship to the employee.

REMEMBER FOR EXAMS

The **COMPANY database** is the standard example used throughout this unit. Know the four entity types (DEPARTMENT, PROJECT, EMPLOYEE, DEPENDENT) and their attributes/relationships thoroughly. Most exam questions will reference this schema.

3. Core ER Model Concepts: Entities and Attributes

The ER model describes the world in terms of **entities**, **relationships**, and **attributes**.

3.1 Entities and Attributes

Entities

A **thing or object** in the real world with an independent existence.

- **Physical Existence:** A person, car, house.
- **Conceptual Existence:** A company, a job, a university course.

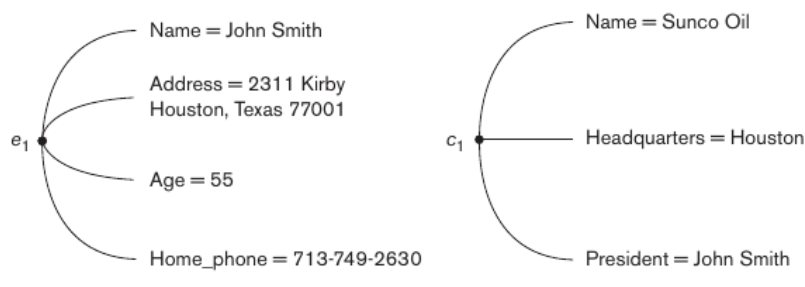


Figure 3.3
Two entities,
EMPLOYEE e_1 , and
COMPANY c_1 , and
their attributes.

Figure 3.3: Two entities, EMPLOYEE e_1 and COMPANY c_1 , and their attributes. An entity is represented as a point connected to its attribute values.

Attributes

Particular properties that describe an entity. Each entity instance has specific values for its attributes.

Classification of Attributes

Table 2: Classification of Attributes

Type	Description	Example
Simple (Atomic)	Indivisible	Age, Sex
Composite	Can be divided into smaller subparts with independent meanings	Address → Street, City, State, Zip
Single-Valued	Holds exactly one value	Age
Multivalued	Can hold a set of values	Colors of a car
Stored	Directly stored in the database	Birth_date
Derived	Computed from other attributes	Age (derived from Birth_date)

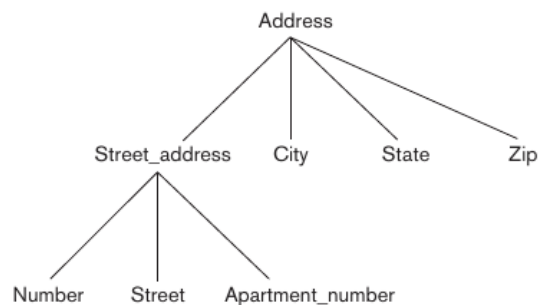


Figure 3.4
A hierarchy of composite attributes.

Figure 3.4: A hierarchy of composite attributes. Address is composed of Street_address, City, State, and Zip. Street_address is further composed of Number, Street, and Apartment_number.

Complex Attributes

Composite and multivalued attributes can be nested arbitrarily. Example notation:

```

{Address_phone( {Phone(Area_code, Phone_number)},
Address(Street_address(Number, Street, Apartment_number), City, State, Zip)
)}
  
```

{Address_phone({Phone(Area_code,Phone_number)},Address(Street_address
(Number,Street,Apartment_number),City,State,Zip))}

Figure 3.5
A complex attribute:
Address_phone.

Figure 3.5: A complex attribute: Address_phone, showing how composite and multivalued attributes can be nested.

NULL Values

Used when an attribute is:

- **Not applicable** (e.g., College_degrees for someone who didn't attend college)
- **Unknown** (either missing or existence is uncertain)

3.2 Entity Types, Entity Sets, Keys, and Value Sets

Entity Types vs. Entity Sets

Table 3: Entity Type vs Entity Set

Concept	Definition	Analogy
Entity Type	Defines a collection of entities with the same attributes. Represents the schema or intension .	The generic concept of an EMPLOYEE
Entity Set (Extension)	The collection of all entities of a particular entity type in the database at a specific point in time.	The actual list: John Smith, Fred Brown, etc.

Important

The terms "Entity Type" and "Entity Set" are often used interchangeably, but they represent **structure vs. data**. Entity Type is the definition (schema); Entity Set is the actual instances (extension).

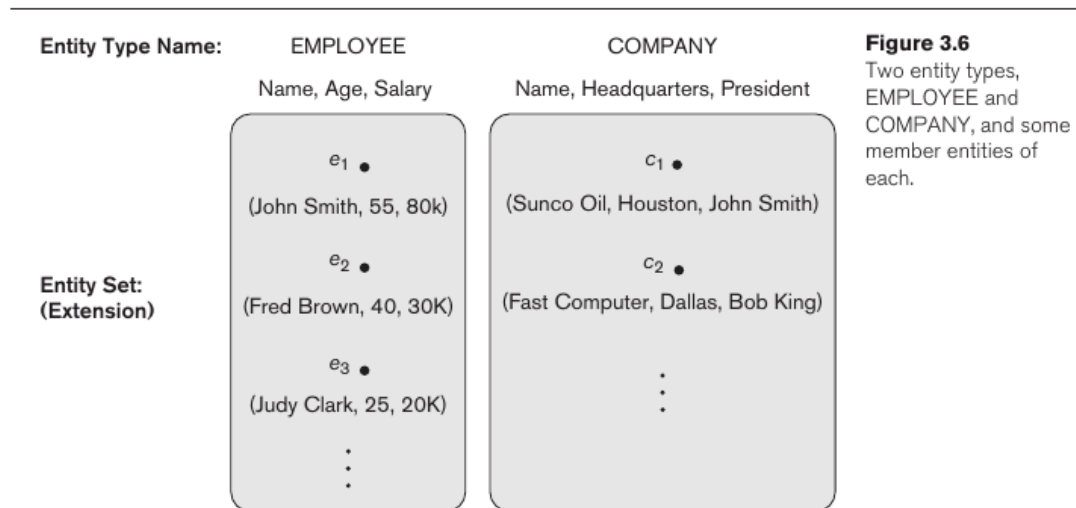


Figure 3.6: Two entity types, *EMPLOYEE* and *COMPANY*, and some member entities of each. The extension (entity set) shows actual data instances.

Key Attributes

An important constraint is the **uniqueness constraint**.

- **Key Attribute:** An attribute whose values are distinct for each individual entity in the entity set (e.g., `Ssn` for a Person, `Name` for a Company).
- **Composite Key:** Several attributes combined to form a key (e.g., a Car's Registration made of `State` and `Number`). A composite key must be **minimal** (no superfluous attributes).

Key Characteristics

- The constraint holds across **every possible entity set over time**, not just the current snapshot.
- An entity type can have **more than one key** (e.g., `Vehicle_id` and `Registration` are both keys for a Car). Each is underlined separately in the diagram.
- Unlike the relational model, there is **no strict concept of a "primary key"** at the ER conceptual stage; it is chosen later during mapping.

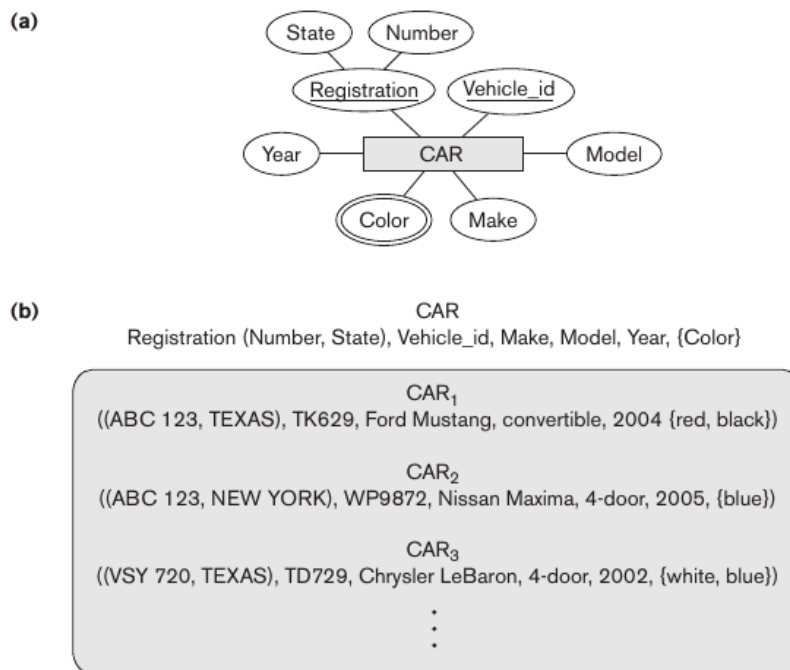


Figure 3.7
The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities.

Figure 3.7: The CAR entity type with two key attributes, Registration and Vehicle_id. (a) ER diagram notation. (b) Entity set with three entities showing how keys uniquely identify each car.

Value Sets (Domains) of Attributes

Specifies the set of values that may be assigned to a simple attribute (analogous to data types like integer, string, boolean, date in programming).

Mathematical Definition

An attribute A of entity set E whose value set is V is defined as a function from E to the power set $P(V)$ of V :

$$A : E \rightarrow P(V)$$

- The value of attribute A for entity e is $A(e)$.
- **NULL Value:** Represented by the empty set $\emptyset$.
- **Single-Valued:** $A(e)$ is restricted to being a singleton set.
- **Composite Attribute:** The value set V is the power set of the Cartesian product of the value sets of its component attributes.

3.3 Initial Conceptual Design of the COMPANY Database

will be converted into proper ER relationships in later refinement steps.)

1. **DEPARTMENT** Entity Type: Keys: Name, Number. Multivalued: Locations.
2. **PROJECT** Entity Type: Keys: Name, Number.
3. **EMPLOYEE** Entity Type: Key: Ssn. Composite: Name.
4. **DEPENDENT** Entity Type: (Relies on Employee).

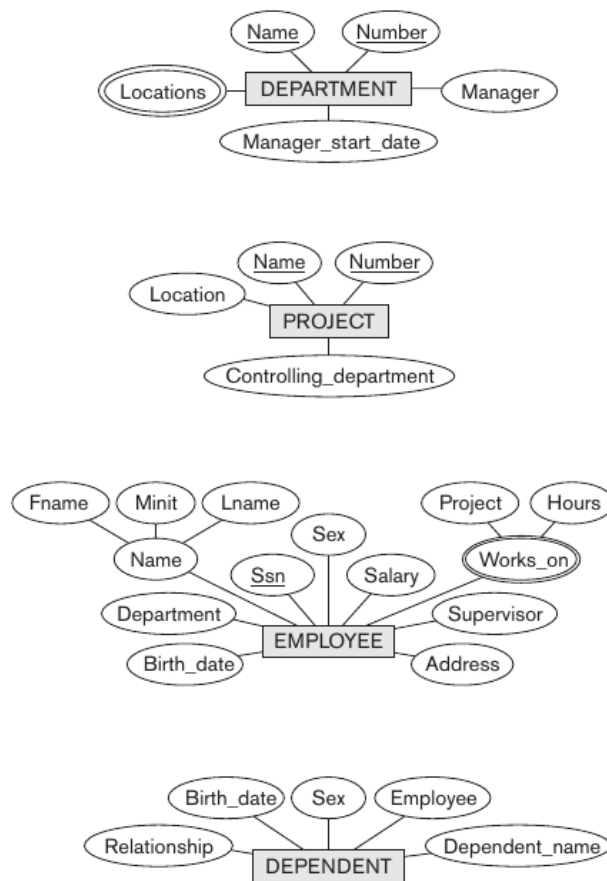


Figure 3.8
Preliminary design of
entity types for the
COMPANY database.
Some of the shown
attributes will be refined
into relationships.

Figure 3.8: Preliminary design of entity types for the COMPANY database. Some of the shown attributes will be refined into relationships.

EXAM TIP

In the initial design, attributes like **Supervisor**, **Department**, and **Manager** are shown as attributes. In the refined design (Section 3.6), these become **relationships**. Watch for this distinction in exam questions!

3.4 Relationship Types, Relationship Sets, Roles, and Constraints

- Mathematically, it is a subset of the Cartesian product of the participating entity sets: $R \subseteq E_1 \times E_2 \times \dots \times E_n$.
- Each instance $r_i = \{e_1, e_2, \dots, e_n\}$ associates individual entities, meaning exactly **one entity from each participating entity type**.

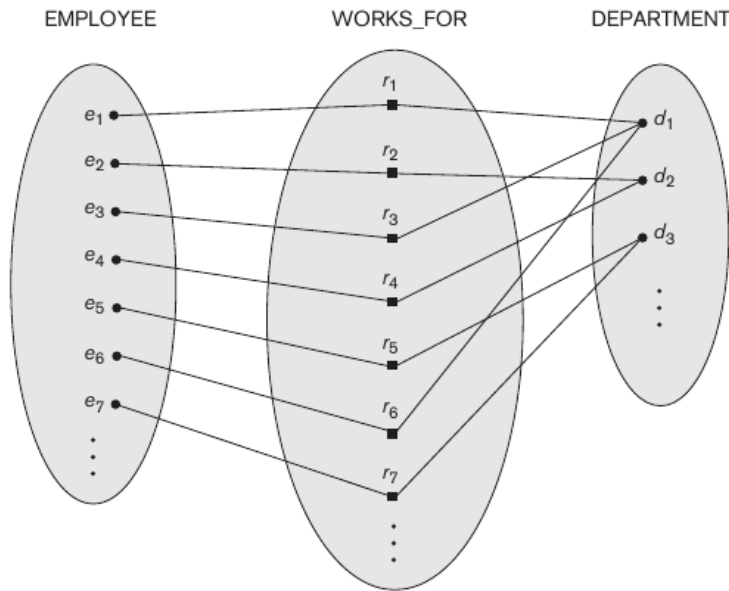


Figure 3.9
Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

Figure 3.9: Some instances in the WORKS_FOR relationship set, which represents a relationship type WORKS_FOR between EMPLOYEE and DEPARTMENT.

B. Degree of a Relationship Type

The degree is the number of participating entity types.

- **Binary:** Degree of two (most common, e.g., WORKS_FOR between EMPLOYEE and DEPARTMENT).
- **Ternary:** Degree of three (e.g., SUPPLY between SUPPLIER, PART, and PROJECT).

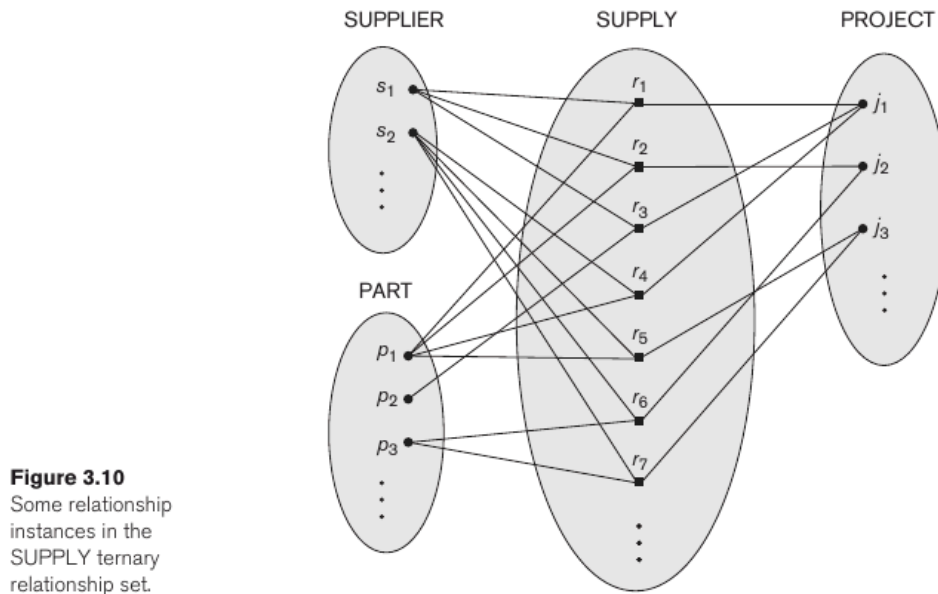


Figure 3.10: Some relationship instances in the SUPPLY ternary relationship set involving SUPPLIER, PART, and PROJECT.

C. Role Names and Recursive Relationships

- **Role Name:** Signifies the specific role a participating entity plays in a relationship instance (e.g., employee vs employer).
- **Recursive (Self-Referencing) Relationships:** Occurs when the same entity type participates more than once in a relationship type, but in different roles.
 - **Example:** The SUPERVISION relationship where the EMPLOYEE entity participates twice: once in the role of **supervisor (1)** and once as the **supervisee (2)**. Role names become absolutely essential here to distinguish meaning.

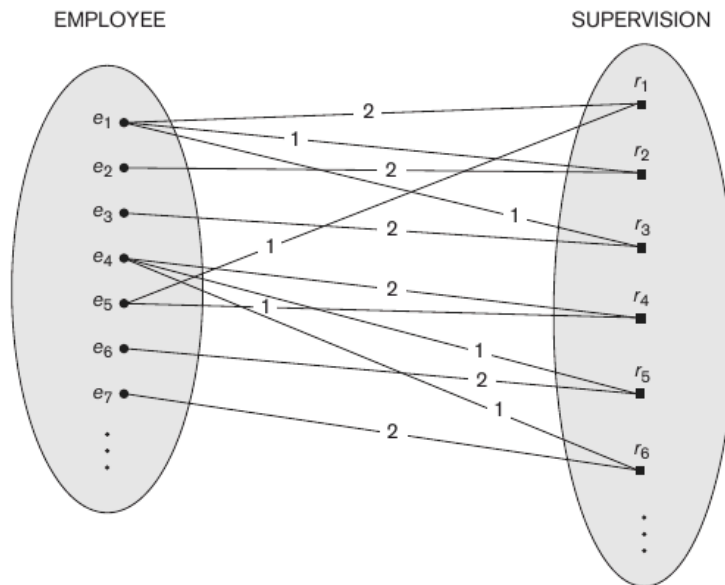


Figure 3.11
A recursive relationship
SUPERVISION
between EMPLOYEE
in the *supervisor* role (1)
and EMPLOYEE in
the *subordinate* role (2).

Figure 3.11: A recursive relationship *SUPERVISION* between *EMPLOYEE* in the *supervisor* role (1) and *EMPLOYEE* in the *subordinate* role (2). Note how the same entity set appears twice with different roles.

D. Structural Constraints on Binary Relationships

Constraints limit the possible combinations of entities in a relationship set, derived from miniworld rules. Together, **Cardinality Ratios** and **Participation Constraints** form the **Structural Constraints**.

1. Cardinality Ratios (Maximum Participation)

Specifies the maximum number of relationship instances an entity can participate in.

Table 4: Cardinality Ratio Types

Ratio	Description	Example
1:1	One entity from A relates to at most one entity from B, and vice versa	MANAGES: An employee manages at most 1 department; a department has at most 1 manager
1:N	One entity from A can relate to many entities from B, but each B relates to at most one A	WORKS_FOR: A department has N employees; an employee works for 1 department
N:1	Reverse of 1:N	Similar to 1:N but viewed from the other side
M:N	Entities from A can relate to many from B, and vice versa	WORKS_ON: An employee works on N projects; a project has M employees

Figure 3.12
A 1:1 relationship,
MANAGES.

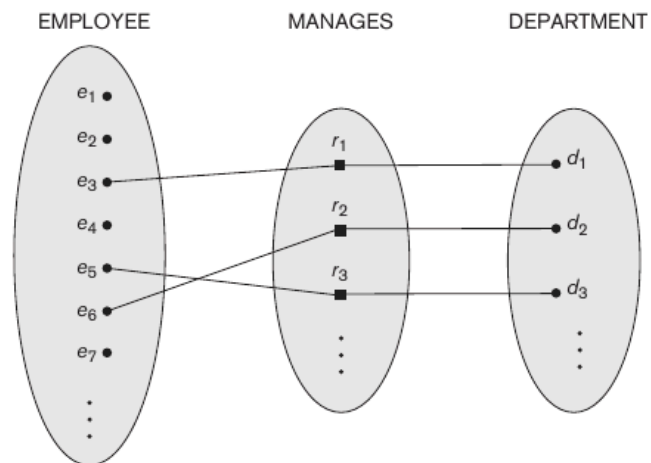


Figure 3.12: A 1:1 relationship, MANAGES. Each employee manages at most one department, and each department has at most one manager.

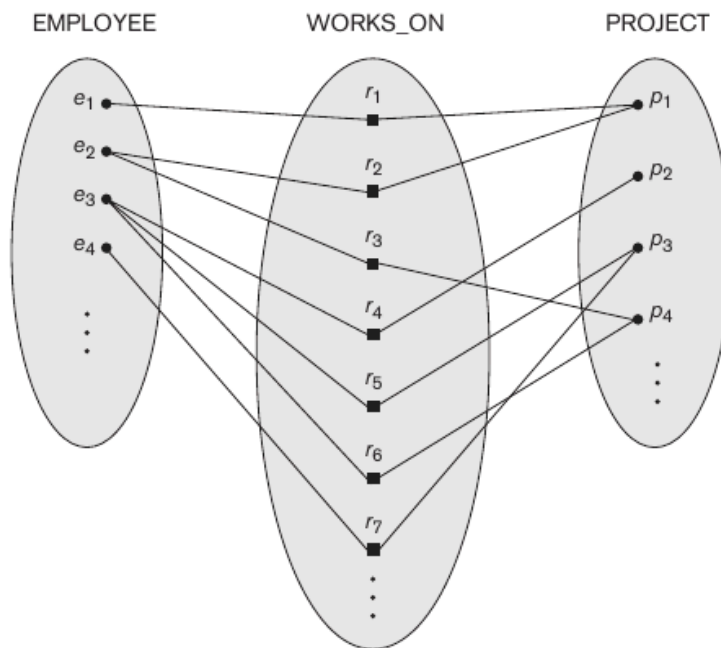


Figure 3.13
An M:N relationship,
WORKS_ON.

Figure 3.13: An M:N relationship, WORKS_ON. An employee can work on multiple projects, and a project can have multiple employees.

2. Participation Constraints / Existence Dependencies (Minimum Participation)

Specifies the minimum number of instances an entity can participate in (whether its existence depends on the relationship).

- **Total Participation (Existence Dependency):** An entity can exist only if it participates in at least one relationship instance (minimum cardinality = 1).
 - **Example:** If every employee **MUST** work for a department, EMPLOYEE has **total participation** in WORKS_FOR.

- **Partial Participation:** An entity does not have to participate in the relationship (minimum cardinality = 0).
 - **Example:** Not every employee is a manager, so EMPLOYEE has **partial participation** in MANAGES.

Visual Notation Summary

- **Single Line** connecting entity to relationship = Partial Participation
- **Double Line** connecting entity to relationship = Total Participation (Existence Dependency)
- **1, M, N** on the lines = Cardinality Ratios

E. Attributes of Relationship Types

Relationship types can also possess their own attributes (e.g., **Hours** on the WORKS_ON relationship, or **Start_date** on the MANAGES relationship).

Migration Rules based on Cardinality:

Table 5: Attribute Migration Rules

Cardinality	Migration Rule	Example
1:1	Attributes can be migrated to either participating entity type	Start_date could belong to EMPLOYEE or DEPARTMENT
1:N	Attributes can be migrated only to the entity on the N-side	Start_date for WORKS_FOR goes to EMPLOYEE
M:N	Attributes cannot be migrated; must remain as relationship attributes	Hours on WORKS_ON stays with the relationship

Why M:N Attributes Cannot Migrate

In an M:N relationship, an attribute value depends on **the combination** of both entities. For example, "Hours" in WORKS_ON depends on both the specific employee AND the specific project. If moved to either entity alone, the information would be lost.

3.5 Weak Entity Types

Entity types that do not have key attributes of their own are called **weak entity types**. (In contrast, regular entity types with keys are called **strong entity types**.)

Key Characteristics of Weak Entities

- **Identification:** Weak entities are identified by being related to specific entities from another entity type, combined with one of their own attribute values.
- **Identifying/Owner Entity Type:** The entity type that the weak entity relies on for identification.
- **Identifying Relationship:** The relationship type that relates a weak entity to its owner.
- **Total Participation:** A weak entity type always has a **total participation constraint** (existence dependency) concerning its identifying relationship (it cannot exist without its owner).

Important Distinction

Not every existence dependency creates a weak entity type! For example, a Driver's License depends on a Person to exist, but has its own key (License Number), so it is a **strong entity**.

Partial Key

The attribute that uniquely identifies weak entities that are related to the **same owner entity** (e.g., The **Name** attribute of a DEPENDENT under a specific EMPLOYEE).

Design Considerations

Weak entity types can sometimes be represented as complex (composite/multivalued) attributes. They are typically kept as weak entity types if they:

- Participate independently in relationship types **other than** their identifying relationship.

Table 6: Strong vs Weak Entity Types

Feature	Strong Entity	Weak Entity
Own Key	Yes (Key attribute)	No (only partial key)
Identification	By own key	By owner entity + partial key
Existence	Independent	Depends on owner (total participation)
Notation	Single rectangle	Double rectangle
Relationship	Regular relationship	Identifying relationship (double diamond)

3.6 Refining the ER Design for the COMPANY Database

The initial conceptual design (Section 3.3) uses attributes to represent relationships. The next step is refining these attributes into **explicit ER relationship types**.

Explicit Relationships for the COMPANY Schema:

Table 7: COMPANY Database Relationships

Relationship	Cardinality	Participation	Attribute
MANAGES (EMPLOYEE, DEPARTMENT)	1:1	Employee: Partial; Department: Total	Start_date
WORKS_FOR (DEPARTMENT, EMPLOYEE)	1:N	Both Total	None
CONTROLS (DEPARTMENT, PROJECT)	1:N	Department: Partial; Project: Total	None
SUPERVISION (EMPLOYEE, EMPLOYEE)	1:N	Both Partial	None
WORKS_ON (EMPLOYEE, PROJECT)	M:N	Both Total	Hours
DEPENDENTS_OF (EMPLOYEE, DEPENDENT)	1:N	Employee: Partial; Dependent: Total	None

Redundancy Cleanup

Once an attribute is refined into a relationship, the original attribute **MUST be removed** from the entity type to prevent redundancy (e.g., Supervisor, Department, and Manager_start_date are removed from the entities and modeled as the relationships above).

3.7 ER Diagrams, Naming Conventions, and Design Issues

A. Summary of Notation for ER Diagrams

Table 8: ER Diagram Notation Summary

Symbol	Meaning
Rectangle Box	Regular (strong) Entity Type
Double Rectangle Box	Weak Entity Type
Diamond Box	Relationship Type
Double Diamond Box	Identifying Relationship Type (for weak entities)
Oval	Attribute
Dotted Oval	Derived Attribute
Double Oval	Multivalued Attribute
Ovals branching from Ovals	Composite Attribute
Solid Underline (in Oval)	Key Attribute
Dotted/Dashed Underline (in Oval)	Partial Key Attribute
Single Line	Partial Participation
Double Line	Total Participation (Existence Dependency)
1, M, N on Lines	Cardinality Ratios

Symbol	Meaning
	Entity
	Weak Entity
	Relationship
	Identifying Relationship
	Attribute
	Key Attribute
	Multivalued Attribute
	Composite Attribute
	Derived Attribute
	Total Participation of E_2 in R
	Cardinality Ratio 1 : N for $E_1 : E_2$ in R
	Structural Constraint (min, max) on Participation of E in R

Figure 3.14
Summary of the
notation for ER
diagrams.

Figure 3.14: Summary of the notation for ER diagrams. This is your quick reference guide for reading and drawing ER diagrams.

B. Naming Conventions

- **Entity / Relationship Names: UPPERCASE** (e.g., EMPLOYEE, WORKS_FOR). Entities should use singular nouns; relationships should use verbs.
- **Attribute Names:** Initial letter capitalized (e.g., Name, Salary). Usually derived from nouns describing the entity.
- **Role Names:** lowercase (e.g., supervisor, supervisee).
- **Readability:** Binary relationship names should ideally make the diagram readable from left to right and top to bottom.

C. Design Choices (Iterative Refinement)

Conceptual design is an **iterative process**:

D. Alternative Notations: (min, max)

Instead of using Cardinality Ratios (1:N, M:N) and Participation Lines (single/double), designers can use the **(min, max) notation** on relationship edges to specify structural constraints.

(min, max) Notation Rules

- **Format:** A pair of integers (min, max) is associated with each participation of an entity e in relationship R .
- **Constraints:** $0 \leq \min \leq \max$ and $\max \geq 1$.
- **Meaning:** Each entity e in E must participate in at least $\min$ and at most $\max$ relationship instances in R .
- **(0, *)** implies **partial participation**.
- **(1, *)** implies **total participation**.

relationship or multiple binary relationships.

A. Semantics of Ternary vs. Multiple Binary Relationships

A ternary relationship usually represents **different information** than three separate binary relationships.

Key Example: SUPPLY

- **Ternary Example:** The SUPPLY relationship relates SUPPLIER(s), PROJECT(j), and PART(p). An instance (s, j, p) explicitly means: Supplier 's' is **currently supplying part 'p' to project 'j'**.
- **Binary Counterparts:** If modeled as three binaries -- CAN_SUPPLY(s, p), USES(j, p), and SUPPLIES(s, j) -- the existence of instances (s, p), (j, p), and (s, j) **does not necessarily imply** the specific ternary instance (s, j, p).
- **Guideline:** Base the decision purely on **semantics**. The typical solution is to include the ternary relationship along with one or more of the binary relationships if they represent distinct meanings and are required by the application.

Example: OFFERS

The ternary relationship **OFFERS** associates INSTRUCTOR, SEMESTER, and COURSE.

- If **OFFERS**(i, s, c) exists, we can logically infer that **TAUGHT_DURING**(i, s) and **OFFERED_DURING**(s, c) must also exist. Thus, these binaries are **redundant** and can be omitted.
- However, a generic **CAN_TEACH**(i, c) relationship (certifying an instructor's ability) **cannot be inferred** just because a class was offered. Thus, CAN_TEACH must be explicitly kept.

Exception

Under strict constraints, binary relationships can replace a ternary one. If CAN_TEACH was strictly 1:1 (an instructor teaches only one course, and a course is taught by only one instructor), the ternary OFFERS could theoretically be inferred from the three binary relationships.

Figure 3.18
Another example of
ternary versus binary
relationship types.

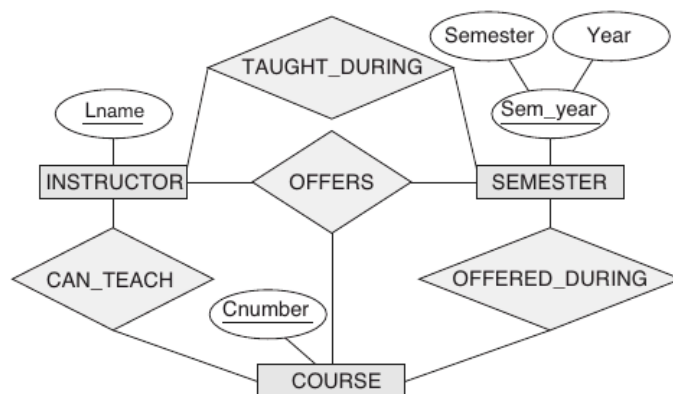


Figure 3.18: Another example of ternary versus binary relationship types. **OFFERS** is the ternary relationship; **TAUGHT_DURING** and **OFFERED_DURING** can be inferred from it and are redundant.

C. Tool Limitations (Workarounds for Binary-Only Systems)

If a database design tool strictly limits models to binary relationships, a ternary relationship (like **SUPPLY**) must be mathematically transformed:

1. **As a Weak Entity Type:** Create a weak entity (e.g., **SUPPLY**) with no partial key and three identifying relationships. The three participating entity types (**SUPPLIER**, **PART**, **PROJECT**) collectively serve as the "owner" entity types.
2. **As a Regular Entity Type (Surrogate Key):** Introduce an artificial or surrogate key (e.g., **Supply_id**) to transform it into a regular, strong entity. Then, relate it to the three entities using three

separate binary relationships.

D. Weak Entity Types with Ternary (n-ary) Identifying Relationships

A weak entity type is not restricted to having a single owner; it can have **multiple owner entity types** through an n-ary identifying relationship.

Example: INTERVIEW Weak Entity

In an employment agency database, an **INTERVIEW** could be a weak entity identified by a combination of two owners: a **CANDIDATE** and a **COMPANY**.

- The weak entity's partial key might be `Dept_date`.
- The **INTERVIEW** is only uniquely identified by knowing the candidate, the company, and the combination of the interview's date and department.

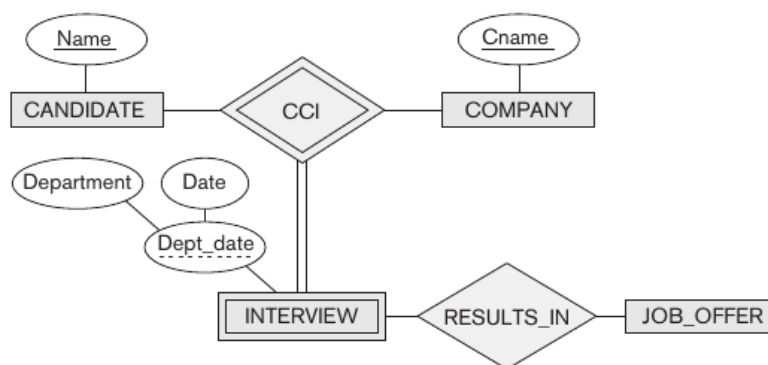


Figure 3.19
A weak entity type **INTERVIEW** with a ternary identifying relationship type **CCI**. **INTERVIEW** is identified by the combination of **CANDIDATE** and **COMPANY**, along with its partial key **Dept_date**.

Figure 3.19: A weak entity type **INTERVIEW** with a ternary identifying relationship type **CCI**. **INTERVIEW** is identified by the combination of **CANDIDATE** and **COMPANY**, along with its partial key **Dept_date**.

4. Visual Summary: ER Diagram Notation & Complete Diagram

Complete COMPANY ER Diagram (Refined)

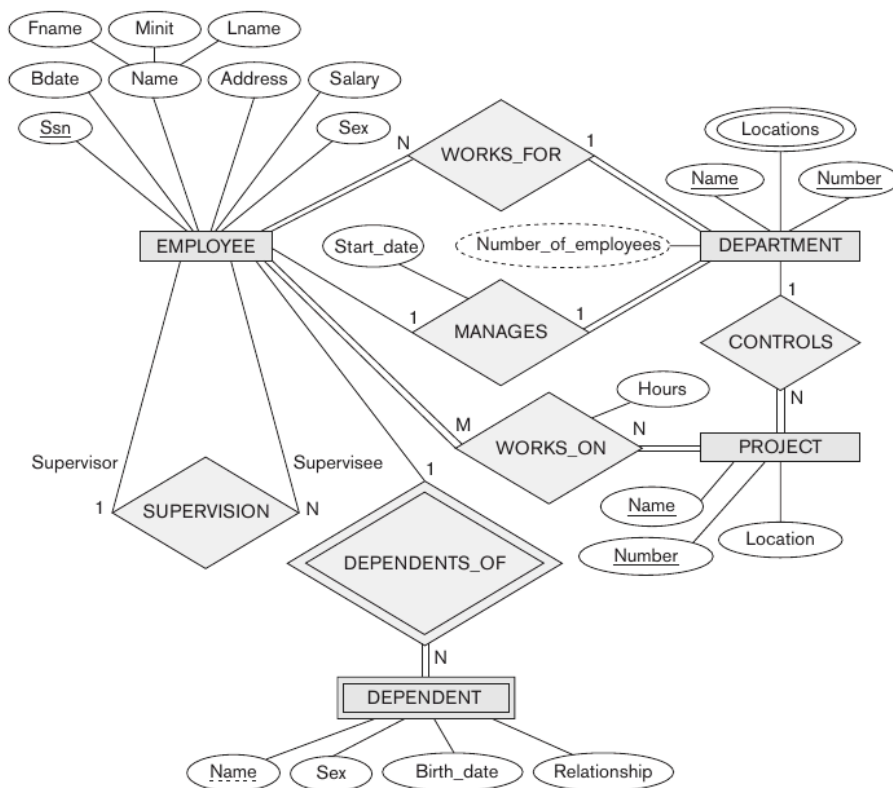


Figure 3.2

An ER schema diagram for the COMPANY database. The diagrammatic notation is introduced gradually throughout this chapter and is summarized in Figure 3.14.

Figure 3.2 (Complete): An ER schema diagram for the COMPANY database. This is the complete, refined diagram showing all entity types, relationships, attributes, and constraints. Study this diagram carefully for exams.

STUDY STRATEGY

Memorize the complete COMPANY ER diagram (Figure 3.2). Be able to draw it from memory and explain the cardinality and participation constraints for each relationship. Common exam questions ask you to identify relationship types, cardinality ratios, and convert this ER diagram to a relational schema.

All Notation Symbols at a Glance

Table 10: Complete ER Notation Quick Reference

Symbol	Meaning	Exam Relevance
Rectangle	Strong Entity	High - used in every diagram
Double Rectangle	Weak Entity	High - identify weak entities
Diamond	Relationship	High - used in every diagram
Double Diamond	Identifying Relationship	High - links weak entity to owner
Oval	Attribute	High - used in every diagram
Double Oval	Multivalued Attribute	Medium - e.g., Locations
Dotted Oval	Derived Attribute	Medium - e.g., Number_of_employees
Branching Ovals	Composite Attribute	High - e.g., Name (Fname, Minit, Lname)
Solid Underline	Key Attribute	High - e.g., Ssn
Dashed Underline	Partial Key	Medium - e.g., Dependent's Name
Single Line	Partial Participation	High - min cardinality = 0
Double Line	Total Participation	High - min cardinality = 1
1, M, N	Cardinality Ratio	High - max participation
(min, max)	Structural Constraint	Medium - alternative notation

5. Exam Questions & Answers

5.1 Short Answer Questions (2-5 Marks)

Q1. What are the two parallel pathways in the database design process?

Answer: The two parallel pathways are:

1. **Data pathway:** Requirements Collection → Conceptual Design → Logical Design → Physical Design
2. **Functionality pathway:** Functional Analysis → Application Program Design → Transaction Implementation

The data pathway produces the database schema, while the functionality pathway produces the application programs.

Q2. What is the difference between conceptual design and logical design?

Answer:

- **Conceptual Design** is DBMS-independent. It uses a high-level data model (like ER model) to create a conceptual schema that can be understood by non-technical users.
- **Logical Design** is DBMS-specific. It maps the conceptual schema to an implementation data model (like the relational model) suitable for a specific type of DBMS.

Q3. Define Entity, Entity Type, and Entity Set. Give an example of each.

Answer:

- **Entity:** A thing or object in the real world with independent existence. *Example:* A specific person named "John Smith."
- **Entity Type:** Defines a collection of entities with the same attributes. Represents the schema/intension. *Example:* The generic concept of EMPLOYEE.
- **Entity Set (Extension):** The collection of all entities of a particular type at a given point in time. *Example:* {John Smith, Fred Brown, Judy Clark} - the actual employees in the database.

Q4. What is the difference between a simple attribute and a composite attribute? Give examples.

Answer:

- **Simple (Atomic) Attribute:** Cannot be divided further. *Example:* Age, Sex, Salary.
- **Composite Attribute:** Can be divided into smaller subparts with independent meanings. *Example:* Address can be divided into Street, City, State, Zip.

Q5. Differentiate between single-valued and multivalued attributes with examples.

Answer:

- **Single-Valued:** Holds exactly one value for a given entity. *Example:* Age (a person has one age), SSN.
- **Multivalued:** Can hold a set of values. *Example:* Colors of a car (a car may be available in multiple colors), Phone numbers.

Q6. What is a derived attribute? How is it different from a stored attribute?

Answer:

- **Stored Attribute:** An attribute whose value is directly stored in the database. *Example:* Birth_date.
- **Derived Attribute:** An attribute whose value is computed from other attributes. *Example:* Age is derived from Birth_date. Derived attributes are shown with dotted ovals in ER diagrams.

Q7. What is a key attribute? What is a composite key?

Answer:

- **Key Attribute:** An attribute whose values are distinct for each individual entity in the entity set. *Example:* SSN for EMPLOYEE.
- **Composite Key:** Several attributes combined to form a key. It must be minimal (no superfluous attributes). *Example:* Car Registration composed of State and Number.

Q8. What is a recursive relationship? Give an example from the COMPANY database.

Answer: A recursive (self-referencing) relationship occurs when the same entity type participates more than once in a relationship type, but in different roles. **Example:** The SUPERVISION relationship in the COMPANY database where EMPLOYEE participates twice - once as supervisor and once as supervisee. Role names (supervisor, supervisee) are essential to distinguish the two roles.

Q9. Explain total participation and partial participation with examples.

Answer:

- **Total Participation (Existence Dependency):** An entity must participate in at least one relationship instance. Minimum cardinality = 1. Shown by a **double line**. *Example:* Every EMPLOYEE must WORKS_FOR a department.
- **Partial Participation:** An entity does not have to participate in the relationship. Minimum cardinality = 0. Shown by a **single line**. *Example:* Not every EMPLOYEE MANAGES a department.

Q10. What are the attribute migration rules for 1:1, 1:N, and M:N relationships?

Answer:

- **1:1:** Attributes can be migrated to **either** participating entity type.
- **1:N:** Attributes can be migrated only to the entity on the **N-side**.
- **M:N:** Attributes **cannot be migrated** to any single entity type; they must remain as relationship attributes.

Q11. What is a weak entity type? How is it different from a strong entity type?

Answer: A weak entity type does not have its own key attribute and is identified through its relationship with a strong entity (owner). Key differences:

Key:	Strong has a key; Weak has only a partial key
Existence:	Strong is independent; Weak depends on its owner (total participation)
Notation:	Strong = single rectangle; Weak = double rectangle
Relationship:	Weak uses identifying relationship (double diamond)

Q12. What is the (min, max) notation? Give an example.

Answer: The (min, max) notation is an alternative way to specify structural constraints on relationship participation. Each entity participation is labeled with a pair (min, max) indicating the minimum and maximum number of relationship instances an entity must/can participate in.

- (0, 1) = Partial participation, at most one relationship
- (1, 1) = Total participation, exactly one relationship
- (0, N) = Partial participation, many possible
- (1, N) = Total participation, at least one required

Q13. When should you use a ternary relationship instead of three binary relationships?

Answer: Use a ternary relationship when the relationship involves a simultaneous association among three entities that cannot be decomposed into independent binary relationships without losing semantic information. *Example:* SUPPLY(s, j, p) means "supplier s supplies part p to project j" - this cannot be inferred from three separate binary relationships because CAN_SUPPLY(s,p), SUPPLIES(s,j), and USES(j,p) existing independently does not mean s is supplying p to j.

5.2 Long Answer Questions (5-10 Marks)

Q1. Describe the four phases of database design in detail. What are the inputs and outputs of each phase?

Answer:

Phase 1: Requirements Collection and Analysis

- **Objective:** Understand and document user requirements
- **Input:** The Miniworld (real-world domain to be modeled)
- **Process:** Interview users, gather specifications
- **Output:** Data Requirements + Functional Requirements

Phase 2: Conceptual Design

- **Objective:** Create a DBMS-independent conceptual schema
- **Input:** Requirements from Phase 1
- **Process:** Use high-level data model (ER model)
- **Output:** Conceptual Schema (ER diagram)
- **Key:** Independent of any specific DBMS

Phase 3: Logical Design (Data Model Mapping)

- **Objective:** Map to an implementation data model
- **Input:** Conceptual schema
- **Process:** Transform ER model to relational/SQL model
- **Output:** Logical (Conceptual) Schema
- **Key:** DBMS-specific

Phase 4: Physical Design & Implementation

- **Objective:** Specify internal storage and implement applications
- **Input:** Logical schema
- **Process:** Define storage structures, indexes, file organizations
- **Output:** Internal Schema + Application Programs

Q2. Draw and explain the complete ER diagram for the COMPANY database. Identify all entity types, attributes, relationships, cardinality ratios, and participation constraints.

Answer: Refer to Figure 3.2 in the notes. The COMPANY database ER diagram contains:

Entity Types (4):

1. **EMPLOYEE** - Attributes: Ssn (key), Fname, Minit, Lname (composite Name), Address, Salary, Sex, Bdate
2. **DEPARTMENT** - Attributes: Name (key), Number (key), Locations (multivalued)
3. **PROJECT** - Attributes: Name (key), Number (key), Location
4. **DEPENDENT** - Weak entity - Attributes: Name (partial key), Sex, Birth_date, Relationship

Relationships (6):

1. **WORKS_FOR (1:N)**: DEPARTMENT to EMPLOYEE. Both total participation.
2. **MANAGES (1:1)**: EMPLOYEE to DEPARTMENT. Employee: partial; Department: total. Attribute: Start_date.
3. **CONTROLS (1:N)**: DEPARTMENT to PROJECT. Department: partial; Project: total.
4. **SUPERVISION (1:N)**: Recursive on EMPLOYEE. Both partial. Roles: supervisor, supervisee.
5. **WORKS_ON (M:N)**: EMPLOYEE to PROJECT. Both total. Attribute: Hours.
6. **DEPENDENTS_OF (1:N)**: Identifying relationship. EMPLOYEE to DEPENDENT. Employee: partial; Dependent: total.

Q3. Explain the different types of attributes in the ER model with examples and their ER notation.**Answer:**

Type	Description	Example	Notation
Simple	Indivisible	Age, Sex	Oval
Composite	Can be subdivided	Name → Fname, Minit, Lname	Oval with branching ovals
Single-Valued	One value per entity	SSN	Single oval
Multivalued	Multiple values possible	Locations of a department	Double oval
Stored	Directly stored	Birth_date	Solid oval
Derived	Computed from others	Age (from Birth_date)	Dotted oval
Key	Uniquely identifies entity	SSN	Oval with solid underline
Partial Key	Uniquely identifies weak entity under same owner	Dependent's Name	Oval with dashed underline

Q4. What are structural constraints on binary relationships? Explain cardinality ratios and participation constraints with examples.

Answer: Structural constraints limit the possible combinations of entities in a relationship set. They consist of two components:

1. Cardinality Ratios (Maximum Participation):

- **1:1:** Each entity in A relates to at most one in B, and vice versa. *Ex:* MANAGES between EMPLOYEE and DEPARTMENT.
- **1:N:** One in A relates to many in B, but each B relates to at most one A. *Ex:* WORKS_FOR where one department has many employees.
- **M:N:** Many-to-many in both directions. *Ex:* WORKS_ON where employees work on multiple projects and projects have multiple employees.

2. Participation Constraints (Minimum Participation):

- **Total:** Entity must participate (min = 1). Shown by double line.
- **Partial:** Entity may not participate (min = 0). Shown by single line.

Q5. Compare and contrast ternary relationships with multiple binary relationships. When would you choose one over the other?

Answer:

Ternary Relationship: Represents a simultaneous association among three entities. An instance (a, b, c) has a specific meaning that depends on all three entities together.

Multiple Binaries: Three separate relationships R(A,B), S(B,C), T(A,C). The existence of individual relationship instances does not imply the existence of the combined ternary relationship.

Key Difference: A ternary SUPPLY(s, j, p) means "supplier s supplies part p to project j specifically." Three binaries CAN_SUPPLY(s,p), SUPPLIES(s,j), USES(j,p) can all exist without s actually supplying p to j.

When to Choose:

- Choose **ternary** when the relationship semantically requires all three entities simultaneously.
- Choose **binaries** when the relationships have independent meanings (e.g., CAN_TEACH certifies ability, independent of whether a class is actually offered).
- Include **both** when they represent distinct meanings required by the application.

Q6. How can a ternary relationship be represented in a tool that only supports binary relationships? Explain both approaches.

Answer: Two approaches:

1. As a Weak Entity Type:

- Create a weak entity (e.g., SUPPLY) with **no partial key**.
- Create three identifying relationships to SUPPLIER, PART, and PROJECT.
- The three participating entities collectively serve as the composite owner/key.
- **Advantage:** Preserves semantics without introducing artificial keys.

2. As a Regular Entity with Surrogate Key:

- Introduce an artificial/surrogate key (e.g., Supply_id).
- This transforms the weak entity into a regular (strong) entity.
- Create three regular binary relationships from SUPPLY to each participating entity.
- **Advantage:** Works with any binary-only tool.
- **Disadvantage:** Introduces an artificial identifier not present in the real world.

5.3 MCQ-style Quick Review

Q1. Which phase of database design is DBMS-independent?

Answer: (b) Conceptual Design. Conceptual design uses a high-level data model (like ER) and does not include implementation details, making it independent of any specific DBMS.

Q2. In ER notation, a double rectangle represents:

Answer: (b) Weak Entity. A single rectangle is a strong/regular entity; a double rectangle is a weak entity that depends on another entity for identification.

Q3. A dotted oval in an ER diagram represents:

Answer: (c) Derived Attribute. Derived attributes are computed from other attributes and are represented by dotted ovals.

Q4. In a 1:N relationship, where can the relationship attributes be migrated?

Answer: (b) To the N-side entity only. In 1:N relationships, attributes can only be migrated to the entity on the many side because each entity on the 1-side can be related to many entities, each with its own attribute values.

Q5. Total participation is represented in an ER diagram by:

Answer: (a) Double line. A double line between an entity and relationship indicates total participation (existence dependency), meaning every entity must participate in at least one relationship instance.

Q6. A recursive relationship is one where:

Answer: (c) The same entity type participates more than once in different roles. Example: SUPERVISION where EMPLOYEE plays both supervisor and supervisee roles.

Q7. The partial key of a weak entity is represented by:

Answer: (b) Dashed underline. A solid underline represents a key attribute of a strong entity; a dashed/dotted underline represents a partial key that uniquely identifies weak entities under the same owner.

Q8. Which of the following is TRUE about weak entities?

Answer: (d) All of the above. Weak entities: (a) have total participation in their identifying relationship, (b) are identified by their owner plus partial key, and (c) are represented by double rectangles.

Q9. In the (min, max) notation, (0, N) means:

Answer: (b) Partial participation, many allowed. The 0 indicates partial participation (not required), and N indicates unlimited maximum participation.

Q10. If a relationship attribute in an M:N relationship is migrated to one of the entities:

Answer: (c) Information will be lost. M:N relationship attributes depend on the combination of both entities. Moving them to either side alone loses the association information.

Q11. The identifying relationship for a weak entity is represented by:

Answer: (b) Double diamond. A regular relationship is a single diamond; an identifying relationship (which links a weak entity to its owner) is a double diamond.

Q12. A composite key must be:

Answer: (b) Minimal. A composite key must not contain any superfluous attributes. Removing any attribute from the key should cause it to no longer uniquely identify entities.

5.4 Key Formulas & Definitions Summary

Table 9: Quick Reference Formulas

Concept	Definition/Formula
Attribute as function	$A : E \rightarrow P(V)$
NULL value	Represented by empty set $\emptyset$
Relationship Set	$R \subseteq E_1 \times E_2 \times \dots \times E_n$
Composite attribute value set	$V = P(V_1 \times V_2 \times \dots \times V_n)$
(min, max) constraints	$0 \leq \text{min} \leq \text{max}$ and $\text{max} \geq 1$
Partial participation	$\text{min} = 0$
Total participation	$\text{min} \geq 1$

FINAL STUDY TIPS

1. **Draw the diagrams:** Practice drawing the complete COMPANY ER diagram from memory.
2. **Know the notation:** Be able to identify any ER symbol and explain what it means.
3. **Understand constraints:** Cardinality ratios and participation constraints are frequently tested.
4. **Compare concepts:** Be ready to compare strong vs weak entities, binary vs ternary, stored vs derived attributes.
5. **Apply migration rules:** Know when and where attributes can be migrated based on cardinality.
6. **Practice the Qs:** Cover all short and long answer questions in this section.



COMPLETE EXAM NOTES

DBMS Unit 3

Relational Model Concepts, Constraints & Relational Algebra

Modules: 5.1, 5.2, 5.3, 5.4, 8.1, 8.2, 8.3, 8.4 & 8.5

With Visual Diagrams, Worked Examples & Q&A

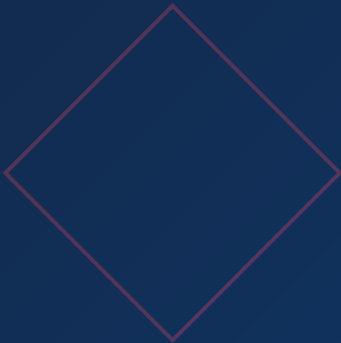


Table of Contents

1. Introduction to the Relational Model

2. Formal Definitions & Core Concepts

2.1 Domains

2.2 Relation Schema

2.3 Relation State

2.4 Mathematical Set Theory Definition

3. Characteristics of Relations

3.1 Ordering of Tuples

3.2 Ordering of Values within a Tuple

3.3 First Normal Form (1NF)

3.4 NULL Values

3.5 Interpretation of a Relation

4. Standard Relational Model Notation

5. Relational Model Constraints

5.1 Categories of Constraints

5.2 Domain Constraints

5.3 Key Constraints & NULL Restrictions

5.4 Relational Databases & Schemas

5.5 Entity Integrity, Referential Integrity & Foreign Keys

5.6 Semantic & Transition Constraints

6. Update Operations & Constraint Violations

6.1 Insert Operation

6.2 Delete Operation

6.3 Update (Modify) Operation

7. Transaction Concept

8. Relational Algebra: Unary Operations

8.1 SELECT Operation

8.2 PROJECT Operation

8.3 Sequences, Assignment & RENAME

9. Set Theory Operations

9.1 Union Compatibility

9.2 UNION

9.3 INTERSECTION

9.4 SET DIFFERENCE

9.5 CARTESIAN PRODUCT

10. Binary Operations: JOIN

10.1 JOIN Operation

10.2 THETA JOIN

10.3 EQUIJOIN

10.4 NATURAL JOIN

10.5 Join Selectivity & N-Way Joins

11. Additional Relational Operations

11.1 Generalized Projection

11.2 Aggregate Functions & Grouping

11.3 OUTER JOIN

11.4 OUTER UNION

12. Query Examples in Relational Algebra

13. Exam Q&A

1. Introduction to the Relational Model

The relational model represents a database as a collection of **relations**. It was introduced by E.F. Codd in 1970 and remains the dominant data model for commercial database systems.

Informal View

A relation resembles a **table of values** or a flat file of records (where records have a simple linear/flat structure).

Logical Structure

- **Row:** Represents a collection of related data values mapping to a fact about a real-world entity or relationship.
- **Column:** Values in a specific column share the same data type. Column headers define how to interpret the row's values.

Formal vs. Informal Terminology

Table 1: Relational Model Terminology Mapping

Informal Term	Formal Term
Table	Relation
Row	Tuple
Column Header	Attribute
Data Type	Domain

1. Introduction to the Relational Model

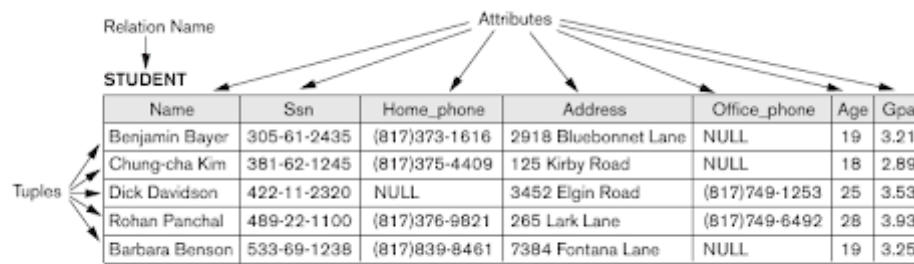


Figure 5.1
The attributes and tuples of a relation **STUDENT**.

Figure 5.1: The attributes and tuples of a relation **STUDENT**. Each row is a tuple representing one student; each column is an attribute with a specific domain.

Exam Tip

Remember the four key terms: **Relation = Table**, **Tuple = Row**, **Attribute = Column**, **Domain = Data Type**. These are fundamental and appear in almost every exam question.

2. Formal Definitions & Core Concepts

2.1 Domains (D)

Definition

A **domain** D is a set of **atomic** (indivisible) values from which data values are drawn.

Components of a Domain Definition:

1. **Name:** Logical definition (e.g., *Usa_phone_numbers*, *Employee_ages*)
2. **Data Type:** E.g., character string, integer, real number
3. **Format:** Specific structural requirements (e.g., (ddd)ddd-dddd for phones)
4. **Additional Information:** e.g., Units of measurement (pounds, kilograms)

Important Note

Multiple attributes in a relation can share the same domain if they represent **different roles** (e.g., *Home_phone* and *Office_phone* both use the *Usa_phone_numbers* domain).

2.2 Relation Schema (R)

Definition

The **relation schema** R is the description or structural blueprint of a relation. Also called the **Relation Intension**.

Notation: $R(A_1, A_2, \dots, A_n)$

- **Components:** A relation name R and a list of attributes $A_1, A_2, \dots, A_n$
- **Attribute A_i :** Represents a specific role played by a domain D in schema R
- **Denoted as:** $\text{dom}(A_i) = D$
- **Degree (or Arity):** The number of attributes (n) in the relation schema

Example: STUDENT(Name : string, Ssn : string, Age : integer)

2.3 Relation State (r)

Definition

The **relation state** (or relation extension) $r(R)$ is the actual data populated within the relation at a specific moment in time.

Notation: $r(R)$ or simply r

- **Definition:** A set of n -tuples $r = \{t_1, t_2, \dots, t_m\}$
- **Tuple t :** An ordered list of n values $t = \langle v_1, v_2, \dots, v_n \rangle$ where each v_i must be an element of $\text{dom}(A_i)$ or be a special NULL value
- **Component Referencing:** The j -th value in tuple t (corresponding to attribute A_j) is denoted as $t[A_j]$, $t.A_j$, or positionally as $t[j]$

2.4 Mathematical Set Theory Definition

Formally, a relation state $r(R)$ of degree n is a mathematical relation on the underlying domains, defined as a **subset of the Cartesian product** of those domains:

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

- **Cartesian Product:** Represents all mathematically possible combinations of values
- **Total Possible Tuples:** Calculated via cardinality of each domain: $|D_1| \times |D_2| \times \dots \times |D_n|$
- **State vs. Schema:** Out of all possible combinations, a relation state r contains only valid tuples representing real-world facts. The **state changes frequently**, while the **schema is relatively static**.

Exam Tip

Remember: **Schema** = **Intension** (structure, static) and **State** = **Extension** (data, dynamic). The schema defines what the relation *is*; the state defines what it *contains right now*.

3. Characteristics of Relations

3.1 Ordering of Tuples

- A relation is mathematically defined as a **set**, so elements have no specific order.
- A relation is **strictly order-insensitive** (unlike physical disk files which have a 1st, 2nd, and last record).
- **Two identical tables with rows shuffled are considered the exact same relation.**

Figure 5.2

The relation STUDENT from Figure 5.1 with a different order of tuples.

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21

$$t = \langle (\text{Name, Dick Davidson}), (\text{Ssn, 422-11-2320}), (\text{Home_phone, NULL}), (\text{Address, 3452 Elgin Road}), (\text{Office_phone, (817)749-1253}), (\text{Age, 25}), (\text{Gpa, 3.53}) \rangle$$

$$t = \langle (\text{Address, 3452 Elgin Road}), (\text{Name, Dick Davidson}), (\text{Ssn, 422-11-2320}), (\text{Age, 25}), (\text{Office_phone, (817)749-1253}), (\text{Gpa, 3.53}), (\text{Home_phone, NULL}) \rangle$$

Figure 5.3

Two identical tuples when the order of attributes and values is not part of relation definition.

Figure 5.2 & 5.3: (Top) The same STUDENT relation with tuples in different order - still the same relation. (Bottom) Two identical tuples when order of attributes is not part of the relation definition (self-describing data).

3.2 Ordering of Values within a Tuple (Alternative Definition)

- **Standard Definition:** A tuple is an ordered list, making attribute order important.
- **Alternative Abstract Definition:** A tuple can be defined without assuming value ordering:
 - Schema R is a **set** of attributes (not a list)

- Relation state r is a finite set of mappings
 - Tuple t : A mapping from R to $\bigcup D$, where $t[A_i] \in D_i$
 - Effectively, a tuple is a set of $(\langle \text{attribute} \rangle, \langle \text{value} \rangle)$ pairs
- This is known as **self-describing data**.

3.3 First Normal Form (1NF) Assumption

Atomicity Requirement

Every value in a tuple must be **indivisible** (atomic).

- **Composite attributes** and **multivalued attributes** are strictly prohibited in the basic flat relational model.
- **Resolution**: Multivalued attributes require separate relations; composite attributes are broken down into simple components.

3.4 NULL Values

Used when a value is **unknown** or **does not apply** to a specific tuple.

Three Interpretations of NULL:

1. Value exists but is currently **unknown**
2. Value exists but is **not available**
3. Attribute **does not apply** to this tuple (value undefined)

Warning

NULLs create ambiguity in arithmetic aggregations and comparisons (e.g., comparing two NULLs). They should be **minimized** during database design.

3.5 Interpretation (Meaning) of a Relation

1. As an Assertion:

- A schema acts as a declaration (e.g., "Students have a Name, SSN, and GPA")

3. Characteristics of Relations

- Tuples are specific facts or instances of that assertion
- Relations can uniformly represent **both entities and relationships**

2. As a Predicate (Logic Programming):

- A schema represents a predicate: $P(A_1, A_2, \dots, A_n)$
- Tuple values are variables that make the predicate true
- **Closed World Assumption:** The only true facts are those physically present in the relation. Any combination not in the relation makes the predicate false.

4. Standard Relational Model Notation

When writing formulas, proofs, or queries in DBMS theory, strictly adhere to the following naming conventions:

Table 2: Standard Notation Conventions

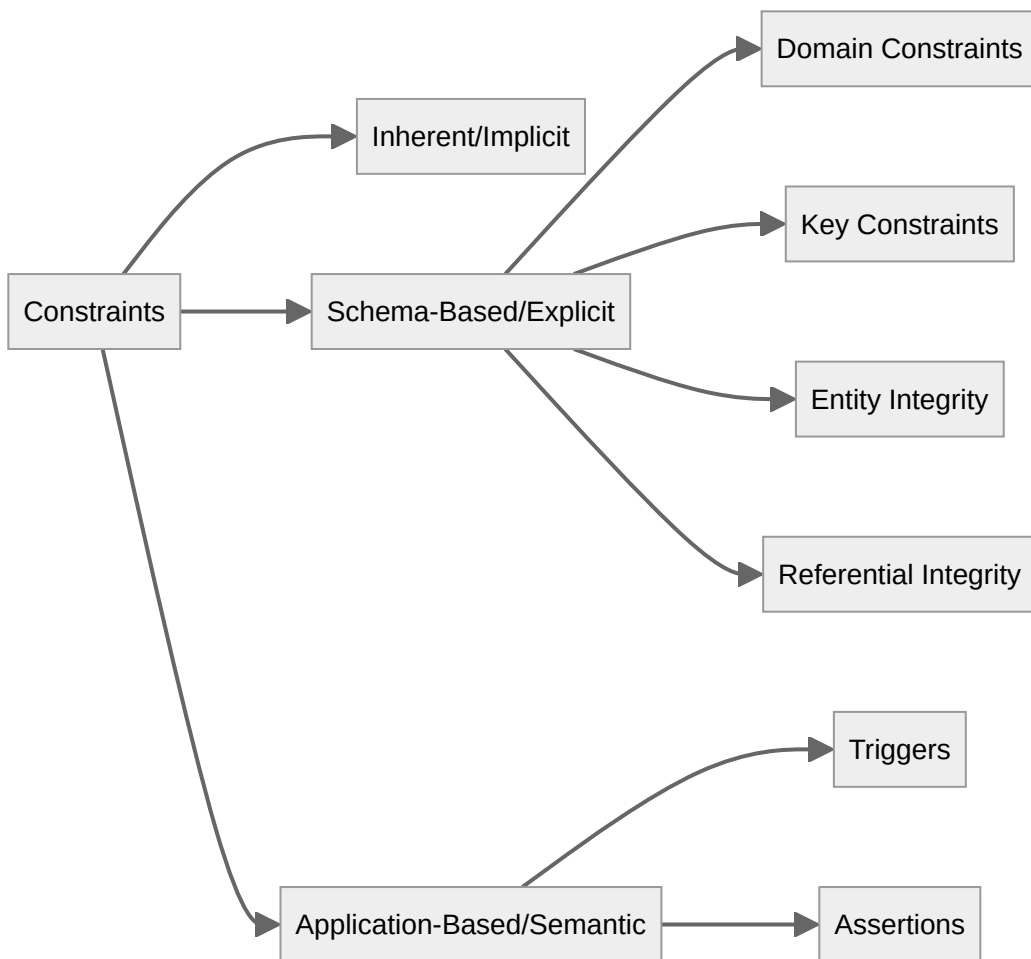
Element	Notation	Example
Relation Schemas	Uppercase letter, typically R of degree n : $R(A_1, \dots, A_n)$	$R(A, B, C)$
Relation Names	Uppercase letters	STUDENT, EMPLOYEE
Relation States	Lowercase letters	r, s
Tuples	Lowercase letters	t, u, v
Attribute Qualification	Dot notation: RelationName.Attribute	STUDENT.Name
Single attribute extraction	$t[A_i]$ or $t.A_i$ or $t[i]$	$t[\text{Ssn}]$
Subtuple extraction	$t[Y]$ where $Y \subseteq R$	$t[\text{Name, Age}]$

Critical Rule

All attributes within a single relation must be **distinct**.

5. Relational Model Constraints & Database Schemas

A database models rules from a "miniworld". **Constraints** enforce these rules to maintain data integrity.



5.1 Three Main Categories of Constraints

1. **Inherent / Implicit Constraints:** Built into the data model itself (e.g., a relation cannot have duplicate tuples; tuple ordering is irrelevant).
2. **Schema-based / Explicit Constraints:** Directly expressed in the schema using DDL (Data Definition Language).

3. Application-based / Semantic Constraints (Business Rules): Too complex for schema DDL; enforced via application programs, assertions, or triggers.

- *Note:* Data Dependencies (Functional & Multivalued Dependencies) form another category used for testing "goodness" of design during normalization.

5.2 Domain Constraints

Definition

Specifies that within each tuple, the value of each attribute A must be an atomic value from the established domain $\text{dom}(A)$.

- **Data Types:** Integers, real numbers, characters, booleans, fixed/variable strings, dates, times, timestamps
- May also include a **subrange** of values or an **enumerated list**

5.3 Key Constraints and NULL Restrictions

Because a relation is mathematically a set, **all tuples must be distinct**. No two tuples can have the exact same combination of values for all attributes.

Superkey (SK)

Definition

A **superkey** SK is a subset of attributes in relation schema R that enforces uniqueness.

Constraint: For any two distinct tuples t_1 and t_2 in state r : $t_1[SK] \neq t_2[SK]$

- A superkey can contain **redundant attributes**
- Every relation has at least one default superkey: the **set of all its attributes**

Key (K)

Definition

A **key** is a **minimal superkey**.

- A key is a superkey, but it **lacks redundancy**
- Removing any attribute from K leaves a set that is **no longer a superkey**
- **Property 1 - Uniqueness:** No two distinct tuples share identical key values
- **Property 2 - Minimality:** No attribute can be removed while maintaining uniqueness
- **Time-Invariant:** A key must hold for all valid current and future relation states

Candidate Keys vs. Primary Keys

- A relation may have **multiple keys**, called **Candidate Keys** (e.g., License_number and Engine_serial_number)
- **Primary Key:** The single candidate key chosen to identify tuples
 - Convention: Primary key attributes are **underlined** in schema diagrams
 - Best practice: Choose the key with the **fewest attributes** (preferably one)
- **Unique Keys:** The remaining, unchosen candidate keys

Figure 5.4
The CAR relation, with two candidate keys: License_number and Engine_serial_number.

CAR				
License_number	Engine_serial_number	Make	Model	Year
Texas ABC-739	A69352	Ford	Mustang	02
Florida TVP-347	B43696	Oldsmobile	Cutlass	05
New York MPO-22	X83554	Oldsmobile	Delta	01
California 432-TFY	C43742	Mercedes	190-D	99
California RSK-629	Y82935	Toyota	Camry	04
Texas RSK-629	U028365	Jaguar	XJS	04

Figure 5.4: The CAR relation with two candidate keys: License_number and Engine_serial_number. Either could serve as the primary key.

Constraints on NULLs

Specific attributes can be restricted from holding NULL values (e.g., via the **NOT NULL** declaration) if a valid value is strictly required.

Exam Tip

Superkey vs. Key: All keys are superkeys, but not all superkeys are keys. A superkey may have extra (redundant) attributes; a key is minimal. Think of it as: Superkey = uniqueness; Key = uniqueness + minimality.

5.4 Relational Databases and Schemas

Relational Database Schema (S)

A collection of relation schemas $R_1, R_2, \dots, R_m$ alongside a set of Integrity Constraints IC .

Relational Database State (DB)

A collection of relation states $\langle r_1, r_2, \dots, r_m \rangle$ where each r_i is a state of R_i . A state is valid only if every r_i simultaneously satisfies all integrity constraints IC .

Attribute Naming Conventions Across Relations

- Identical real-world concepts can have **different names** across relations (e.g., Dnumber in DEPARTMENT vs. Dno in EMPLOYEE)
- Different concepts can **share the same name** if in different relations (e.g., Name for a project vs. Name for a department)
- **Exception:** If the same real-world concept serves multiple roles *within the same relation*, attributes must have distinct names (e.g., Ssn and Super_ssn in EMPLOYEE)

EMPLOYEE

<u>Fname</u>	<u>Minit</u>	<u>Lname</u>	<u>Ssn</u>	<u>Bdate</u>	<u>Address</u>	<u>Sex</u>	<u>Salary</u>	<u>Super_ssn</u>	<u>Dno</u>
--------------	--------------	--------------	------------	--------------	----------------	------------	---------------	------------------	------------

DEPARTMENT

<u>Dname</u>	<u>Dnumber</u>	<u>Mgr_ssn</u>	<u>Mgr_start_date</u>
--------------	----------------	----------------	-----------------------

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
----------------	------------------

PROJECT

<u>Pname</u>	<u>Pnumber</u>	<u>Plocation</u>	<u>Dnum</u>
--------------	----------------	------------------	-------------

WORKS_ON

<u>Essn</u>	<u>Pno</u>	<u>Hours</u>
-------------	------------	--------------

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	<u>Sex</u>	<u>Bdate</u>	<u>Relationship</u>
-------------	-----------------------	------------	--------------	---------------------

Figure 5.5

Schema diagram for the COMPANY relational database schema.

Figure 5.5: Schema diagram for the COMPANY relational database schema showing six relations and their attributes.

Figure 5.6

One possible database state for the COMPANY relational database schema.

EMPLOYEE

<u>Fname</u>	<u>Minit</u>	<u>Lname</u>	<u>Ssn</u>	<u>Bdate</u>	<u>Address</u>	<u>Sex</u>	<u>Salary</u>	<u>Super_ssn</u>	<u>Dno</u>
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	NULL	1

DEPARTMENT

<u>Dname</u>	<u>Dnumber</u>	<u>Mgr_ssn</u>	<u>Mgr_start_date</u>
Research	5	333445555	1988-05-22
Administration	4	987654321	1995-01-01
Headquarters	1	888665555	1981-06-19

DEPT_LOCATIONS

<u>Dnumber</u>	<u>Dlocation</u>
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

WORKS_ON

<u>Essn</u>	<u>Pno</u>	<u>Hours</u>
123456789	1	32.5
123456789	2	7.5
666884444	3	40.0
453453453	1	20.0
453453453	2	20.0
333445555	2	10.0
333445555	3	10.0
333445555	10	10.0
333445555	20	10.0
999887777	30	30.0
999887777	10	10.0
987987987	10	35.0
987987987	30	5.0
987654321	30	20.0
987654321	20	15.0
888665555	20	NULL

PROJECT

<u>Pname</u>	<u>Pnumber</u>	<u>Plocation</u>	<u>Dnum</u>
ProductX	1	Bellaire	5
ProductY	2	Sugarland	5
ProductZ	3	Houston	5
Computerization	10	Stafford	4
Reorganization	20	Houston	1
Newbenefits	30	Stafford	4

DEPENDENT

<u>Essn</u>	<u>Dependent_name</u>	<u>Sex</u>	<u>Bdate</u>	<u>Relationship</u>
333445555	Alice	F	1986-04-05	Daughter
333445555	Theodore	M	1983-10-25	Son
333445555	Joy	F	1958-05-03	Spouse
987654321	Abner	M	1942-02-28	Spouse
123456789	Michael	M	1968-01-04	Son
123456789	Alice	F	1988-12-30	Daughter
123456789	Elizabeth	F	1967-05-05	Spouse

Figure 5.6: One possible database state for the COMPANY relational database schema - all tables populated with sample data.

Entity Integrity Constraint

Rule

No primary key value can be NULL.

Rationale: The primary key identifies the tuple. A NULL primary key renders the tuple unidentifiable and impossible to reference securely.

Referential Integrity Constraint

Rule

A tuple in one relation referring to another must refer to an **existing, valid tuple** in that target relation.

Foreign Key (FK)

Definition

A foreign key is the formal mechanism for referential integrity. A set of attributes FK in a referencing relation R_1 that references a referenced relation R_2 .

Rules for Foreign Keys:

1. The attributes in FK must share the same domain(s) as the primary key PK of R_2
2. A value of FK in tuple t_1 of state r_1 must either:
 - Match a value of PK for some tuple t_2 in state r_2 , such that $t_1[FK] = t_2[PK]$
 - Or be NULL (provided the schema allows)
- **Self-Referencing:** A foreign key can reference its own relation (e.g., Super_ssn in EMPLOYEE referencing Ssn in EMPLOYEE)
- **Diagrammatic Notation:** Represented by a directed arc from the foreign key pointing to the primary key of the referenced relation

Figure 5.7

Referential integrity constraints displayed on the COMPANY relational database schema.

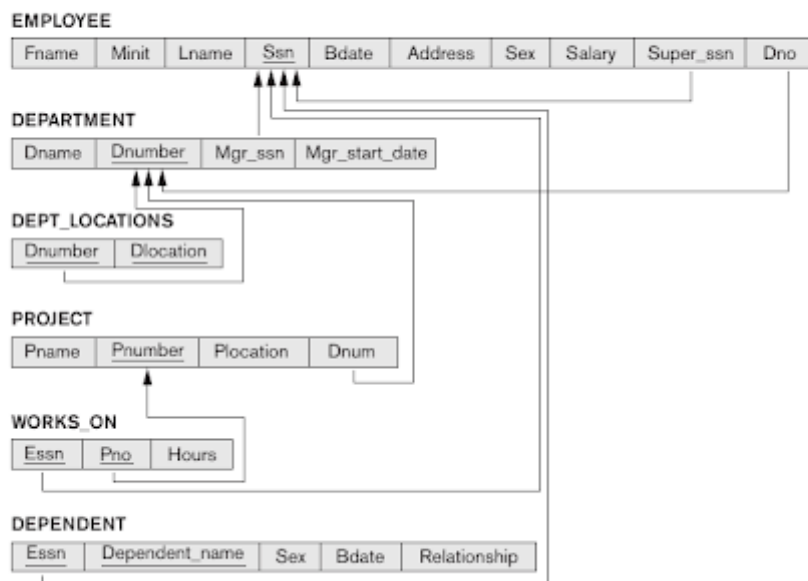


Figure 5.7: Referential integrity constraints displayed on the COMPANY relational database schema. Arcs show foreign key references.

5.6 Semantic and Transition Constraints

Semantic Integrity Constraints (State Constraints)

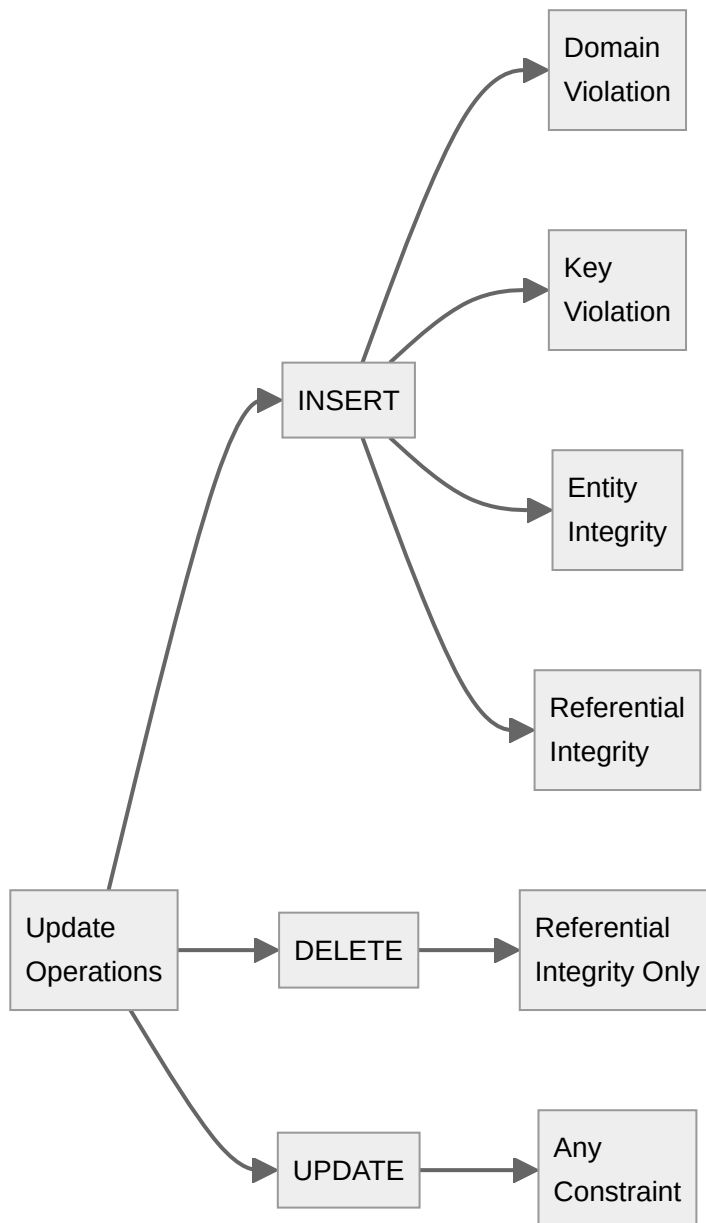
- Define constraints a valid database state must satisfy that fall outside standard DDL capabilities
- Example: "An employee's salary cannot exceed their supervisor's salary"
- Enforced through **application logic, SQL Triggers, or Assertions**

Transition Constraints

- Define restrictions based on **state changes** rather than static states
- Example: "A salary can only be increased, never decreased"
- Enforced using **active rules, triggers, or application program logic**

6. Update Operations & Constraint Violations

Relational model operations are categorized into **retrievals** (querying) and **updates** (modifying). Every update must guarantee the resulting state remains valid.



6.1 The Insert Operation

Provides a list of attribute values for a new tuple to be inserted into relation R .

Potential Violations:

1. **Domain Constraints:** Value falls outside the attribute's domain or has wrong data type
2. **Key Constraints:** New tuple's key value already exists in another tuple
3. **Entity Integrity:** Any part of the new tuple's primary key is NULL
4. **Referential Integrity:** Foreign key value refers to a non-existent tuple

Violation Handling:

- **Default:** Reject the insertion
- **Alternative:** Prompt user to correct the issue; rarely, cascade the insert

6.2 The Delete Operation

Removes a tuple (or tuples) selected via a specified condition.

Key Fact

A Delete can **ONLY violate Referential Integrity** - when the tuple being deleted is currently referenced by foreign keys from other tuples.

Violation Handling Options:

Table 3: Delete Operation Violation Handling

Option	Action	Example
RESTRICT (default)	Reject deletion to prevent orphaned references	Cannot delete employee who has dependents
CASCADE	Automatically delete all referencing tuples	Delete employee + all their dependent records
SET NULL	Set referencing foreign key to NULL	Set Super_ssn to NULL for subordinates
SET DEFAULT	Replace with a valid default reference	Assign to default department

Note: Combinations are possible - cascade in one table, set null in another.

6.3 The Update (Modify) Operation

Changes values of one or more attributes in existing tuples.

Potential Violations:

- **Updating Non-Key Attributes:** Generally safe. Only risks Domain Constraint violations.
- **Updating a Primary Key:** Functionally identical to deleting the old tuple and inserting a new one. Can trigger ANY constraint violation.
- **Updating a Foreign Key:** Risks Referential Integrity if the new value doesn't reference a valid tuple (unless set to NULL).

Violation Handling: Mirrors Delete strategies (Restrict, Cascade, Set Null/Default).

Exam Tip

Remember which operations can violate which constraints:

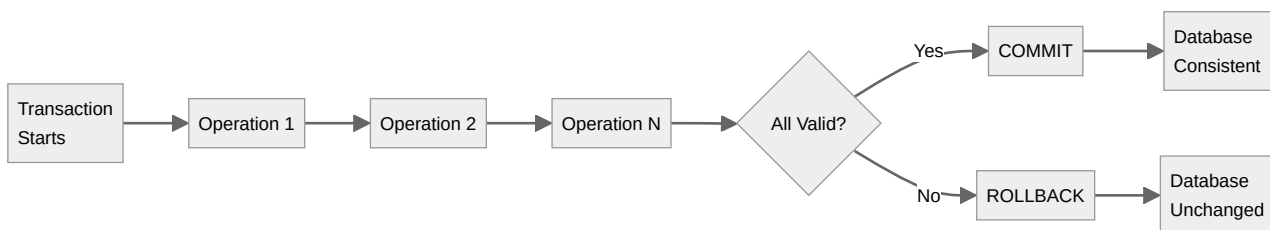
- **INSERT:** Can violate ALL four (Domain, Key, Entity Integrity, Referential Integrity)
- **DELETE:** Only Referential Integrity
- **UPDATE:** Depends on what's being updated (non-key = domain only; PK = all; FK = referential)

7. The Transaction Concept

Definition

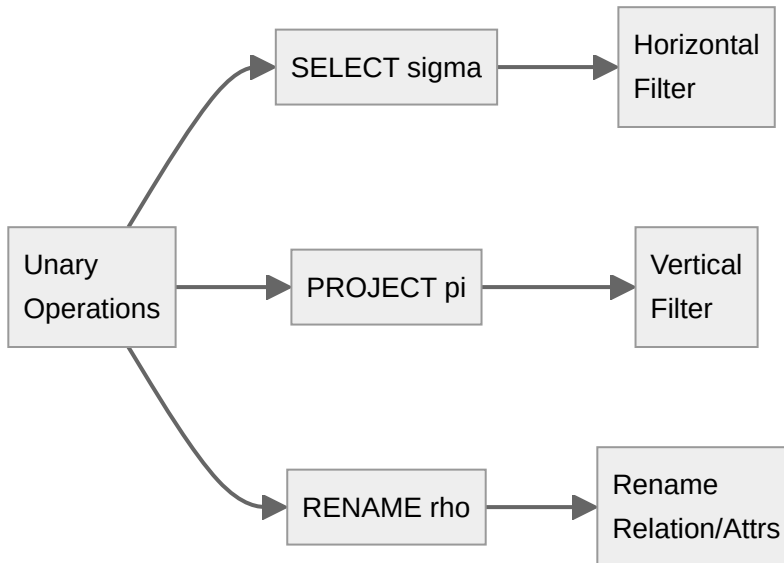
A **transaction** represents a logical, atomic unit of work against the database. It is an executing application program that includes database operations (reading, inserting, deleting, or updating).

- **Core Requirement:** By the end of execution, the database must be in a valid and consistent state (all schemas and constraints fully satisfied).
- **Atomicity:** A single transaction groups multiple operations. They are treated as an atomic whole - **either all succeed, or none are permanently applied.**
- **Environment:** Essential in OLTP (Online Transaction Processing) systems handling hundreds of concurrent transactions per second.



8. Relational Algebra: Unary Operations

Relational algebra provides a formal foundation for relational database operations. **Unary operations** are applied to a single relation.



8.1 The SELECT Operation (σ)

Definition

The SELECT operation filters a subset of tuples from a relation that satisfies a specific condition. It acts as a **horizontal partition**.

Notation: $\sigma_{\langle \text{selection condition} \rangle}(R)$

- σ (sigma): The SELECT operator
- Selection condition: A Boolean expression defining the filtering logic
- R : A relational algebra expression resulting in a relation

Selection Condition Format:

- Clauses: $\langle \text{attribute} \rangle \langle \text{comparison op} \rangle \langle \text{constant} \rangle$ OR $\langle \text{attr1} \rangle \langle \text{op} \rangle \langle \text{attr2} \rangle$
- **Comparison Operators:** $=, \neq, <, \leq, >, \geq$

- Can be applied to ordered domains (numbers, dates)
- For unordered domains, only $=$, $\neq$ are valid
- **Boolean Connectives:** AND, OR, NOT

Mathematical Characteristics:

- **Degree:** Identical to the degree of R
- **Selectivity:** Number of tuples in result $\leq$ number in R . The fraction selected is called the condition's **selectivity**.
- **Commutativity:** $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$
- **Cascading:** $\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c1 \text{ AND } c2}(R)$

SQL Equivalent: Maps to the **WHERE** clause.

8.2 The PROJECT Operation (π)

Definition

The PROJECT operation selects specific columns from a relation while discarding the rest, creating a **vertical partition**.

Notation: $\pi_{\langle \text{attribute list} \rangle}(R)$

- π (pi): The PROJECT operator
- Attribute list: The desired sublist of attributes to retain

Mathematical Characteristics:

- **Degree:** Equal to the number of attributes in the attribute list
- **Duplicate Elimination:** If the attribute list contains only non-key attributes, identical tuples are **automatically removed** (set theory requirement)
- **Result Size:** Number of tuples $\leq |R|$; equality holds only if the projection forms a **superkey**
- **Not Commutative**

SQL Equivalent: Maps to the **SELECT** clause. Use **DISTINCT** to enforce true projection behavior.

Figure 8.1

Results of SELECT and PROJECT operations. (a) $\sigma_{(Dno=4 \text{ AND } Salary > 25000) \text{ OR } (Dno=5 \text{ AND } Salary > 30000)}(EMPLOYEE)$.
 (b) $\pi_{Lname, Fname, Salary}(EMPLOYEE)$. (c) $\pi_{Sex, Salary}(EMPLOYEE)$.

(a)

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5

(b)

Lname	Fname	Salary
Smith	John	30000
Wong	Franklin	40000
Zelaya	Alicia	25000
Wallace	Jennifer	43000
Narayan	Ramesh	38000
English	Joyce	25000
Jabbar	Ahmad	25000
Borg	James	55000

(c)

Sex	Salary
M	30000
M	40000
F	25000
F	43000
M	38000
M	25000
M	55000

Figure 8.1: Results of SELECT and PROJECT operations: (a) Selection with condition, (b) Projection on Lname, Fname, Salary, (c) Projection on Sex, Salary.

SELECT vs PROJECT - Critical Difference

- **SELECT (σ):** Filters **rows** (horizontal). Keeps all columns.
- **PROJECT (π):** Filters **columns** (vertical). Keeps selected columns only, removes duplicates.

8.3 Sequences of Operations, Assignment, and RENAME (ρ)

Complex queries often require compounding multiple operations.

In-line Expressions: Operations can be nested mathematically.

Example: $\pi_{Fname, Lname, Salary}(\sigma_{Dno=5}(EMPLOYEE))$

Intermediate Relations & Assignment ($\leftarrow$):

Break down step-by-step:

```
R1 <- sigma_Dno=5(EMPLOYEE)
R <- pi_Fname, Lname, Salary(R1)
```

Implicit Renaming: Rename attributes while defining intermediate results:

```
TEMP <- pi_Fname,Lname,Salary(DEP5)
R(First_name, Last_name, Salary) <- TEMP
```

The Formal RENAME Operator (ρ)

Definition

A formal unary operator to rename a relation, its attributes, or both.

Notation Forms:

1. **Rename Relation & Attributes:** $\rho_{S(B_1, \dots, B_n)}(R)$ - Changes relation name to S and attributes to $B_1, \dots, B_n$
2. **Rename Relation Only:** $\rho_S(R)$
3. **Rename Attributes Only:** $\rho_{(B_1, \dots, B_n)}(R)$

SQL Equivalent: Aliasing using the **AS** keyword (e.g., SELECT E.Fname AS First_name FROM EMPLOYEE AS E)

(a)

Fname	Lname	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

(b)

TEMP

Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

R

First_name	Last_name	Salary
John	Smith	30000
Franklin	Wong	40000
Ramesh	Narayan	38000
Joyce	English	25000

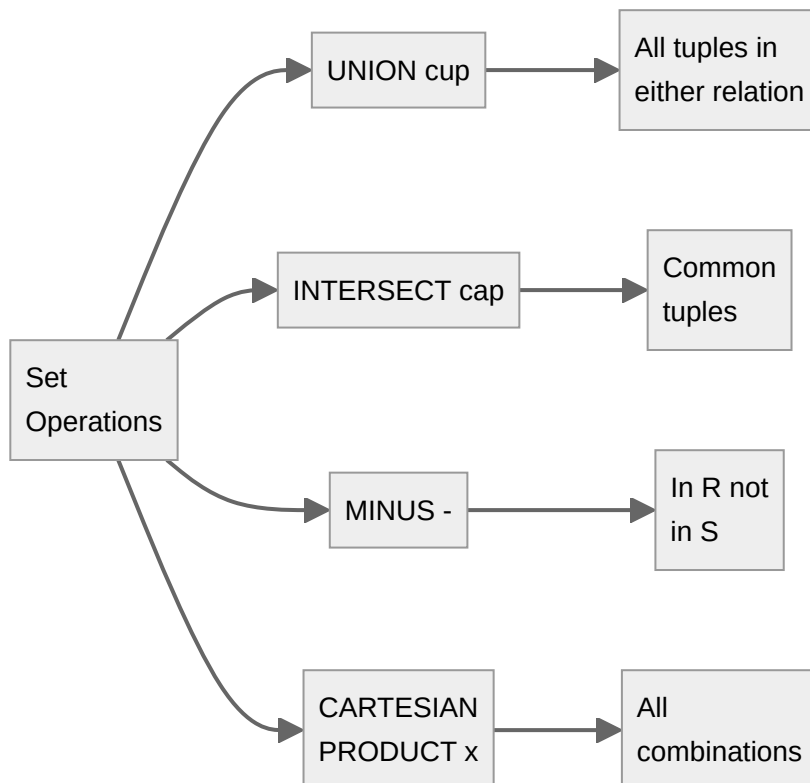
Figure 8.2

Results of a sequence of operations. (a) $\pi_{Fname, Lname, Salary}(\sigma_{Dno=5}(EMPLOYEE))$.
(b) Using intermediate relations and renaming of attributes.

Figure 8.2: Results of a sequence of operations: (a) Projection, (b) Using intermediate relations and renaming of attributes.

9. Relational Algebra: Set Theory Operations

These operations merge elements of two sets. Relations must satisfy **structural compatibility**.



9.1 Union Compatibility (Type Compatibility)

Definition

For UNION, INTERSECTION, and SET DIFFERENCE to be valid, relations R and S must be **Union Compatible**.

Two Conditions:

1. Same degree: $\text{degree}(R) = \text{degree}(S)$
2. Corresponding attribute pairs share the same domain: $\text{dom}(A_i) = \text{dom}(B_i)$ for $1 \leq i \leq n$

By convention, the resulting relation inherits attribute names from the first relation R .

9.2 The UNION Operation ($\cup$)

Definition

Combines tuples of two union-compatible relations. Result contains all tuples in R , in S , or in **both**.

Notation: $R \cup S$

- **Duplicate Elimination:** Duplicates appear only once in the result
- **Properties:** Commutative and Associative
- **SQL:** UNION (or UNION ALL to retain duplicates)

RESULT1	RESULT2	RESULT
Ssn	Ssn	Ssn
123456789	333445555	123456789
333445555	888665555	333445555
666884444		666884444
453453453		453453453
		888665555

Figure 8.3
Result of the UNION operation
 $RESULT \leftarrow RESULT1 \cup RESULT2$.

Figure 8.3: Result of the UNION operation: $RESULT = RESULT1 \cup RESULT2$. Duplicates are eliminated.

9.3 The INTERSECTION Operation ($\cap$)

Definition

Finds common tuples between two union-compatible relations. Result contains **only** tuples present in **both** R and S .

Notation: $R \cap S$

- **Properties:** Commutative and Associative
- **Expressing via other operations:** $R \cap S = R - (R - S)$
- **SQL:** INTERSECT

9.4 The SET DIFFERENCE / MINUS Operation ($-$)

Definition

Finds tuples in R but NOT in S .

Notation: $R - S$

- **Properties:** NOT Commutative ($R - S \neq S - R$)
- **SQL:** EXCEPT

Figure 8.4

The set operations UNION, INTERSECTION, and MINUS. (a) Two union-compatible relations. (b) $\text{STUDENT} \cup \text{INSTRUCTOR}$. (c) $\text{STUDENT} \cap \text{INSTRUCTOR}$. (d) $\text{STUDENT} - \text{INSTRUCTOR}$. (e) $\text{INSTRUCTOR} - \text{STUDENT}$.

(a) STUDENT		INSTRUCTOR		(b)	
Fn	Ln	Fname	Lname	Fn	Ln
Susan	Yao	John	Smith	Susan	Yao
Ramesh	Shah	Ricardo	Browne	Ramesh	Shah
Johnny	Kohler	Susan	Yao	Johnny	Kohler
Barbara	Jones	Francis	Johnson	Barbara	Jones
Amy	Ford	Ramesh	Shah	Amy	Ford
Jimmy	Wang			Jimmy	Wang
Ernest	Gilbert			Ernest	Gilbert
				John	Smith
				Ricardo	Browne
				Francis	Johnson

(c)		(d)		(e)	
Fn	Ln	Fn	Ln	Fname	Lname
Susan	Yao	Johnny	Kohler	John	Smith
Ramesh	Shah	Barbara	Jones	Ricardo	Browne
		Amy	Ford	Francis	Johnson
		Jimmy	Wang		
		Ernest	Gilbert		

Figure 8.4: The set operations UNION, INTERSECTION, and MINUS: (a) Two union-compatible relations, (b) UNION, (c) INTERSECTION, (d) STUDENT - INSTRUCTOR, (e) INSTRUCTOR - STUDENT.

9.5 The CARTESIAN PRODUCT ($\times$)

Definition

Creates an **exhaustive combination** of tuples from two relations.

Notation: $R \times S$

- **Result Degree:** If R has degree n and S has degree m , result has degree $n + m$
- **Cardinality:** If $|R| = n_r$ and $|S| = n_s$, result contains $n_r \times n_s$ tuples
- **Usage Warning:** Usually meaningless on its own; almost always followed by a SELECT to filter meaningful combinations
- **SQL:** CROSS JOIN

Figure 8.5

The CARTESIAN PRODUCT (CROSS PRODUCT) operation.

FEMALE_EMPS									
Fname	Minit	Lname	Ssn	Bdate	Address	Sex	Salary	Super_ssn	Dno
Alicia	J	Zelaya	999887777	1968-07-19	3321 Castle, Spring, TX	F	25000	987654321	4
Jennifer	S	Wallace	987654321	1941-08-20	291 Berry, Bellaire, TX	F	43000	888665555	4
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

EMPNames		
Fname	Lname	Ssn
Alicia	Zelaya	999887777
Jennifer	Wallace	987654321
Joyce	English	453453453

EMP_DEPENDENTS							
Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Alicia	Zelaya	999887777	333445555	Alice	F	1986-04-05	...
Alicia	Zelaya	999887777	333445555	Theodore	M	1983-10-25	...
Alicia	Zelaya	999887777	333445555	Joy	F	1958-05-03	...
Alicia	Zelaya	999887777	987654321	Abner	M	1942-02-28	...
Alicia	Zelaya	999887777	123456789	Michael	M	1988-01-04	...
Alicia	Zelaya	999887777	123456789	Alice	F	1988-12-30	...
Alicia	Zelaya	999887777	123456789	Elizabeth	F	1967-05-05	...
Jennifer	Wallace	987654321	333445555	Alice	F	1986-04-05	...
Jennifer	Wallace	987654321	333445555	Theodore	M	1983-10-25	...
Jennifer	Wallace	987654321	333445555	Joy	F	1958-05-03	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...
Jennifer	Wallace	987654321	123456789	Michael	M	1988-01-04	...
Jennifer	Wallace	987654321	123456789	Alice	F	1988-12-30	...
Jennifer	Wallace	987654321	123456789	Elizabeth	F	1967-05-05	...
Joyce	English	453453453	333445555	Alice	F	1986-04-05	...
Joyce	English	453453453	333445555	Theodore	M	1983-10-25	...
Joyce	English	453453453	333445555	Joy	F	1958-05-03	...
Joyce	English	453453453	987654321	Abner	M	1942-02-28	...
Joyce	English	453453453	123456789	Michael	M	1988-01-04	...
Joyce	English	453453453	123456789	Alice	F	1988-12-30	...
Joyce	English	453453453	123456789	Elizabeth	F	1967-05-05	...

ACTUAL_DEPENDENTS							
Fname	Lname	Ssn	Essn	Dependent_name	Sex	Bdate	...
Jennifer	Wallace	987654321	987654321	Abner	M	1942-02-28	...

RESULT		
Fname	Lname	Dependent_name
Jennifer	Wallace	Abner

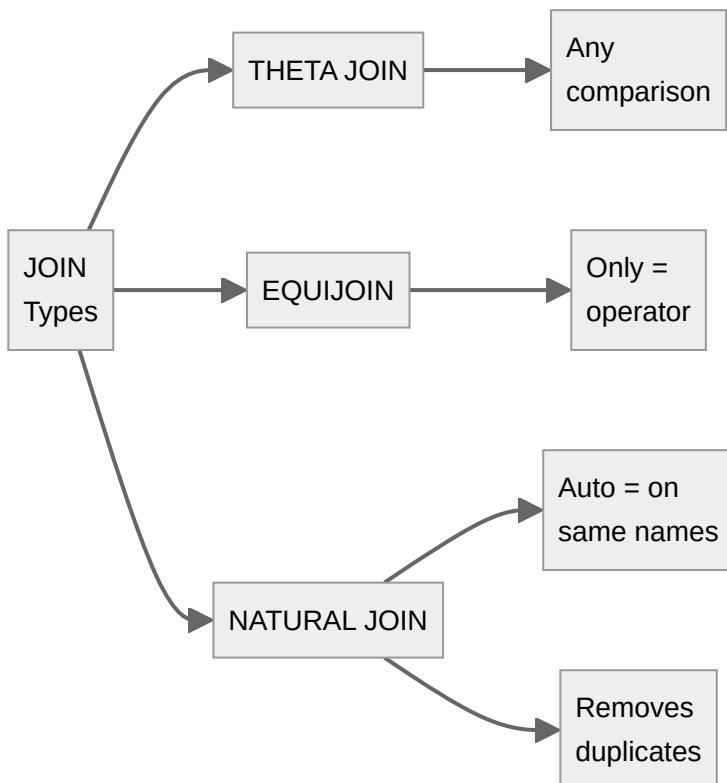
Figure 8.5: The CARTESIAN PRODUCT (CROSS PRODUCT) operation showing FEMALE_EMPS x EMPNames, and how a SELECT filters to get ACTUAL_DEPENDENTS.

Exam Tip

Cartesian Product vs. Join: Cartesian Product creates ALL possible combinations ($n \times m$ tuples). JOIN is a Cartesian Product **followed by a SELECT** that keeps only matching tuples. In practice, always use JOIN instead of raw Cartesian Product.

10. Binary Relational Operations: JOIN

Because CARTESIAN PRODUCT + SELECT is the most common way to combine related data, relational algebra defines a dedicated operation: the **JOIN**.



10.1 The JOIN Operation ($\bowtie$)

Definition

Combines related tuples from two relations into a single "longer" tuple based on a condition. Technically an **Inner Join**.

Notation: $R \bowtie_{\langle \text{join condition} \rangle} S$

- **Resulting Schema:** Degree = degree(R) + degree(S)
- **Mechanism:** Evaluates join condition for every combination; only TRUE combinations included

- **Information Loss Warning:** Tuples without a match, or with NULL in join attributes, are **dropped entirely**

Figure 8.6Result of the JOIN operation $DEPT_MGR \leftarrow DEPARTMENT \bowtie_{Mgr_ssn=Ssn} EMPLOYEE$.**DEPT_MGR**

Dname	Dnumber	Mgr_ssn	...	Fname	Minit	Lname	Ssn	...
Research	5	333445555	...	Franklin	T	Wong	333445555	...
Administration	4	987654321	...	Jennifer	S	Wallace	987654321	...
Headquarters	1	888665555	...	James	E	Borg	888665555	...

Figure 8.6: Result of the JOIN operation $DEPT_MGR = DEPARTMENT JOIN_{Mgr_ssn=Ssn} EMPLOYEE$, combining department data with manager employee data.

10.2 THETA JOIN

The most general form of JOIN.

- Join condition uses clauses like $A_i \theta B_j$ where θ can be any comparison: $=, \neq, <, \leq, >, \geq$
- Compared attributes must share the same domain

10.3 EQUIJOIN

A subset of Theta Join where the only operator is **equality** ($=$).

- **Characteristic:** Result always contains **redundant data** - the join attributes from both relations have identical values in every tuple

10.4 NATURAL JOIN (*)

Definition

Created to eliminate the **redundant attribute** generated by EQUIJOIN. Implicitly equates attributes sharing the same name, then automatically drops one duplicate column.

Notation: $R * S$

- **Renaming Requirement:** If join attributes logically match but have different names, $RENAME(\rho)$ must be applied first

- **Multiple Attributes:** If multiple attributes share names, implicitly joins on **all of them** using AND

(a)

PROJ_DEPT

Pname	Pnumber	Plocation	Dnum	Dname	Mgr_ssn	Mgr_start_date
ProductX	1	Bellaire	5	Research	333445555	1988-05-22
ProductY	2	Sugarland	5	Research	333445555	1988-05-22
ProductZ	3	Houston	5	Research	333445555	1988-05-22
Computerization	10	Stafford	4	Administration	987654321	1995-01-01
Reorganization	20	Houston	1	Headquarters	888665555	1981-06-19
Newbenefits	30	Stafford	4	Administration	987654321	1995-01-01

(b)

DEPT_LOCS

Dname	Dnumber	Mgr_ssn	Mgr_start_date	Location
Headquarters	1	888665555	1981-06-19	Houston
Administration	4	987654321	1995-01-01	Stafford
Research	5	333445555	1988-05-22	Bellaire
Research	5	333445555	1988-05-22	Sugarland
Research	5	333445555	1988-05-22	Houston

Figure 8.7

Results of two natural join operations. (a) $\text{proj_dept} \leftarrow \text{project} \bowtie \text{dept}$.
 (b) $\text{dept_locs} \leftarrow \text{department} \bowtie \text{dept_locations}$.

Figure 8.7: Results of two natural join operations: (a) $\text{PROJ_DEPT} = \text{PROJECT} * \text{DEPARTMENT}$, (b) $\text{DEPT_LOCS} = \text{DEPARTMENT} * \text{DEPT_LOCATIONS}$.

10.5 Join Selectivity and N-Way Joins

- **Join Selectivity Ratio:** Expected join result size divided by theoretical maximum ($|R| \times |S|$). Calculates the fraction of combinations that qualify.
- **N-Way Joins:** Joins can be cascaded across three or more relations (e.g., $(\text{PROJECT} * \text{DEPARTMENT}) * \text{EMPLOYEE}$)
- **SQL Implementations:**
 1. Join conditions in WHERE clause
 2. Explicit syntax: INNER JOIN ... ON ... (preferred in SQL2+)

11. Additional Relational Operations

11.1 Generalized Projection

Definition

Extends standard projection to allow **functions of attributes** (arithmetic operations, constants) in the projection list.

Notation: $\pi_{F_1, F_2, \dots, F_n}(R)$ where F_i are functions.

- Commonly used for report generation with calculated columns
- Example: Computing 'Net Salary' or 'Bonus'

11.2 Aggregate Functions and Grouping ($\mathcal{F}$)

Definition

Groups tuples and applies statistical summaries independently to each group. Preserves **closure property** of relational algebra (result is a relation, not a scalar).

Notation: $\langle \text{grouping attributes} \rangle \mathcal{F}_{\langle \text{function list} \rangle}(R)$

Resulting Schema:

- New schema contains grouping attributes + one new attribute per computed function
- **Implicit Renaming:** If no explicit renaming, computed attributes get names like COUNT_Ssn or AVERAGE_Salary
- **NULL Handling:** NULL values are **entirely ignored** during aggregation
- **Duplicates:** Not eliminated before aggregation (so SUM/AVERAGE calculate correctly)

Common Aggregate Functions: SUM, AVERAGE, MAXIMUM, MINIMUM, COUNT

R

(a)

Dno	No_of_employees	Average_sal
5	4	33250
4	3	31000
1	1	55000

(b)

Dno	Count_ssn	Average_salary
5	4	33250
4	3	31000
1	1	55000

(c)

Count_ssn	Average_salary
8	35125

Figure 8.10

The aggregate function operation.

- a. $\text{PR}(\text{Dno}, \text{No_of_employees}, \text{Average_sal}) (\text{Dno} \bowtie \text{COUNT Ssn, AVERAGE Salary (EMPLOYEE)})$.
 b. $\text{Dno} \bowtie \text{COUNT Ssn, AVERAGE Salary (EMPLOYEE)}$.
 c. $\bowtie \text{COUNT Ssn, AVERAGE Salary (EMPLOYEE)}$.

Figure 8.10: The aggregate function operation: (a) Group by Dno with COUNT and AVERAGE, (b) Same with renamed output, (c) No grouping - overall aggregates.

11.3 OUTER JOIN Operations

Standard joins (Inner Joins) lose information if tuples lack a matching counterpart. **Outer joins** keep unmatched tuples and pad with NULL.

Table 4: Types of Outer Join

Type	Keeps	Missing Data
LEFT OUTER JOIN	All tuples from left relation R	Columns from S padded with NULLs
RIGHT OUTER JOIN	All tuples from right relation S	Columns from R padded with NULLs
FULL OUTER JOIN	All tuples from both R and S	Missing columns padded with NULLs

Figure 8.12

The result of a LEFT OUTER JOIN operation.

RESULT

Fname	Minit	Lname	Dname
John	B	Smith	NULL
Franklin	T	Wong	Research
Alicia	J	Zelaya	NULL
Jennifer	S	Wallace	Administration
Ramesh	K	Narayan	NULL
Joyce	A	English	NULL
Ahmad	V	Jabbar	NULL
James	E	Borg	Headquarters

Figure 8.12: The result of a LEFT OUTER JOIN operation. Every employee appears; those without departments show NULL in Dname.

11.4 The OUTER UNION Operation

Definition

Takes the union of tuples from two relations that share **some common attributes** but are not fully union compatible (partially compatible).

- Operates equivalently to a **FULL OUTER JOIN** on common attributes
- Attributes present in only one relation are kept; missing attributes padded with NULL
- **Caution:** Shared attribute names must mean the exact same real-world concept

12. Examples of Queries in Relational Algebra

All queries assume a standard **COMPANY database** with tables: EMPLOYEE, DEPARTMENT, PROJECT, WORKS_ON, and DEPENDENT.

Query 1: Selection, Join, and Projection

Problem

Retrieve the name and address of all employees who work for the 'Research' department.

Step-by-Step:

```
RESEARCH_DEPT <- sigma_Dname='Research' (DEPARTMENT)
RESEARCH_EMPS <- RESEARCH_DEPT JOIN Dnumber=Dno EMPLOYEE
RESULT <- pi_Fname,Lname,Address (RESEARCH_EMPS)
```

In-line: $\pi_{Fname,Lname,Address}(\sigma_{Dname='Research'}(DEPARTMENT) \bowtie_{Dnumber=Dno} EMPLOYEE)$

Query 2: Cascading Joins

Problem

For every project in 'Stafford', list project number, controlling department number, and manager's last name, address, and birth date.

```
STAFFORD_PROJS <- sigma_Plocation='Stafford' (PROJECT)
CONTR_DEPTS <- STAFFORD_PROJS JOIN Dnum=Dnumber DEPARTMENT
PROJ_DEPT_MGRS <- CONTR_DEPTS JOIN Mgr_ssn=Ssn EMPLOYEE
RESULT <- pi_Pnumber,Dnum,Lname,Address,Bdate (PROJ_DEPT_MGRS)
```

Query 3: Using the DIVISION Operator ($\div$)

Problem

Find names of employees who work on **all** projects controlled by department 5.

```
DEPT5_PROJS <- pi_Pnumber(sigma_Dnum=5(PROJECT))
EMP_PROJ <- pi_Ssn,Pno(WORKS_ON)
RESULT_EMP_SSNS <- EMP_PROJ DIVIDE BY DEPT5_PROJS
RESULT <- pi_Fname,Lname(RESULT_EMP_SSNS JOIN Ssn=Ssn EMPLOYEE)
```

Exam Tip

The **DIVISION** operator is specifically useful for queries involving **universal quantification** (the word "all"). If you see "find employees who work on ALL projects..." think DIVISION.

Query 4: Using the UNION Operator ($\cup$)

Problem

List project numbers involving an employee named 'Smith', either as a worker OR as a manager.

```
SMITH <- sigma_Lname='Smith'(EMPLOYEE)
SMITH_WORKER <- pi_Pno(SMITH JOIN Ssn=Essn WORKS_ON)
MGRS <- pi_Lname,Dnumber(SMITH JOIN Ssn=Mgr_ssn DEPARTMENT)
SMITH_MANAGES <- pi_Pnumber(MGRS JOIN Dnumber=Dnum PROJECT)
RESULT <- SMITH_WORKER UNION SMITH_MANAGES
```

Query 5: Using Aggregate Functions ($\mathcal{F}$)

Problem

List names of employees with two or more dependents.

```

EMP_DEPENDENT_COUNT <- _Ssn F_COUNT Dependent_name(DEPENDENT)
RESULT_EMPS <- sigma_Count_dep>=2(EMP_DEPENDENT_COUNT)
RESULT <- pi_Fname,Lname(RESULT_EMPS JOIN Ssn=Ssn EMPLOYEE)

```

Query 6: Using the MINUS Operator (−)

Problem

Retrieve names of employees who have **no** dependents.

```

ALL_EMPS <- pi_Ssn(EMPLOYEE)
EMPS_WITH_DEPS <- pi_Essn(DEPENDENT)
EMPS_WITHOUT <- ALL_EMPS - EMPS_WITH_DEPS
RESULT <- pi_Fname,Lname(EMPS_WITHOUT JOIN Ssn=Ssn EMPLOYEE)

```

SUPERVISION

(Borg's Ssn is 888665555)
(Ssn) (Super_ssn)

Ssn1	Ssn2
123456789	333445555
333445555	888665555
999887777	987654321
987654321	888665555
666884444	333445555
453453453	333445555
987987987	987654321
888665555	null

RESULT1

Ssn
333445555
987654321

(Supervised by Borg)

RESULT2

Ssn
123456789
999887777
666884444
453453453
987987987

(Supervised by
Borg's subordinates)

RESULT

Ssn
123456789
999887777
666884444
453453453
987987987
333445555
987654321

(RESULT1 $\cup$ RESULT2)

Figure 8.11

A two-level recursive query.

Figure 8.11: A two-level recursive query on SUPERVISION. Finds all employees supervised by Borg directly (RESULT1) and through subordinates (RESULT2), then UNIONs them.

Query 7: Using the INTERSECTION Operator ($\cap$)

12. Examples of Queries in Relational Algebra

```
MGRS <- pi_Mgr_ssn(DEPARTMENT)
EMPS_WITH_DEPS <- pi_Essn(DEPENDENT)
MGRS_WITH_DEPS <- MGRS INTERSECT EMPS_WITH_DEPS
RESULT <- pi_Fname,Lname(MGRS_WITH_DEPS JOIN Ssn=Ssn EMPLOYEE)
```

13. Exam Questions & Answers

Q1. What is the difference between a relation schema and a relation state?

A: A **relation schema** (intension) is the structural blueprint of a relation - it defines the relation name, attributes, and their domains. It is relatively **static**. A **relation state** (extension) is the actual data populated in the relation at a specific moment - it is the set of tuples. It **changes frequently** as data is inserted, updated, or deleted.

Q2. Explain the First Normal Form (1NF) assumption.

A: 1NF requires that every value in a tuple be **atomic** (indivisible). This means composite attributes and multivalued attributes are prohibited in the basic relational model. Composite attributes must be broken into simple components, and multivalued attributes require separate relations.

Q3. What are the three possible meanings of NULL?

A: (1) Value **exists but is unknown**, (2) Value **exists but is not available**, (3) Attribute **does not apply** to this tuple (undefined). NULLs should be minimized as they cause ambiguity in comparisons and aggregations.

Q4. Differentiate between Superkey and Key with an example.

A: A **superkey** is any set of attributes that uniquely identifies tuples (may contain redundant attributes). A **key** is a **minimal superkey** - removing any attribute destroys the uniqueness property. Example: In a CAR relation, {License_number} is a key. {License_number, Engine_serial_number} is a superkey but not a key (it's not minimal). {License_number, Color} is also a superkey but not minimal.

Q5. What is the difference between a Candidate Key and a Primary Key?

A: A relation can have multiple keys called **candidate keys**. The **primary key** is the one candidate key chosen to uniquely identify tuples. Remaining candidate keys are called **unique keys**. Primary key attributes are underlined in schema notation. Best practice: choose the key with fewest attributes.

Q6. State and explain the Entity Integrity Constraint.

A: No **primary key value can be NULL**. The primary key is used to identify tuples. If it were NULL, the tuple would be unidentifiable and impossible to reference securely. This constraint is enforced by the DBMS.

Q7. Define Foreign Key and explain Referential Integrity.

A: A **foreign key** is a set of attributes in a referencing relation that references the primary key of a referenced relation. **Referential Integrity** states that a foreign key value must either match an existing primary key value in the referenced relation OR be NULL. This ensures consistency among related tuples.

Q8. What are the three categories of constraints?

A: (1) **Inherent/Implicit** - built into the model (no duplicate tuples), (2) **Schema-based/Explicit** - expressed via DDL (domain constraints, key constraints, integrity constraints), (3) **Application-based/Semantic** - business rules too complex for DDL, enforced via triggers or application logic.

Q9. List the violation handling options for DELETE and explain each.

A: (1) **RESTRICT** (default) - reject deletion if referenced, (2) **CASCADE** - automatically delete referencing tuples, (3) **SET NULL** - set foreign key to NULL, (4) **SET DEFAULT** - replace with a default reference. SET NULL cannot be used if the FK is part of a primary key (would violate entity integrity).

Q10. What constraint violations can an INSERT operation cause?

A: INSERT can violate **all four** major constraints: (1) **Domain** - wrong data type or out of range, (2) **Key** - duplicate key value, (3) **Entity Integrity** - NULL in primary key, (4) **Referential Integrity** - foreign key references non-existent tuple. Default handling: reject the insertion.

Q11. Define Transaction and explain Atomicity.

A: A **transaction** is a logical, atomic unit of work against the database. **Atomicity** means the transaction is treated as an indivisible unit - either all operations succeed and are committed, or none are applied (rolled back). The database must be left in a valid, consistent state.

Q12. What is the difference between SELECT and PROJECT in relational algebra?

A: **SELECT** (σ) is a **horizontal** filter - it selects **rows** (tuples) that satisfy a condition, keeping all columns. Notation: $\sigma_{\text{condition}}(R)$. **PROJECT** (π) is a **vertical** filter - it selects **columns** (attributes), discarding the rest, and removes duplicate tuples. Notation: $\pi_{\text{attrs}}(R)$.

Q13. Explain Union Compatibility. What operations require it?

A: Two relations are **union compatible** if: (1) they have the same degree, and (2) corresponding attributes share the same domain. Required for: **UNION**, **INTERSECTION**, and **SET DIFFERENCE**. **CARTESIAN PRODUCT** and **JOIN** do NOT require union compatibility.

Q14. Differentiate between EQUIJOIN and NATURAL JOIN.

A: **EQUIJOIN** uses only the equality operator ($=$) in its join condition. The result contains **redundant data** - both join attributes appear with identical values. **NATURAL JOIN** ($*$) is a special **EQUIJOIN** that automatically: (1) equates attributes with the same name, (2) **removes the duplicate column** from the result.

Q15. What are the types of OUTER JOIN? Explain each.

A: (1) **LEFT OUTER JOIN** - keeps all tuples from the left relation, pads missing right columns with NULL, (2) **RIGHT OUTER JOIN** - keeps all tuples from the right relation, pads missing left columns with NULL, (3) **FULL OUTER JOIN** - keeps all tuples from both relations, pads missing columns with NULL. Essential when complete datasets are needed.

Q16. When is the DIVISION operator used? Give an example scenario.

A: The **DIVISION** operator ($\div$) is used for queries involving **universal quantification** - when you need to find items related to "**all**" items in another set. Example: "Find employees who work on ALL projects controlled by department 5." This cannot be expressed using only basic operations.

Q17. Explain Aggregate Functions with grouping in relational algebra.

A: Notation: $\langle \text{group attrs} \rangle \mathcal{F}_{\langle \text{functions} \rangle}(R)$. Groups tuples by specified attributes, then applies aggregate functions (SUM, COUNT, AVG, MAX, MIN) to each group. The result is a **relation** (preserving closure), not a scalar. NULL values are ignored. Without grouping attributes, computes overall aggregates.

Q18. What is the Closed World Assumption?

A: The **Closed World Assumption** is a logical principle stating that the only true facts in the universe are those physically present within the relation extensions. Any combination of values not present in the relation makes the corresponding predicate false. This is fundamental to how relational databases interpret negation.

Q19. Explain the RENAME operator with all its forms.

A: The **RENAME** operator (ρ) has three forms: (1) $\rho_{S(B_1, \dots, B_n)}(R)$ - rename both relation and attributes, (2) $\rho_S(R)$ - rename relation only, (3) $\rho_{(B_1, \dots, B_n)}(R)$ - rename attributes only. Essential when performing Natural Join on attributes with different names, or when avoiding naming conflicts.

Q20. Compare and contrast DELETE constraint violations with INSERT.

A: **INSERT** can violate ALL four constraints (Domain, Key, Entity Integrity, Referential Integrity). **DELETE** can ONLY violate **Referential Integrity** - when the deleted tuple is referenced by foreign keys. This is because deleting removes data; it cannot create invalid data types, duplicate keys, or NULL primary keys.

Quick Reference - Exam Cheat Sheet

Operation	Symbol	Type	What It Does
SELECT	σ	Unary	Filters rows (horizontal)
PROJECT	π	Unary	Filters columns (vertical), removes duplicates
RENAME	ρ	Unary	Renames relation/attributes
UNION	$\cup$	Set	All tuples in either relation
INTERSECT	$\cap$	Set	Common tuples only
MINUS	$-$	Set	In R but not in S
CARTESIAN PRODUCT	$\times$	Set	All combinations
JOIN	$\bowtie$	Binary	Product + Select condition
NATURAL JOIN	$*$	Binary	Equijoin + remove duplicates
DIVISION	$\div$	Binary	Universal quantification ("all")



DATABASE MANAGEMENT SYSTEMS

Unit 4

SQL Data Definition, Data Types, Constraints & Advanced Queries

Complete Exam Notes

With Diagrams, Examples & Q&A



Table of Contents

1. Introduction to SQL Data Definition

2. Schema and Catalog Concepts in SQL

2.1 SQL Schema

2.2 SQL Catalog

3. The CREATE TABLE Command

3.1 Syntax and Structure

3.2 Base Tables vs. Virtual Tables

3.3 Ordering

3.4 Handling Circular and Forward References

4. Attribute Data Types and Domains

4.1 Numeric Data Types

4.2 Character-String Data Types

4.3 Bit-String Data Types

4.4 Boolean Data Type

4.5 Temporal Data Types

4.6 Domains and User-Defined Types

5. Specifying Constraints in SQL

5.1 Attribute Constraints and Defaults

5.2 Domain Constraints (CHECK)

5.3 Key Constraints

5.4 Referential Integrity (FOREIGN KEY)

5.5 Naming Constraints

5.6 Tuple-Based Constraints

6. Basic Retrieval Queries in SQL

6.1 SELECT-FROM-WHERE Structure

6.2 Aliasing and Tuple Variables

6.3 Asterisk and WHERE Clauses

6.4 DISTINCT vs. ALL

6.5 Set Operations

6.6 Pattern Matching (LIKE)

6.7 Arithmetic and BETWEEN

6.8 ORDER BY

7. Data Manipulation (INSERT, DELETE, UPDATE)

8. Complex SQL Retrieval Queries

8.1 NULL Values and Three-Valued Logic

8.2 Nested Queries and IN

8.3 Correlated Nested Queries

8.4 Explicit Sets and Aliasing

8.5 Joined Tables and Outer Joins

8.6 Aggregate Functions

8.7 GROUP BY and HAVING

9. Views in SQL

10. Schema Modification Statements

11. Key Diagrams Summary

12. Exam Q&A

1. Introduction to SQL Data Definition

SQL (Structured Query Language) is the primary language for interacting with relational databases. The terminology in SQL differs slightly from formal relational model terms:

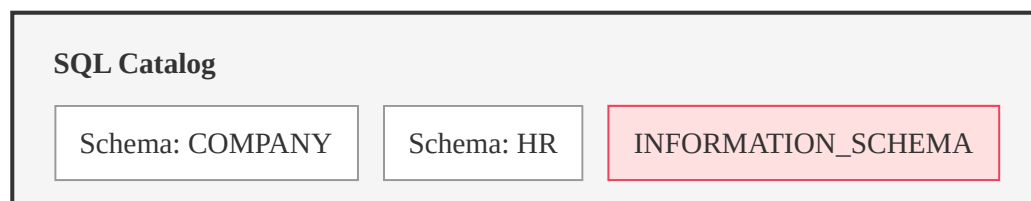
Table 1: SQL vs Relational Model Terminology

SQL Term	Relational Model Term
Table	Relation
Row	Tuple
Column	Attribute

Data Definition Language (DDL): The core SQL command for data definition is the `CREATE` statement. It is used to define schemas, tables (relations), types, domains, views, assertions, and triggers.

2. Schema and Catalog Concepts in SQL

Modern SQL uses **schemas** and **catalogs** to group and manage database objects hierarchically.



2.1 SQL Schema

- **Definition:** A grouping of tables and other constructs belonging to the same database application.
- **Components:** Identified by a schema name and includes an **authorization identifier** (owner account). Contains descriptors for tables, types, constraints, views, and domains.
- **Creation:**

```
CREATE SCHEMA COMPANY AUTHORIZATION 'Jsmith';
```

- **Privileges:** The ability to create schemas must be explicitly granted by the Database Administrator (DBA).

2.2 SQL Catalog

- **Definition:** A named collection of schemas.
- **Environment:** Database installations have a default environment and schema. Users can reference tables in their default schema without explicitly naming it.
- **INFORMATION_SCHEMA:** A special mandatory schema within every catalog containing metadata (information on all schemas and element descriptors).
- **Cross-Referencing:** Integrity constraints can only be defined between relations in schemas within the same catalog.

3. The CREATE TABLE Command

The `CREATE TABLE` statement specifies a new relation by providing its name, attributes, and initial constraints.

3.1 Syntax and Structure

- Attributes are specified with a **name**, a **data type (domain)**, and optional **constraints** (e.g., `NOT NULL`).
- Key, entity integrity, and referential integrity constraints can be specified with the attribute declaration or added later via `ALTER TABLE`.
- **Explicit Schema Attachment:** Use dot notation: `CREATE TABLE COMPANY.EMPLOYEE ...`

CREATE TABLE EMPLOYEE			Figure 6.1 SQL CREATE TABLE data definition statement for defining the COMPANY schema from Figure 5.7.
(Fname	VARCHAR(15)	NOT NULL,	
Minit	CHAR,		
Lname	VARCHAR(15)	NOT NULL,	
Ssn	CHAR(9)	NOT NULL,	
Bdate	DATE,		
Address	VARCHAR(30),		
Sex	CHAR,		
Salary	DECIMAL(10,2),		
Super_ssn	CHAR(9),		
Dno	INT	NOT NULL,	
PRIMARY KEY (Ssn),			
CREATE TABLE DEPARTMENT			
(Dname	VARCHAR(15)	NOT NULL,	
Dnumber	INT	NOT NULL,	
Mgr_ssn	CHAR(9)	NOT NULL,	
Mgr_start_date	DATE,		
PRIMARY KEY (Dnumber),			
UNIQUE (Dname),			
FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn) ;			
CREATE TABLE DEPT_LOCATIONS			
(Dnumber	INT	NOT NULL,	
Dlocation	VARCHAR(15)	NOT NULL,	
PRIMARY KEY (Dnumber, Dlocation),			
FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber) ;			
CREATE TABLE PROJECT			
(Pname	VARCHAR(15)	NOT NULL,	
Pnumber	INT	NOT NULL,	
Plocation	VARCHAR(15),		
Dnum	INT	NOT NULL,	
PRIMARY KEY (Pnumber),			
UNIQUE (Pname),			
FOREIGN KEY (Dnum) REFERENCES DEPARTMENT(Dnumber) ;			
CREATE TABLE WORKS_ON			
(Essn	CHAR(9)	NOT NULL,	
Pno	INT	NOT NULL,	
Hours	DECIMAL(3,1)	NOT NULL,	
PRIMARY KEY (Essn, Pno),			
FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn),			
FOREIGN KEY (Pno) REFERENCES PROJECT(Pnumber) ;			
CREATE TABLE DEPENDENT			
(Essn	CHAR(9)	NOT NULL,	
Dependent_name	VARCHAR(15)	NOT NULL,	
Sex	CHAR,		
Bdate	DATE,		
Relationship	VARCHAR(8),		
PRIMARY KEY (Essn, Dependent_name),			
FOREIGN KEY (Essn) REFERENCES EMPLOYEE(Ssn) ;			

Figure 1: SQL CREATE TABLE statements defining the COMPANY database schema (Figure 6.1)

3.2 Base Tables vs. Virtual Tables

Table 2: Base Tables vs Virtual Tables

Property	Base Table	Virtual Table (View)
Created via	CREATE TABLE	CREATE VIEW
Physical storage	Physically stored as a file	May or may not have physical file
Data storage	Stores actual data	Stores only query definition

3.3 Ordering

- **Attributes (Columns):** Ordered in the exact sequence specified in CREATE TABLE .
- **Tuples (Rows):** Unordered within the table.

3.4 Handling Circular and Forward References

4. Attribute Data Types and Domains

SQL provides a robust set of basic data types for attribute specification.

SQL Data Types

Numeric

Character

Bit-String

Boolean

Temporal

Domains / UDT

4.1 Numeric Data Types

Table 3: Numeric Data Types in SQL

Category	Data Type	Description
Integer	INTEGER / INT	Standard integer
	SMALLINT	Smaller range integer
Floating-point	FLOAT / REAL	Single precision
	DOUBLE PRECISION	Double precision
Formatted	DECIMAL(i, j) / NUMERIC(i, j)	i = total digits, j = digits after decimal

4.2 Character-String Data Types

- **Fixed-length:** CHAR(n) or CHARACTER(n) . Shorter strings are right-padded with blanks.
- **Varying-length:** VARCHAR(n) , CHAR VARYING(n) , or CHARACTER VARYING(n) .
- **Characteristics:** Enclosed in single quotes (e.g., 'Smith'), case-sensitive, ordered lexicographically.
- **Concatenation:** || operator (e.g., 'abc' || 'XYZ' yields 'abcXYZ').
- **Large Objects:** CLOB(20M) for massive text values.

4.3 Bit-String Data Types

- **Fixed-length:** BIT(n)
- **Varying-length:** BIT VARYING(n)
- **Format:** Enclosed in single quotes, preceded by B (e.g., B'10101').
- **Large Objects:** BLOB for binary data like images.

4.4 Boolean Data Type

Uses **three-valued logic** due to SQL's handling of NULLs. Possible values: `TRUE` , `FALSE` , and `UNKNOWN` .

4.5 Temporal Data Types

Table 4: Temporal Data Types

Type	Format	Notes
<code>DATE</code>	<code>YYYY-MM-DD</code>	10 positions, strictly validated
<code>TIME</code>	<code>HH:MM:SS</code>	8+ positions; <code>TIME(i)</code> adds fractional seconds
<code>TIME WITH TIME ZONE</code>	<code>+HH:MM</code>	6 extra positions for UTC displacement
<code>TIMESTAMP</code>	Date + Time + fractions	Combines <code>DATE</code> and <code>TIME</code>
<code>INTERVAL</code>	Relative value	Used to increment/decrement dates

Exam Tip

Temporal values are string-like, enclosed in single quotes and prefixed by their keyword: `DATE '2014-09-27'` , `TIME '09:12:47'` . They can be compared using standard operators.

4.6 Domains and User-Defined Types (UDTs)

Domain Creation

```
CREATE DOMAIN SSN_TYPE AS CHAR(9);
```

Can be used in place of standard types (e.g., `Ssn SSN_TYPE`) and can include `DEFAULT` values.

- **User-Defined Types:** Created via `CREATE TYPE` (object-relational SQL) to serve as attribute types or basis for tables.

5. Specifying Constraints in SQL

Constraints maintain database integrity and restrict data to valid parameters.

SQL Constraints**Attribute**NOT NULL, DEFAULT,
CHECK**Table**PRIMARY KEY, FOREIGN KEY, UNIQUE,
CHECK**Schema**ASSERTION,
TRIGGER

5.1 Attribute Constraints and Defaults

Table 5: Attribute-Level Constraints

Constraint	Purpose	Example
NOT NULL	Prevents NULL values	Dno INT NOT NULL
DEFAULT	Automatic fallback value	Dno INT NOT NULL DEFAULT 1

5.2 Domain Constraints (CHECK)

Used to restrict permitted values for attributes or domains:

```
Dnumber INT NOT NULL CHECK (Dnumber > 0 AND Dnumber < 21);
CREATE DOMAIN D_NUM AS INTEGER CHECK (D_NUM > 0 AND D_NUM < 21);
```

5.3 Key Constraints

Table 6: Key Constraints Comparison

Property	PRIMARY KEY	UNIQUE
Null values allowed	No	Yes (one NULL typically)
Number per table	Only one	Multiple allowed
Purpose	Main unique identifier	Alternate/candidate keys
Inline (single attr)	Dnumber INT PRIMARY KEY	Ssn CHAR(9) UNIQUE
End of table	PRIMARY KEY (Dnumber, Dlocation)	UNIQUE (Dname)

5.4 Referential Integrity Constraints (FOREIGN KEY)

Connects a foreign key in the referencing table to a primary/unique key in the referenced table.

```

CREATE TABLE EMPLOYEE
(
    ...,
    Dno INT NOT NULL DEFAULT 1,
    CONSTRAINT EMPCHK
    PRIMARY KEY (Ssn),
    CONSTRAINT EMPSUPERFK
    FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
    ON DELETE SET NULL ON UPDATE CASCADE,
    CONSTRAINT EMPDEPTFK
    FOREIGN KEY (Dno) REFERENCES DEPARTMENT(Dnumber)
    ON DELETE SET DEFAULT ON UPDATE CASCADE);
CREATE TABLE DEPARTMENT
(
    ...,
    Mgr_ssn CHAR(9) NOT NULL DEFAULT '888665555',
    ...,
    CONSTRAINT DEPTPK
    PRIMARY KEY (Dnumber),
    CONSTRAINT DEPTSK
    UNIQUE (Dname),
    CONSTRAINT DEPTMGRFK
    FOREIGN KEY (Mgr_ssn) REFERENCES EMPLOYEE(Ssn)
    ON DELETE SET DEFAULT ON UPDATE CASCADE);
CREATE TABLE DEPT_LOCATIONS
(
    ...,
    PRIMARY KEY (Dnumber, Dlocation),
    FOREIGN KEY (Dnumber) REFERENCES DEPARTMENT(Dnumber)
    ON DELETE CASCADE ON UPDATE CASCADE);

```

Figure 6.2
Example illustrating how default attribute values and referential integrity triggered actions are specified in SQL.

Figure 2: Specifying default values and referential integrity triggered actions (Figure 6.2)

Table 7: Referential Triggered Actions

Action	ON DELETE	ON UPDATE
RESTRICT	Rejects delete causing violation (default)	Rejects update causing violation (default)
SET NULL	Sets referencing attr to NULL	Sets referencing attr to NULL
SET DEFAULT	Sets referencing attr to default	Sets referencing attr to default
CASCADE	Deletes all referencing tuples	Updates FK in all referencing tuples

Exam Tip

CASCADE is ideal for weak entities or relationship tables (e.g., WORKS_ON). Use SET NULL when the relationship is optional.

```

FOREIGN KEY (Super_ssn) REFERENCES EMPLOYEE(Ssn)
ON DELETE SET NULL
ON UPDATE CASCADE;

```

5.5 Giving Names to Constraints

Using the CONSTRAINT keyword to assign unique names for later modification:

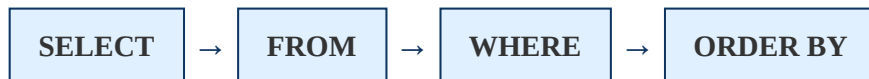
5.6 Tuple-Based Constraints (Row-level CHECK)

Constraints applied at the end of `CREATE TABLE` that compare attributes within the same row:

```
CHECK (Dept_create_date <= Mgr_start_date)
```

6. Basic Retrieval Queries in SQL

SQL retrieves information primarily via the `SELECT` statement. In practical SQL, a table is treated as a **multiset (bag)**, meaning duplicates are allowed by default.



columns tables condition sorting

6.1 The SELECT-FROM-WHERE Structure

Basic Query Structure

```
SELECT <attribute list>
FROM   <table list>
WHERE  <condition>;
```

- **SELECT:** Projection attributes (which columns)
- **FROM:** Relations needed (which tables)
- **WHERE:** Boolean condition (selection/join condition)

Comparison Operators: `=`, `<`, `<=`, `>`, `>=`, `<>` (not equal)

Figure 6.3

Results of SQL queries when applied to the COMPANY database state shown in Figure 5.6. (a) Q0. (b) Q1. (c) Q2. (d) Q8. (e) Q9. (f) Q10. (g) Q1C.

(a)

Bdate	Address
1965-01-09	731Fondren, Houston, TX

(b)

Fname	Lname	Address
John	Smith	731 Fondren, Houston, TX
Franklin	Wong	638 Voss, Houston, TX
Ramesh	Narayan	975 Fire Oak, Humble, TX
Joyce	English	5631 Rice, Houston, TX

(c)

Pnumber	Dnum	Lname	Address	Bdate
10	4	Wallace	291Berry, Bellaire, TX	1941-06-20
30	4	Wallace	291Berry, Bellaire, TX	1941-06-20

(f)

Sen	Dname
123456789	Research
333445555	Research
999887777	Research
987654321	Research
666884444	Research
453453453	Research
987987987	Research
888665555	Research
123456789	Administration
333445555	Administration
999887777	Administration
987654321	Administration
666884444	Administration
453453453	Administration
987987987	Administration
888665555	Administration
123456789	Headquarters
333445555	Headquarters
999887777	Headquarters
987654321	Headquarters
666884444	Headquarters
453453453	Headquarters
987987987	Headquarters
888665555	Headquarters

(d)

E.Fname	E.Lname	S.Fname	S.Lname
John	Smith	Franklin	Wong
Franklin	Wong	James	Borg
Alicia	Zelaya	Jennifer	Wallace
Jennifer	Wallace	James	Borg
Ramesh	Narayan	Franklin	Wong
Joyce	English	Franklin	Wong
Ahmad	Jabbar	Jennifer	Wallace

(e)

E.Fname
123456789
333445555
999887777
987654321
666884444
453453453
987987987
888665555

(g)

Fname	Minit	Lname	Sen	Bdate	Address	Sex	Salary	Super_ssn	Dno
John	B	Smith	123456789	1965-09-01	731 Fondren, Houston, TX	M	30000	333445555	5
Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5

Figure 3: Results of SQL queries applied to the COMPANY database state (Figure 6.3)

Figure 6.4

Results of additional SQL queries when applied to the COMPANY database state shown in Figure 5.6. (a) Q11. (b) Q11A. (c) Q16. (d) Q18.

(a)

Salary
30000
40000
25000
43000
38000
25000
25000
55000

(b)

Salary
30000
40000
25000
43000
38000
55000

(c)

Fname	Lname

(d)

Fname	Lname
James	Borg

Figure 4: Additional SQL query results on the COMPANY database (Figure 6.4)

6.2 Aliasing and Tuple Variables

When querying multiple tables with identical attribute names:

- **Fully Qualified Names:** Prefix table name: `EMPLOYEE.Fname`
- **Aliasing (AS keyword):** Create shorthand names:

```
SELECT E.Fname, E.Lname, S.Fname AS Supervisor_Fname
FROM EMPLOYEE AS E, EMPLOYEE AS S
```

- **Missing WHERE:** Generates the **CROSS PRODUCT** (Cartesian product) when multiple tables are listed.

6.4 Tables as Sets vs. Multisets (DISTINCT vs. ALL)

Table 8: DISTINCT vs ALL

Keyword	Behavior
SELECT ALL (default)	Retains duplicate tuples
SELECT DISTINCT	Eliminates duplicate tuples

6.5 Set Operations

Require **type-compatible** queries (same number of attributes, same data types in order).

Table 9: Set Operations in SQL

Operation	Eliminates Duplicates?	Keeps Duplicates?
Union	UNION	UNION ALL
Difference	EXCEPT	EXCEPT ALL
Intersection	INTERSECT	INTERSECT ALL

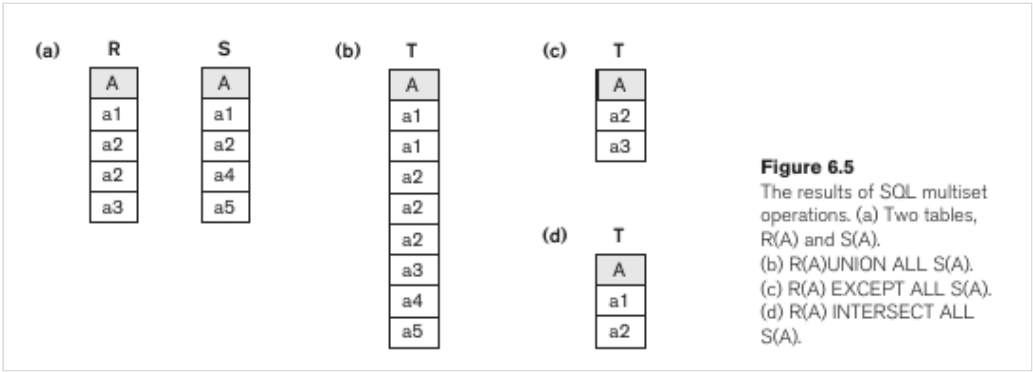


Figure 5: Results of SQL multiset operations: UNION ALL, EXCEPT ALL, INTERSECT ALL (Figure 6.5)

6.6 Substring Pattern Matching (LIKE)

Table 10: LIKE Wildcard Characters

Wildcard	Meaning	Example
%	Replaces any number of characters (0 or more)	'Smith%' matches Smith, Smithson

_

Replaces exactly one character

'Smi_h' matches Smith, Smigh

```
-- Escape for literal % or _
LIKE 'AB\_CD\%EF' ESCAPE '\'
```

6.7 Arithmetic and Range Operators

- **Arithmetic:** +, -, *, / in SELECT clause
- **BETWEEN:** Shorthand for inclusive range:

```
WHERE Salary BETWEEN 30000 AND 50000
-- Equivalent to: WHERE Salary >= 30000 AND Salary <= 50000
```

6.8 ORDER BY

Determines sorting of output tuples:

```
SELECT Fname, Lname, Salary FROM EMPLOYEE
WHERE Dno = 5
ORDER BY Salary DESC, Lname ASC;
```

Exam Tip

Mandatory order of clauses: SELECT -> FROM -> WHERE -> ORDER BY . You cannot reorder these!

7. Data Manipulation in SQL (INSERT, DELETE, UPDATE)

DML Commands

INSERT

Add rows

DELETE

Remove rows

UPDATE

Modify rows

7.1 The INSERT Command

INSERT Variations

```
-- Simple insertion (all columns)
INSERT INTO EMPLOYEE VALUES ('John','B','Smith',...);

-- Explicit columns
INSERT INTO EMPLOYEE (Fname, Lname, Ssn) VALUES
('John','Smith','123456789');

-- Multiple rows
INSERT INTO DEPARTMENT VALUES ('Research',5,'333445555','1986-05-22'),
                                ('Admin',4,'987654321','1995-01-01');

-- Insert from query (bulk)
INSERT INTO WORKS_ON_INFO (Emp_name, Proj_name, Hours)
SELECT E.Lname, P.Pname, W.Hours
FROM PROJECT P, WORKS_ON W, EMPLOYEE E
WHERE ...;
```

Important

Attributes not listed get their `DEFAULT` value or `NULL`. You cannot omit attributes that have `NOT NULL` without a default value.

7.2 The DELETE Command

```
-- Targeted deletion
DELETE FROM EMPLOYEE WHERE Lname = 'Smith';

-- Mass deletion (empties the table but keeps structure)
DELETE FROM EMPLOYEE;
```

7.3 The UPDATE Command

```
-- Modify specific tuples
UPDATE EMPLOYEE SET Salary = Salary * 1.1 WHERE Dno = 5;

-- Modify with condition
UPDATE EMPLOYEE SET Super_ssn = '333445555' WHERE Ssn = '999887777';
```

8. More Complex SQL Retrieval Queries

8.1 NULL Values and Three-Valued Logic

NULL values represent three scenarios:

1. **Unknown Value:** Exists but not known (e.g., date of birth)
2. **Unavailable/Withheld:** Known but hidden (e.g., unlisted phone)
3. **Not Applicable:** Attribute doesn't apply (e.g., degree for non-college attendee)

Each NULL is treated as **distinct** from every other NULL. Comparisons with NULL result in **UNKNOWN**.

Table 11: Logical Connectives in Three-Valued Logic

AND	TRUE	FALSE	UNKNOWN
TRUE	TRUE	FALSE	UNKNOWN
FALSE	FALSE	FALSE	FALSE
UNKNOWN	UNKNOWN	FALSE	UNKNOWN

OR	TRUE	FALSE	UNKNOWN
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	UNKNOWN
UNKNOWN	TRUE	UNKNOWN	UNKNOWN

NOT	Result
TRUE	FALSE
FALSE	TRUE
UNKNOWN	UNKNOWN

Critical Rule

WHERE clause only selects tuples that evaluate to **TRUE**. Tuples evaluating to **FALSE** or **UNKNOWN** are excluded. Use `IS NULL` or `IS NOT NULL` to check for NULLs (`NOT = NULL`).

```
SELECT Fname FROM EMPLOYEE WHERE Super_ssn IS NULL;
```

8.2 Nested Queries, Tuples, and Set Comparisons

A query embedded inside another query. The embedded block is the **inner/nested query**; the wrapping block is the **outer query**.

Table 12: Nested Query Operators

Operator	Description
IN	Checks if value exists in result set
= ANY / = SOME	Equivalent to IN
> ALL	Value greater than every value in set
< ANY	Value less than at least one value
Scalar =	Use when subquery returns exactly one value

```
-- IN with nested query
SELECT DISTINCT Pnumber FROM PROJECT
WHERE Pnumber IN (SELECT Pno FROM WORKS_ON WHERE Essn = '123456789');

-- Tuple comparison
WHERE (Pno, Hours) IN (SELECT Pno, Hours FROM WORKS_ON WHERE Essn =
'123456789');
```

8.3 Correlated Nested Queries

A nested query is **correlated** if it references an attribute from a relation declared in the outer query's FROM clause.

Execution: The correlated inner query is evaluated **once for each tuple** in the outer query. This is less efficient than uncorrelated queries.

Scope Rule: If an attribute exists in both inner and outer queries, it defaults to the **innermost** query. Use aliases to refer to the outer query explicitly.

Exam Tip

Correlated queries using `=` or `IN` can often be rewritten as a single block query using `JOIN` conditions, which is more efficient.

8.4 Explicit Sets and Renaming

```
-- Explicit set (no subquery needed)
WHERE Pno IN (1, 2, 3);

-- Renaming with AS
SELECT E.Lname AS Employee_name, S.Lname AS Supervisor_name
FROM EMPLOYEE AS E, EMPLOYEE AS S
WHERE E.Super_ssn = S.Ssn;
```

8.5 Joined Tables and Outer Joins

SQL permits defining joins directly in the `FROM` clause rather than mixing join conditions with filtering in `WHERE`.

Table 13: Join Types in SQL

Join Type	Description	Syntax
<code>INNER JOIN</code>	Only matching tuples	<code>FROM T1 JOIN T2 ON condition</code>
<code>NATURAL JOIN</code>	Equijoin on all same-named attrs	<code>FROM T1 NATURAL JOIN T2</code>
<code>LEFT OUTER JOIN</code>	All left table tuples + matches	<code>FROM T1 LEFT JOIN T2 ON ...</code>
<code>RIGHT OUTER JOIN</code>	All right table tuples + matches	<code>FROM T1 RIGHT JOIN T2 ON ...</code>
<code>FULL OUTER JOIN</code>	All tuples from both tables	<code>FROM T1 FULL JOIN T2 ON ...</code>

INNER JOIN

Only Matching

LEFT OUTER JOIN

All Left + Matches

RIGHT OUTER JOIN

All Right + Matches

FULL OUTER JOIN

All from Both


```
-- Inner join example
FROM (EMPLOYEE JOIN DEPARTMENT ON Dno = Dnumber)

-- Left outer join
FROM EMPLOYEE LEFT JOIN DEPARTMENT ON Dno = Dnumber
```

8.6 Aggregate Functions

Aggregate functions compute a single summary value from a collection of values.

Table 14: Aggregate Functions

Function	Purpose
COUNT(*)	Total rows (includes NULLs)
COUNT(Attribute)	Non-NULL values in column
COUNT(DISTINCT Attr)	Unique non-NULL values
SUM	Total of numeric values
AVG	Average of numeric values
MAX	Maximum value
MIN	Minimum value

Key Point

Aggregate functions **automatically eliminate NULLs** before calculating. If all values are NULL, result is NULL (except COUNT(*) which returns 0).

8.7 GROUP BY and HAVING Clauses

Used to divide aggregate data into subcategories (e.g., average salary per department).

Execution Order

1. **WHERE** -- Filters individual tuples first
2. **GROUP BY** -- Groups remaining tuples
3. **HAVING** -- Filters groups based on aggregate conditions

Golden Rule

Any attribute in **SELECT** that is **not** inside an aggregate function **must** appear in **GROUP BY**.

Figure 7.1

Results of GROUP BY and HAVING. (a) Q24, (b) Q26.

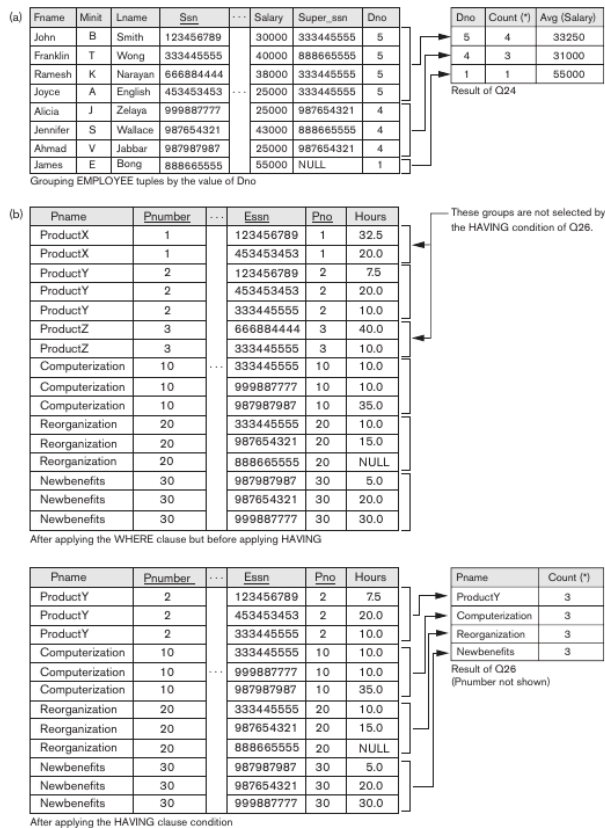


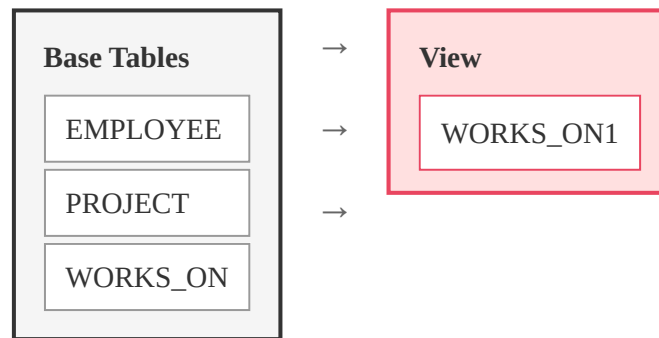
Figure 6: Results of GROUP BY and HAVING queries (Q24, Q26) showing grouping and filtering (Figure 7.1)

```
-- Average salary per department
SELECT Dno, AVG(Salary) AS Avg_Sal, COUNT(*) AS Emp_Count
FROM EMPLOYEE
GROUP BY Dno
HAVING COUNT(*) > 2;
```

9. Views in SQL (Virtual Tables)

A view is a **virtual table** derived from the results of a pre-configured query.

9.1 The Concept of a View



User queries the view; data comes from base tables dynamically

9.2 Specifying Views

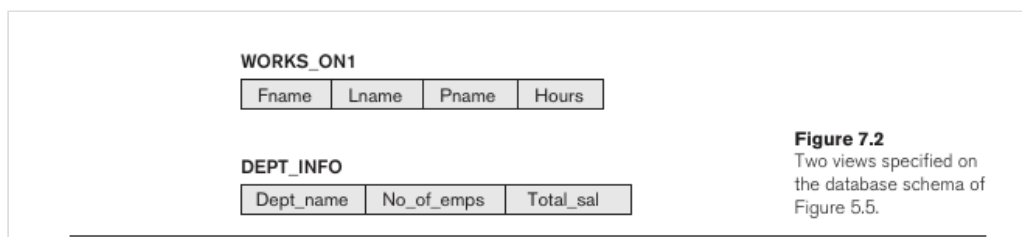


Figure 7: Two views specified on the *COMPANY* database schema: *WORKS_ON1* and *DEPT_INFO* (Figure 7.2)

CREATE VIEW Examples

```

-- Basic view
CREATE VIEW WORKS_ON1 AS
SELECT Fname, Lname, Pname, Hours
FROM EMPLOYEE, PROJECT, WORKS_ON
WHERE Ssn = Essn AND Pno = Pnumber;

-- View with explicit column names (for aggregates)
CREATE VIEW DEPT_INFO(Dept_name, No_of_emps, Total_sal) AS
SELECT Dname, COUNT(*), SUM(Salary)
FROM DEPARTMENT, EMPLOYEE
WHERE Dnumber = Dno
GROUP BY Dname;

-- Query a view like a table
SELECT Fname, Lname FROM WORKS_ON1 WHERE Pname = 'ProductX';

-- Drop a view
DROP VIEW WORKS_ON1;

```

10. Schema Modification Statements

SQL allows evolving the schema while the database remains operational.

10.1 The DROP Command

Removes named schema elements entirely.

Table 15: DROP Behavior Options

Option	Behavior
CASCADE	Drops target AND all dependent objects (constraints, views, etc.)
RESTRICT	Prevents drop if any other object references it

```
DROP SCHEMA COMPANY CASCADE;
DROP TABLE DEPENDENT CASCADE;
```

Important Distinction

`DROP TABLE` removes data AND structure from the catalog. `DELETE` only removes records, leaving the empty table structure intact.

10.2 The ALTER Command

Modifies the structural definition of existing base tables.

Table 16: ALTER TABLE Operations

Operation	Syntax
Add column	<code>ALTER TABLE T ADD COLUMN ColName VARCHAR(12);</code>
Drop column	<code>ALTER TABLE T DROP COLUMN Address CASCADE;</code>
Drop default	<code>ALTER TABLE T ALTER COLUMN C DROP DEFAULT;</code>
Set default	<code>ALTER TABLE T ALTER COLUMN C SET DEFAULT 'value';</code>
Drop constraint	<code>ALTER TABLE T DROP CONSTRAINT Name CASCADE;</code>
Add constraint	<code>ALTER TABLE T ADD CONSTRAINT Name ...;</code>

Note

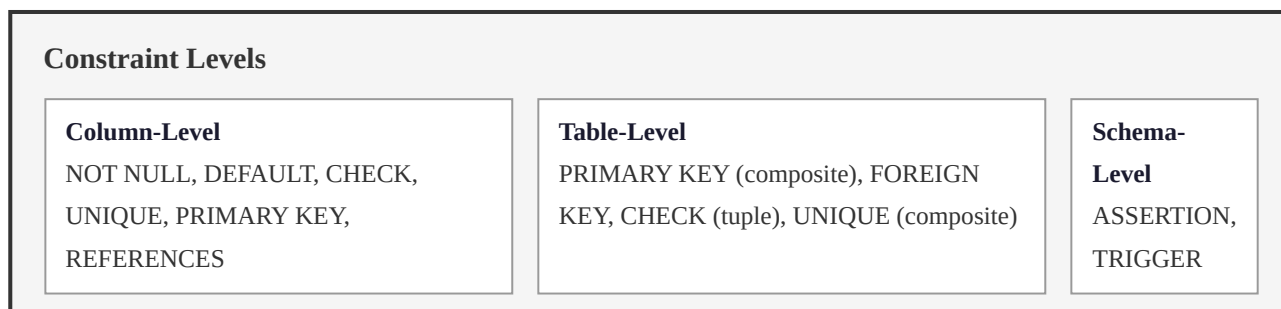
When adding a column, existing tuples get NULL for the new column, so you cannot use `NOT NULL` unless you also define a `DEFAULT` value.

11. Key Diagrams Summary

SQL Query Execution Flow



Constraint Types Hierarchy



12. Exam Questions & Answers

Q1. What is the difference between a Table, Base Relation, and View in SQL?

Table is the SQL term for a relation. A **Base Table (Base Relation)** is physically created and stored as a file by the DBMS using `CREATE TABLE`. A **View** is a virtual table created with `CREATE VIEW` that stores only the query definition, not actual data. Views dynamically reflect data from base tables.

Q2. How do you handle circular/forward references when creating tables with foreign keys?

Omit the problematic foreign key constraints from the initial `CREATE TABLE` statement and add them later using `ALTER TABLE`. For example, if `EMPLOYEE.Super_ssn` references `EMPLOYEE.Ssn` (self-reference) or if `DEPARTMENT.Mgr_ssn` references `EMPLOYEE` before `EMPLOYEE` is created.

Q3. What is the difference between CHAR and VARCHAR?

CHAR(n) is fixed-length; if a shorter string is entered, it is right-padded with blank characters. VARCHAR(n) is varying-length; it stores only the actual characters entered without padding. Use CHAR for fixed-length data (like SSN, postal codes) and VARCHAR for variable-length data (like names, addresses).

Q4. Explain DECIMAL(i,j) with an example.

DECIMAL(i, j) (or NUMERIC(i, j)) stores formatted numbers where i is the precision (total number of decimal digits) and j is the scale (digits after the decimal point). Example: DECIMAL(10, 2) can store values like 12345678.90 (10 total digits, 2 after decimal).

Q5. What are the differences between PRIMARY KEY and UNIQUE constraints?

Feature	PRIMARY KEY	UNIQUE
NULL allowed	No	Yes
Number per table	Only one	Multiple
Purpose	Main unique identifier	Alternate key

Q6. What are referential triggered actions and list them?

Referential triggered actions define how the DBMS handles violations caused by UPDATE/DELETE on referenced tuples:

- **RESTRICT (default):** Rejects the operation causing violation
- **SET NULL:** Changes referencing attribute to NULL
- **SET DEFAULT:** Changes referencing attribute to its default value
- **CASCADE:** Propagates the change (delete/update referencing tuples)

Q7. Why can't you use = NULL to check for NULL values?

Because each NULL is treated as distinct from every other NULL. Any comparison with NULL results in **UNKNOWN** (three-valued logic), not TRUE or FALSE. The WHERE clause only selects tuples evaluating to TRUE. Therefore, use `IS NULL` or `IS NOT NULL` instead.

Q8. Explain three-valued logic in SQL with truth tables.

SQL uses three truth values: TRUE, FALSE, and UNKNOWN. Comparisons involving NULL produce UNKNOWN.

- **AND:** TRUE AND UNKNOWN = UNKNOWN; FALSE AND UNKNOWN = FALSE; UNKNOWN AND UNKNOWN = UNKNOWN
- **OR:** TRUE OR UNKNOWN = TRUE; FALSE OR UNKNOWN = UNKNOWN; UNKNOWN OR UNKNOWN = UNKNOWN
- **NOT:** NOT UNKNOWN = UNKNOWN

WHERE clause only keeps tuples evaluating to TRUE.

Q9. What is a correlated nested query and how does it differ from a regular nested query?

A **correlated nested query** references an attribute from a relation in the outer query's FROM clause. It is evaluated **once for each tuple** in the outer query. A regular (uncorrelated) nested query is independent of the outer query and is evaluated only once. Correlated queries are generally less efficient.

Q10. What is the difference between INNER JOIN and LEFT OUTER JOIN?

INNER JOIN includes only tuples that have matching tuples in both relations. **LEFT OUTER JOIN** keeps every tuple from the left table, and for tuples with no match in the right table, the unmatched attributes are padded with NULL. Similarly, **RIGHT OUTER JOIN** keeps all right table tuples, and **FULL OUTER JOIN** keeps all tuples from both tables.

Q11. What is the difference between COUNT(*), COUNT(Attribute), and COUNT(DISTINCT Attribute)?

- `COUNT( *)` : Counts total rows including those with NULLs
- `COUNT(Attribute)` : Counts only non-NULL values in that column
- `COUNT(DISTINCT Attribute)` : Counts unique non-NULL values only

Q12. What is the correct order of SQL query clauses and execution?

Writing order: SELECT -> FROM -> WHERE -> GROUP BY -> HAVING -> ORDER BY

Execution order:

1. FROM (identify tables)
2. WHERE (filter individual rows)
3. GROUP BY (group the rows)
4. HAVING (filter groups)
5. SELECT (project columns)
6. ORDER BY (sort results)

Q13. What is the golden rule of GROUP BY?

Any attribute that appears in the SELECT clause and is **NOT** wrapped in an aggregate function (COUNT, SUM, AVG, MAX, MIN) **MUST** appear in the GROUP BY clause. This ensures each output row has a single value for the non-aggregated column.

Q14. What is a View and what are its advantages?

A **View** is a virtual table derived from a query. It stores only the query definition, not data.

- **Simplification:** Users can query complex joins with simple SELECT statements
- **Security:** Restricts access to specific rows or columns
- **Data Independence:** Abstracts underlying schema changes
- **Up-to-date:** Automatically reflects changes in base tables

Q15. What is the difference between DROP TABLE and DELETE?

`DROP TABLE` removes both the data AND the structural definition of the table from the catalog entirely. `DELETE` only removes records (tuples) from the table, leaving the empty table structure intact. `DROP` is a DDL command; `DELETE` is a DML command.

Q16. What are the DROP behavior options CASCADE and RESTRICT?

- **CASCADE:** Aggressively drops the target element AND automatically drops any constraints, views, or other elements that depend on it.
- **RESTRICT:** Defensively prevents the drop from occurring if the element is referenced by any other part of the schema. Use `RESTRICT` when you want to be safe and manually handle dependencies.

Q17. Explain the purpose and usage of the LIKE operator with examples.

`LIKE` is used for pattern matching in strings:

- `WHERE Lname LIKE 'S%' -- Names starting with S`
- `WHERE Fname LIKE '_o%' -- Names with 'o' as second letter`
- `WHERE Address LIKE '%Houston%' -- Addresses containing Houston`
- Use `ESCAPE` to search for literal % or _: `LIKE '50\% OFF' ESCAPE '\'`

Q18. What is the difference between a Schema and a Catalog?

A **Schema** is a grouping of tables and constructs belonging to one database application, identified by a schema name and authorization identifier. A **Catalog** is a named collection of schemas. Every catalog contains a special `INFORMATION_SCHEMA` that holds metadata about all schemas and elements.

Q19. What are set operations in SQL and what is their requirement?

SQL supports `UNION`, `INTERSECT`, and `EXCEPT` (difference). The participating queries must be **type-compatible**: same number of attributes with compatible data types in the same order. Appending `ALL` (e.g., `UNION ALL`) retains duplicates; without `ALL`, duplicates are eliminated.

Q20. How do aggregate functions handle NULL values?

Aggregate functions **automatically eliminate NULL values** before computing the result. If the entire collection is NULL, the function returns NULL (except `COUNT(*)` which returns 0). `COUNT(*)` counts all rows including NULLs; `COUNT(attr)` counts only non-NULL values.

Good Luck with Your Exam!

DBMS Unit 4 - Complete Notes

EXAM PREPARATION NOTES

Database Design & Normalization

*ER-to-Relational Mapping, Design Guidelines, Functional Dependencies,
and Normal Forms*

Complete Revision Guide

With Diagrams, Algorithms & Practice Q&A

Table of Contents

Part 1: ER-to-Relational Mapping

- Step 1 — Mapping Regular (Strong) Entity Types
- Step 2 — Mapping Weak Entity Types
- Step 3 — Mapping Binary 1:1 Relationships
- Step 4 — Mapping Binary 1:N Relationships
- Step 5 — Mapping Binary M:N Relationships
- Step 6 — Mapping Multivalued Attributes
- Step 7 — Mapping N-ary Relationships
- Summary of Correspondences & JOINS

Part 2: Informal Design Guidelines

- Guideline 1 — Clear Semantics
- Guideline 2 — Redundancy & Update Anomalies
- Guideline 3 — NULL Values
- Guideline 4 — Spurious Tuples
- Summary of Design Guidelines

Part 3: Functional Dependencies & Normal Forms

- Functional Dependencies (FDs)
- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce-Codd Normal Form (BCNF)
- Armstrong's Axioms & Inference Rules
- Closure of Attribute Sets

Part 4: Exam Practice Q&A

Part 5: Quick Reference Cheat Sheet

Part 1: ER-to-Relational Mapping

These notes cover the comprehensive algorithm used to map a **Conceptual Schema (ER Model)** into a **Logical Schema (Relational Model)**.

Assumptions & Preliminaries

- The mapping creates tables with **simple, single-valued attributes**.
- Relational model constraints (**primary keys, unique keys, referential integrity**) are defined during mapping.
- Examples are based on the **COMPANY Database** (ER Schema and final mapped Relational Schema).

Step 1: Mapping of Regular (Strong) Entity Types

Algorithm

1. For each regular entity type E , create a relation (table) R .
2. Include all the simple attributes of E in R .
3. For composite attributes, include only their **simple component attributes**.
4. Choose one key attribute of E as the **Primary Key (PK)** for R . If the key is composite, the set of simple attributes forming it together become the PK.

Important Detail

If multiple keys were identified during conceptual design, keep this knowledge. It is used to specify additional (unique) keys and for indexing/analysis purposes later.

Relations created in this step are called **entity relations** because each tuple represents an entity instance. **Foreign keys and relationship attributes are NOT included yet** — they are added in later steps.

Example from COMPANY Database

Table 1.1 Entity Relations from COMPANY Database

Entity Type	Mapped Relation	Primary Key	Unique Key
EMPLOYEE	EMPLOYEE(Fname, Minit, Lname, Ssn, Bdate, Address, Sex, Salary)	Ssn	—
DEPARTMENT	DEPARTMENT(Dname, Dnumber)	Dnumber	Dname
PROJECT	PROJECT(Pname, Pnumber, Plocation)	Pnumber	Pname

EMPLOYEE		
string	Fname	
string	Minit	
string	Lname	
string	Ssn	PK
date	Bdate	
string	Address	
string	Sex	
float	Salary	

DEPARTMENT		
string	Dname	UK
int	Dnumber	PK

PROJECT		
string	Pname	UK
int	Pnumber	PK
string	Plocation	

Figure 1.1: Entity Relations after Step 1 (Strong Entities)

Step 2: Mapping of Weak Entity Types

Algorithm

1. For each weak entity type W with owner entity E , create a relation R .
2. Include all simple attributes (or simple components of composite attributes) of W .
3. **Foreign Key:** Include the primary key attribute(s) of the owner entity E as foreign keys in R . This maps the identifying relationship.
4. **Primary Key:** The PK of R is the combination of the owner's PK and the partial key of the weak entity (if any).

Referential Triggers (CASCADE)

Use the **propagate (CASCADE)** option for the referential triggered action (ON UPDATE and ON DELETE) on the foreign key. A weak entity has an **existence dependency** on its owner.

Note on Nested Weak Entities

If a weak entity $W1$ is owned by another weak entity $W2$, map $W2$ first to determine its primary key before mapping $W1$.

Example from COMPANY Database

Weak Entity: DEPENDENT (Owner: EMPLOYEE)

Mapped Relation: DEPENDENT(Essn, Dependent_name, Sex, Bdate, Relationship)

- **FK:** Essn (references EMPLOYEE.Ssn)
- **PK:** {Essn, Dependent_name} — Dependent_name is the partial key

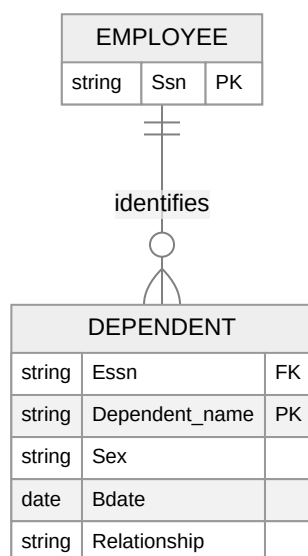


Figure 1.2: Weak Entity Mapping (DEPENDENT)

Step 3: Mapping of Binary 1:1 Relationship Types

For a 1:1 relationship R between entities $E1$ and $E2$, there are **three approaches**:

Table 1.2 Three Approaches for 1:1 Relationship Mapping

Approach	Description	When to Use
1. Foreign Key (Preferred)	Choose one relation (entity with total participation) and include the PK of the other as a FK. Include any relationship attributes.	Most useful; minimizes NULLs
2. Merged Relation	Merge both entity types and the relationship into a single table.	Only when both participations are total (same number of tuples)
3. Cross-Reference	Create a third relation with PKs of both as FKs; one FK is unique.	Requires extra JOINS; rarely preferred

Exam Tip: Why Total Participation Gets the FK

If we put the FK in a table where participation is **partial** (e.g., putting Department_managed in EMPLOYEE), it creates massive NULL values. If only 2% of employees manage a department, 98% of tuples would have NULLs in that column.

Example from COMPANY Database

Relationship: MANAGES (1:1 between EMPLOYEE and DEPARTMENT)

- DEPARTMENT has **total participation** (every department MUST have a manager).
- Solution: Add **Mgr_ssn** (FK referencing EMPLOYEE.Ssn) and relationship attribute **Mgr_start_date** into the DEPARTMENT table.

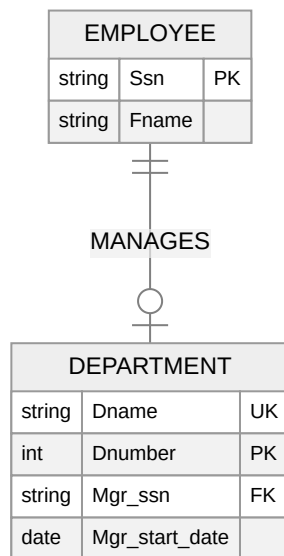


Figure 1.3: 1:1 Relationship Mapping (Foreign Key Approach)

Step 4: Mapping of Binary 1:N Relationship Types

Two possible approaches:

Table 1.3 Approaches for 1:N Relationship Mapping

Approach	Description	When to Use
1. Foreign Key (Preferred)	Identify the relation on the N-side . Include the PK of the 1-side as a FK. Include any relationship attributes.	Default choice
2. Relationship Relation	Create a separate relation with PKs of both as FKs. PK is same as PK of N-side.	Only if very few tuples participate (avoid NULLs)

Why the N-Side?

Each entity on the N-side is related to **at most ONE** entity on the 1-side. Therefore, each tuple needs only **one FK value**.

Examples from COMPANY Database

Table 1.4 1:N Mapping Examples

Relationship	N-Side	Foreign Key Added
WORKS_FOR (Dept → Emp)	EMPLOYEE	Dno references DEPARTMENT.Dnumber
CONTROLS (Dept → Proj)	PROJECT	Dnum references DEPARTMENT.Dnumber
SUPERVISION (Recursive on Emp)	EMPLOYEE	Super_ssn references EMPLOYEE.Ssn

Step 5: Mapping of Binary M:N Relationship Types

Algorithm

1. We **cannot** use the foreign key approach for M:N because neither side can hold a single value representing the relationship.
2. **Mandatory:** Use the **Relationship Relation (Cross-Reference)** approach.
3. Create a new relation *R*. Include the PKs of the participating relations as foreign keys in *R*.
4. The **combination of these foreign keys** forms the PK of *R*.
5. Include any simple attributes of the M:N relationship in *R*.

Referential Triggers

Specify **CASCADE** for both ON UPDATE and ON DELETE on the foreign keys because relationship instances are **existence-dependent** on the entities they relate.

Example from COMPANY Database

Relationship: WORKS_ON (M:N between EMPLOYEE and PROJECT)

Mapped Relation: WORKS_ON(Essn, Pno, Hours)

- **FKs:** Essn references EMPLOYEE.Ssn; Pno references PROJECT.Pnumber
- **PK:** {Essn, Pno}
- **Relationship attribute:** Hours

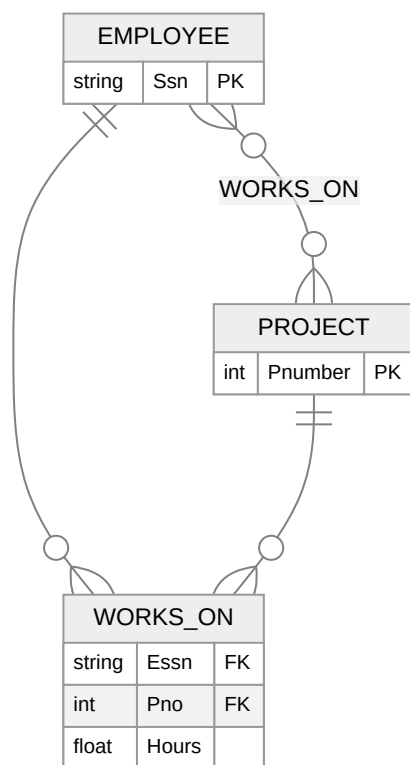


Figure 1.4: M:N Relationship Mapping (Relationship Relation)

Step 6: Mapping of Multivalued Attributes

Algorithm

1. The basic relational model doesn't allow lists/sets of values in a single tuple.
2. For each multivalued attribute A of entity E , create a new relation R .
3. Include an attribute corresponding to A , plus the primary key of the owner entity as a foreign key in R .
4. **Primary Key:** The combination of the FK and A . (If A is composite, include its simple components).

Modern DB Note

In newer object-relational systems allowing **array data types**, this step can be skipped by using an array attribute instead of a new table.

- **PK:** {Dnumber, Dlocation}

If Department 5 has 3 locations, there will be 3 separate tuples in this table for Dept 5.

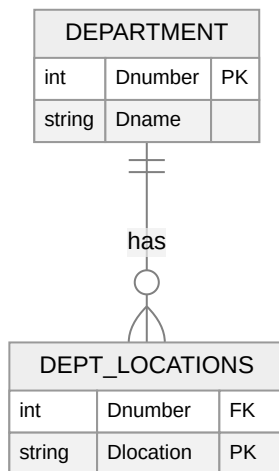


Figure 1.5: Multivalued Attribute Mapping

Step 7: Mapping of N-ary Relationship Types

Algorithm

1. Map similarly to M:N relationships using the **relationship relation** option.
2. Create a new relation R . Include the PKs of **all participating entity types** as foreign keys.
3. Include any simple relationship attributes.
4. **Primary Key:** Usually the combination of all the foreign keys.

Exception Constraint

If the cardinality constraint on any entity E is **1**, then the FK referencing E is **excluded from the primary key** of R .

Example (Ternary SUPPLY)

Relationship: Ternary SUPPLY relating SUPPLIER ($Sname$), PART ($Part_no$), and PROJECT ($Proj_name$)

Mapped Relation: SUPPLY($Sname$, $Proj_name$, $Part_no$, Quantity)

- **PK:** {Sname, Proj_name, Part_no} — the combination of all three FKs

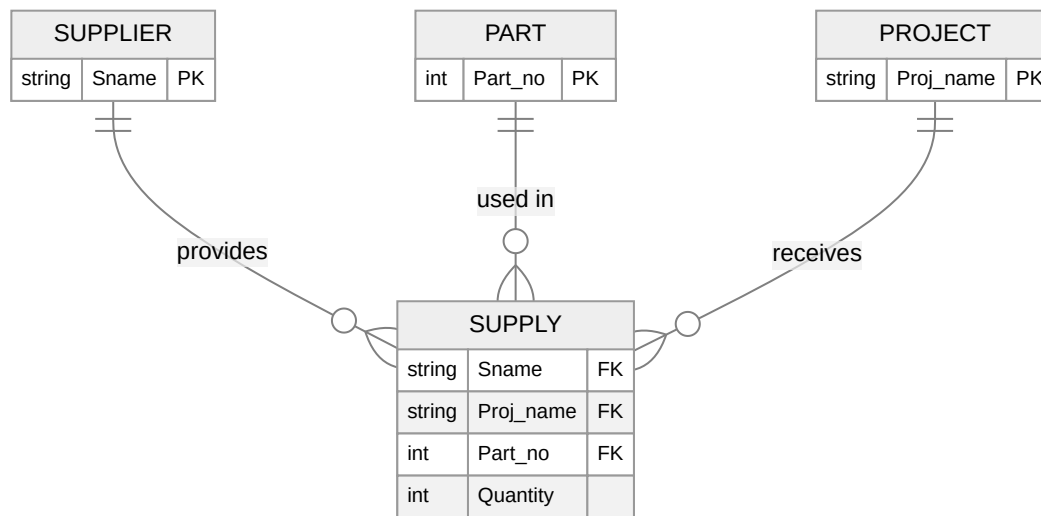


Figure 1.6: N-ary Relationship Mapping

Summary of Correspondences & Materializing Relationships via JOINS

Table 1.5 ER Model to Relational Model Correspondences

ER Model Construct	Relational Model Construct
Entity type	Entity relation
1:1 or 1:N relationship type	Foreign key (or relationship relation)
M:N relationship type	Relationship relation and two foreign keys
N-ary relationship type	Relationship relation and N foreign keys
Simple attribute	Attribute
Composite attribute	Set of simple component attributes
Multivalued attribute	Relation and foreign key
Value set	Domain
Key attribute	Primary (or secondary) key

Core Concept: Materializing Relationships via JOINS

In the relational model, relationships are represented logically by **sharing attributes** (PK in one table → FK in another over the same domain).

How to Combine Data

To materialize the relationship, use the **EQUIJOIN** operation (or **NATURAL JOIN** if attribute names match exactly) over the PK/FK pairs.

Table 1.6 JOIN Requirements by Relationship Type

Relationship Type	Mapping	Join Operations Required
1:1 or 1:N	Foreign key mapping	1 join operation
M:N	Relationship relation mapping	2 join operations
N-ary	N-ary relationship mapping	N join operations

Warning: Avoid Meaningless JOINS

Users must accurately know the PK/FK attributes. Applying an EQUIJOIN on non-related attributes (e.g., joining PROJECT and DEPT_LOCATIONS simply because Dlocation = Plocation) leads to **meaningless or spurious data**.

Drawbacks of the Basic Relational Model

1. **Data Repetition:** Mapping multivalued attributes requires duplicating the primary key of the owner for every value. DEPT_LOCATIONS requires three tuples for Department 5 just to list three cities.
2. **Lack of Advanced Algebra:** Basic relational algebra lacks NEST or COMPRESS operations. Cannot generate a single consolidated tuple like <'5', {'Bellaire', 'Sugarland', 'Houston'}>.
3. **Solution:** Object data models and Object-Relational systems solve this by supporting built-in list or array data types directly within attributes.

Part 2: Informal Design Guidelines for Relation Schemas

Before diving into formal normalization theory, we measure the quality of a relation schema using four core **informal guidelines**:

1. Making sure the semantics (meaning) of the attributes are clear.
2. Reducing redundant information in tuples.
3. Reducing NULL values in tuples.
4. Disallowing the possibility of generating spurious (invalid) tuples.

Guideline 1: Imparting Clear Semantics to Attributes

Guideline 1 Statement

Design a relation schema so that it is easy to explain its meaning. **Do not combine attributes from multiple entity types and relationship types into a single relation.**

Rule of Thumb

If a relation corresponds to **exactly one entity type** or **one relationship type**, it will not have semantic ambiguities.

Examples of Good Design

Table 2.1 Well-Designed Relations (Clear Semantics)

Relation	Represents	Keys
EMPLOYEE	One employee per tuple	PK: Ssn; FK: Dnumber
DEPARTMENT	One department per tuple	PK: Dnumber
PROJECT	One project per tuple	PK: Pnumber
DEPT_LOCATIONS	Multivalued attribute of department	PK: {Dnumber, Dlocation}
WORKS_ON	M:N relationship between employees and projects	PK: {Essn, Pno}

Examples of Violating Guideline 1

Table 2.2 Poorly-Designed Relations (Semantic Ambiguity)

Bad Relation	Violation
EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn)	Mixes employee attributes with department attributes
EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation)	Mixes employee attributes, project attributes, and WORKS_ON relationship attributes

Key Takeaway

These "fat" schemas are perfectly fine to be used as **Views** for end-users, but they are terrible as **Base Relations** (underlying stored tables) because they cause data maintenance issues.

Guideline 2: Redundant Information and Update Anomalies

A major goal of schema design is **minimizing storage space**. Grouping too many attributes together creates massive redundancy, which directly leads to **Update Anomalies**.

The Three Types of Update Anomalies

Using the badly designed **EMP_DEPT** relation as an example:

1. Insertion Anomalies

- **Scenario A (Consistency Risk):** To insert a new employee working in Dept 5, you must manually enter all department details (Dname, Dmgr_ssn) perfectly. If you misspell "Research", the database becomes inconsistent. Good design: just enter Dnumber = 5.
- **Scenario B (Entity Integrity Risk):** How do you insert a new department without employees? You'd need NULLs for all employee attributes. But Ssn is the PK and cannot be NULL — **violating Entity Integrity**.

2. Deletion Anomalies

- If employee 'Borg, James E.' is the last employee in "Headquarters" (Dept 1), deleting his tuple **inadvertently destroys all information about that department**. Good design keeps DEPARTMENT and EMPLOYEE as independent tables.

3. Modification Anomalies

- If Dept 5 gets a new manager, you must find and **update every single employee tuple** in Dept 5. If the system crashes halfway, or you miss a row, the database claims Dept 5 has two different managers simultaneously.

Guideline 2 Statement (Exam Must-Know)

Design base relation schemas so that **no insertion, deletion, or modification anomalies are present**. If anomalies must be present (e.g., for performance/materialized views), note them clearly and use automatic mechanisms (triggers or stored procedures) to ensure consistency.

Guideline 3: NULL Values in Tuples

If you create "fat" relations containing attributes that don't apply to every tuple, your database will be filled with **NULL values**.

Problems Caused by NULLs

1. **Wasted Space:** At the storage level.

2. **Unpredictable Comparisons:** SELECT and JOIN operations involving comparisons become unpredictable.
3. **Skewed Aggregations:** Functions like COUNT or SUM handle NULLs differently, leading to incorrect results.
4. **Ambiguity in Meaning:** A NULL can mean three completely different things:
 - **Not Applicable** — e.g., Visa_status for a domestic U.S. student.
 - **Unknown** — e.g., an employee's Date_of_birth is unknown.
 - **Known but Absent** — e.g., the employee has a Home_Phone_Number but it hasn't been recorded yet.

Guideline 3 Statement

Avoid placing attributes in a base relation whose values may frequently be NULL. If unavoidable, ensure they only apply to rare/exceptional cases.

Practical Example

If only 15% of employees have a private office, **DO NOT** put Office_number in EMPLOYEE. Instead, create: **EMP_OFFICES(Essn, Office_number)**. This saves 85% of tuples from carrying a meaningless NULL.

Guideline 4: Generation of Spurious Tuples

When decomposing a bad relation into smaller ones, it must be done **carefully**. If decomposed incorrectly, joining them back together generates **fake, invalid data** called spurious tuples.

Example of Terrible Decomposition

Take the bad EMP_PROJ table and split it into:

1. EMP_LOCS(Ename, Plocation)
2. EMP_PROJ1(Ssn, Pnumber, Hours, Pname, Plocation)

The Problem

We lost the direct link between the employee and the specific project they work on. When we try to reconstruct using NATURAL JOIN, the join condition defaults to the shared attribute: **Plocation**.

Employee "Smith, John B." works in "Bellaire" and "Sugarland". Because we join merely on Plocation, the operation creates combinations that never existed! It pairs Smith with other people's projects just because those projects happen to be in Sugarland or Bellaire.

Guideline 4 Statement

Design relation schemas so that they can be joined with equality conditions on attributes that are appropriately related (**Primary Key to Foreign Key pairs**). This guarantees no spurious tuples are generated. Avoid joining on matching attributes that are **not PK/FK relationships**.

Formal Property

The formal mathematical property that guarantees a join won't produce spurious tuples is called the **nonadditive (or lossless) join property**.

Summary of Design Guidelines

A problematic relational schema design can be identified informally by looking for:

Table 2.3 Four Warning Signs of Bad Schema Design

Sign	What It Means	Solution
Ambiguity	Mixing multiple entities/relationships making it hard to explain	Separate into entity/relationship relations
Anomalies	Redundant work during insertions/modifications; accidental data loss during deletions	Decompose to eliminate redundancy
Waste	Massive NULL values causing inefficient storage and complicated queries	Extract optional attributes to separate tables
Spurious Data	Invalid records generated during joins on non-PK/FK attributes	Ensure lossless (nonadditive) join decomposition

Formal Tools Coming Next

Formal tools used to prevent these issues — **Functional Dependencies** and **Normal Forms like BCNF** — are covered in Part 3.

Part 3: Functional Dependencies & Normal Forms

14.2 Functional Dependencies (FDs)

Before normalizing a database, we need a formal tool to analyze relation schemas. The single most important concept in relational schema design is the **Functional Dependency**.

Definition of Functional Dependency

Formal Definition

A functional dependency is a constraint between two sets of attributes from the database.

- **Notation:** $X \rightarrow Y$ (Read as: " X functionally determines Y " or " Y is functionally dependent on X ").
- **Definition:** For any two tuples t_1 and t_2 in a legal relation state r of R , if they have the same value for X ($t_1[X] = t_2[X]$), they must also have the same value for Y ($t_1[Y] = t_2[Y]$).
- **Meaning:** The values of the X component **uniquely determine** the values of the Y component.

Critical Note: Direction Matters

Just because $X \rightarrow Y$ holds, it does **NOT** mean $Y \rightarrow X$ holds.

Connection to Keys

If a set of attributes X is a candidate key of relation R , it means no two tuples can have the same X value. Therefore, X functionally determines **all other attributes** in R .

If X is a key, then $X \rightarrow Y$ for **all attributes** Y in R .

Semantic Property vs. Relation State

Exam Must-Know: FDs are Semantic, Not Syntactic

A functional dependency is a property of the **semantics (meaning)** of the attributes in the real world, **NOT** a property of a single specific snapshot (state) of the database.

Table 3.1 Proving vs. Disproving FDs

Action	Method	Example
Proving an FD	You cannot look at a populated table and automatically infer an FD. You must know the real-world rules.	$\{\text{State, Driver_license_number}\} \rightarrow \text{Ssn}$ holds true in the real world.
Disproving an FD	Use a single counterexample to definitively disprove an FD.	In TEACH, Teacher 'Smith' teaches 'Data Structures' and 'Data Management' $\rightarrow$ Teacher $\rightarrow$ Course is ruled out.

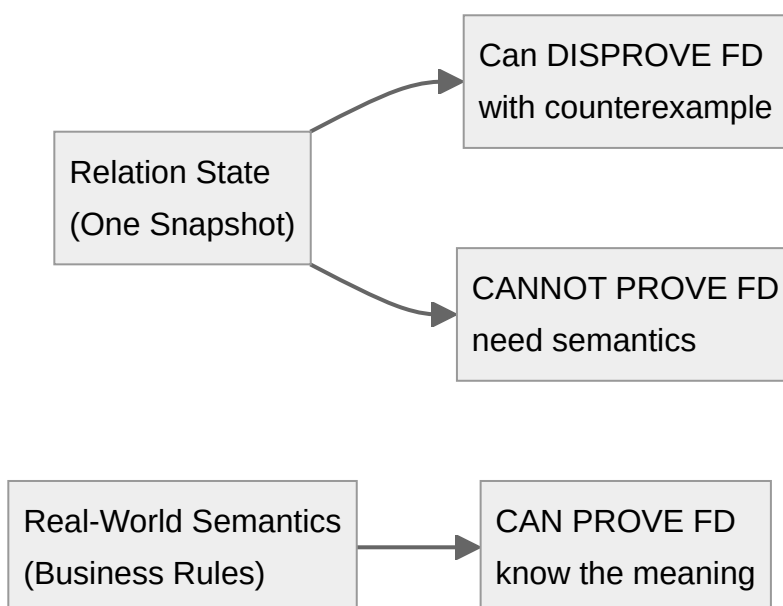


Figure 3.1: FDs depend on semantics, not data snapshots

14.3 Normal Forms Based on Primary Keys

Properties and Terminology

When decomposing relations to achieve higher normal forms, two critical properties must be maintained:

1. **Nonadditive (Lossless) Join Property:** Guarantees that no spurious (fake) tuples are generated when joining the decomposed tables back together. This is **mandatory**.
2. **Dependency Preservation Property:** Ensures every original functional dependency is represented in at least one of the new relations. This is **highly desirable**, though sometimes sacrificed.

Denormalization

The **intentional** process of storing joined, lower-normal-form relations to improve read/query performance (trading space/anomalies for speed).

Key Definitions Review

Table 3.2 Key Terminology for Normalization

Term	Definition
Superkey	A set of attributes that uniquely identifies a tuple.
Candidate Key	A minimal superkey (removing any attribute ruins uniqueness).
Primary Key (PK)	The specific candidate key chosen to identify tuples.
Prime Attribute	An attribute that is part of any candidate key.
Nonprime Attribute	An attribute that is NOT part of any candidate key.

14.3.4 First Normal Form (1NF)

The Rule (1NF)

The domain of an attribute must include only **atomic (simple, indivisible)** values. A tuple cannot have a set of values, nor can it contain a nested relation (a table within a table).

Violation Type 1: Multivalued Attributes

Suppose DEPARTMENT has a Dlocations attribute containing {Bellaire, Sugarland, Houston}. This violates 1NF.

Table 3.3 Solutions for 1NF Violation (Multivalued Attributes)

Solution	Approach	Quality
Solution 1 (Best)	Remove multivalued attribute, create new relation: DEPT_LOCATIONS(Dnumber, Dlocation). PK = {Dnumber, Dlocation}.	Correct, normalized
Solution 2 (Bad)	Expand PK to {Dnumber, Dlocation} and repeat all other department info on multiple rows.	Massive redundancy
Solution 3 (Bad)	Create columns Dlocation1, Dlocation2, Dlocation3.	NULLs if fewer locations; hard to query

Violation Type 2: Nested Relations

Suppose EMP_PROJ contains Ssn, Ename, and a nested table PROJS(Pnumber, Hours) inside each employee's row.

Solution: Unnest the Relation

Extract nested attributes into a new table and propagate the parent's PK:

- New Table 1: EMP_PROJ1(Ssn, Ename)
- New Table 2: EMP_PROJ2(Ssn, Pnumber, Hours) → PK is {Ssn, Pnumber}

Modern Exception

Modern BLOB/CLOB data types used for images/videos are treated as single **atomic values**, so they do not violate 1NF.

14.3.5 Second Normal Form (2NF)

The Rule (2NF)

A relation is in 2NF if it is in 1NF and **every nonprime attribute is fully functionally dependent on the primary key**. (There are no partial dependencies).

Full vs. Partial Dependency

- **Full Dependency:** $X \rightarrow Y$ is full if removing **any** attribute from X destroys the dependency.
- **Partial Dependency:** $X \rightarrow Y$ is partial if you can remove an attribute from X and the dependency still holds.
- **Shortcut:** If a table has a **single-attribute Primary Key**, it is automatically in 2NF (partial dependencies require a composite key).

2NF Normalization Example

Relation: EMP_PROJ(Ssn, Pnumber, Hours, Ename, Pname, Plocation)

Primary Key: {Ssn, Pnumber}

Table 3.4 Dependencies in EMP_PROJ

Dependency	Type	Status
{Ssn, Pnumber} $\rightarrow$ Hours	Full Dependency	Good
Ssn $\rightarrow$ Ename	Partial Dependency	Bad! Ename depends only on part of the key
Pnumber $\rightarrow$ {Pname, Plocation}	Partial Dependency	Bad! Project details depend only on Pnumber

Decomposition to achieve 2NF:

1. EP1(Ssn, Pnumber, Hours) $\rightarrow$ Keeps the full dependency
2. EP2(Ssn, Ename) $\rightarrow$ Extracts the employee partial dependency
3. EP3(Pnumber, Pname, Plocation) $\rightarrow$ Extracts the project partial dependency

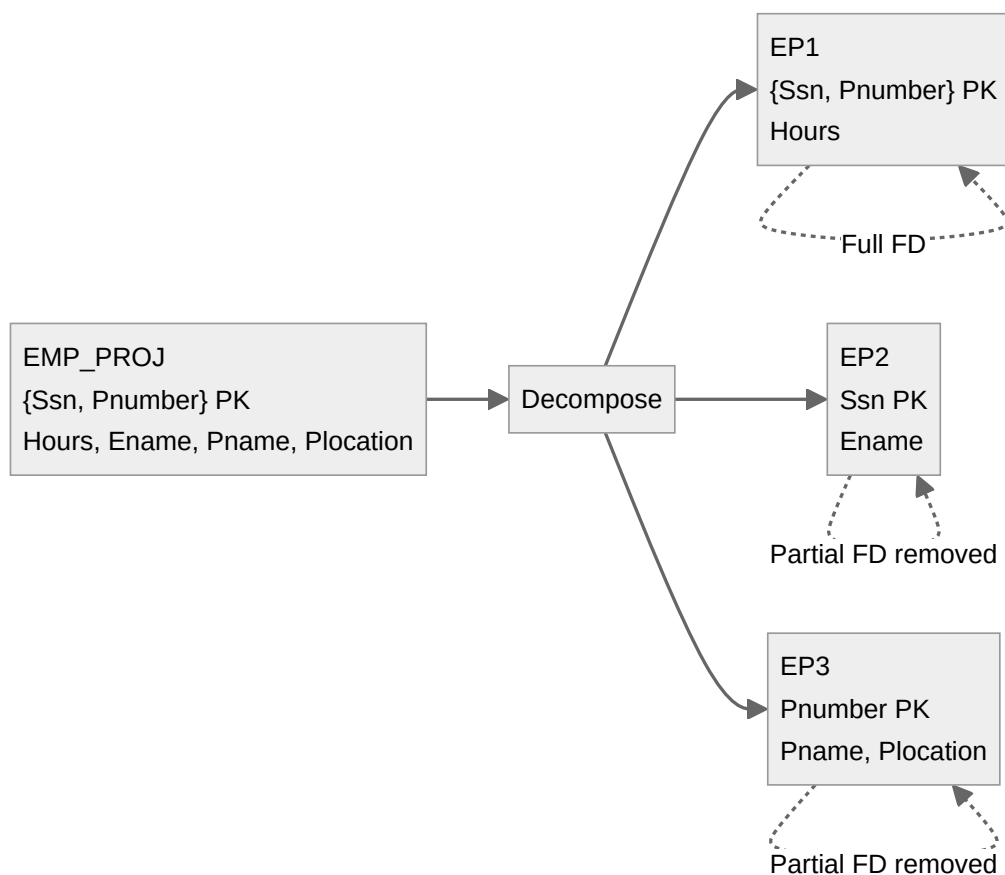


Figure 3.2: 2NF Decomposition Example

14.3.6 Third Normal Form (3NF)

The Rule (3NF)

A relation is in 3NF if it satisfies 2NF and **no nonprime attribute is transitively dependent on the primary key**.

Transitive Dependency

A dependency $X \rightarrow Y$ is **transitive** if there is an intermediate set of attributes Z (which is **not a candidate key**) such that $X \rightarrow Z$ and $Z \rightarrow Y$.

3NF Normalization Example

Relation: EMP_DEPT(Ename, Ssn, Bdate, Address, Dnumber, Dname, Dmgr_ssn)

PK: Ssn

- Dependencies: $Ssn \rightarrow Dnumber$ holds.

Intuitive Check

A department's name describes the **department**, not the employee. Therefore it should not be in the employee table.

Decomposition to achieve 3NF:

1. ED1(Ssn, Ename, Bdate, Address, Dnumber) → Employee info (Dnumber is a FK)
2. ED2(Dnumber, Dname, Dmgr_ssn) → Department info

14.4 General Definitions of 2NF and 3NF

The definitions in Section 14.3 only focused on the **primary key**. However, a robust relational design must evaluate **all candidate keys**.

General Rule for "Prime Attribute"

An attribute is prime if it is a member of **any candidate key** of the relation (not just the primary key).

14.4.1 General Definition of 2NF

General 2NF Rule

A relation schema R is in 2NF if every nonprime attribute A in R is **not partially dependent on any candidate key** of R .

General 2NF Example: LOTS

Relation: LOTS(Property_id#, County_name, Lot#, Area, Price, Tax_rate)

Table 3.5 Candidate Keys and FDs in LOTS

FD	Description
$\text{Property_id\#} \rightarrow \{\text{County_name}, \text{Lot\#}, \text{Area}, \text{Price}, \text{Tax_rate}\}$	Property ID determines everything
$\{\text{County_name}, \text{Lot\#}\} \rightarrow \{\text{Property_id\#}, \text{Area}, \text{Price}, \text{Tax_rate}\}$	County + Lot number also determines everything
$\text{County_name} \rightarrow \text{Tax_rate}$	Tax rate is fixed per county
$\text{Area} \rightarrow \text{Price}$	Price is strictly determined by area

Candidate Keys: $\text{Property_id\#}$ AND $\{\text{County_name}, \text{Lot\#}\}$

The 2NF Violation: $\text{County_name} \rightarrow \text{Tax_rate}$ causes a partial dependency! Tax_rate is a nonprime attribute, and it depends on County_name , which is only **part** of the candidate key $\{\text{County_name}, \text{Lot\#}\}$.

Decomposition to achieve 2NF:

1. $\text{LOTS1}(\underline{\text{Property_id\#}}, \text{County_name}, \text{Lot\#}, \text{Area}, \text{Price}) \rightarrow$ Keeps non-violating data
2. $\text{LOTS2}(\underline{\text{County_name}}, \text{Tax_rate}) \rightarrow$ Extracts the partial dependency

14.4.2 General Definition of 3NF

General 3NF Rule

A relation schema R is in 3NF if, whenever a nontrivial functional dependency $X \rightarrow A$ holds in R , either:

- a. X is a **superkey** of R , OR
- b. A is a **prime attribute** of R .

Alternative General Definition

Every nonprime attribute must be (1) **fully functionally dependent** on every key AND (2) **nontransitively dependent** on every key.

General 3NF Example: Continuing with LOTS1

Look at the newly formed LOTS1 table. It is in 2NF, but is it in 3NF?

The 3NF Violation: We still have $\text{Area} \rightarrow \text{Price}$ in LOTS1.

- Is Area (X) a superkey? **No**.
- Is Price (A) a prime attribute? **No**.
- Therefore, it violates the general definition of 3NF (Price is transitively dependent on the keys via Area).

Decomposition to achieve 3NF:

1. LOTS1A(Property_id#, County_name, Lot#, Area)
2. LOTS1B(Area, Price)

Efficiency Note

If a relation passes the general 3NF test, it automatically passes 2NF as well, meaning you could decompose LOTS straight into LOTS1A, LOTS1B, and LOTS2 without stopping at 2NF.

14.5 Boyce-Codd Normal Form (BCNF)

BCNF was proposed as a **simpler, but stricter**, form of 3NF. Look at clause (b) in the general 3NF definition: "or A is a prime attribute". This clause creates a **loophole** that allows certain problematic functional dependencies to "slip through" because the right-hand side happens to be part of a candidate key. **BCNF closes this loophole.**

The Rule (BCNF)

A relation schema R is in BCNF if whenever a nontrivial functional dependency $X \rightarrow A$ holds in R , X is a **superkey** of R .

Notice that clause 'b' from 3NF has been **completely removed**.

BCNF vs 3NF Example: LOTS1A

Relation: LOTS1A(Property_id#, County_name, Lot#, Area)

Candidate Keys: Property_id# and {County_name, Lot#}

New Constraint: Suppose lot sizes are strictly assigned by county (e.g., DeKalb only has 0.5-1.0 acre lots, Fulton has 1.1-2.0). This gives us: **Area** $\rightarrow$ **County_name**

Testing LOTS1A against Normal Forms

Table 3.6 LOTS1A Normal Form Analysis

Normal Form	Test	Result
Is it in 3NF?	Area $\rightarrow$ County_name: Area is NOT a superkey, BUT County_name IS a prime attribute. Satisfies clause (b).	YES (technically in 3NF)
Is it in BCNF?	Area $\rightarrow$ County_name: Area is NOT a superkey. No other clauses.	NO (violates BCNF)

The Problem with 3NF Here

Even though LOTS1A is technically in 3NF, it still causes **redundancy**. If 1,000 lots are 0.6 acres, we will repeatedly store that they belong to DeKalb county.

Decomposition to achieve BCNF:

1. LOTS1AX(Property_id#, Area, Lot#)
2. LOTS1AY(Area, County_name)

The Dependency Preservation Trade-off

Critical Exam Concept: BCNF vs Dependency Preservation

By decomposing into BCNF, notice what happened to FD2: {County_name, Lot#} $\rightarrow$ Area. This functional dependency is **lost**. We can no longer easily enforce it because County_name and Lot# no longer exist in the same table.

This proves: While BCNF is excellent for eliminating redundancy, it sometimes forces us to sacrifice the **Dependency Preservation Property**.

Chapter 15: Inference Rules and Closures for FDs

When dealing with functional dependencies, it's often necessary to deduce new dependencies from a known set. We use formal rules and algorithms to do this systematically.

Notation Simplification

Moving forward, sets of attributes will be concatenated for convenience. Example: $X \cup Y$ will be written simply as XY .

Armstrong's Axioms (The Basic Inference Rules)

Armstrong proposed three foundational rules (IR1 - IR3) that are both **sound** and **complete**:

- **Sound:** Any FD you derive is guaranteed to be true in every legal relation state.
- **Complete:** Using these rules repeatedly will generate the entire possible set of derived FDs (called the **Closure**, denoted F^+).

Table 3.7 Armstrong's Axioms

Rule	Name	Statement	Meaning
IR1	Reflexive	If $Y \subseteq X$, then $X \rightarrow Y$	A set always determines itself or any subset (trivial FDs)
IR2	Augmentation	If $X \rightarrow Y$, then $XZ \rightarrow YZ$	Adding same attributes to both sides preserves the FD
IR3	Transitive	If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$	FDs behave transitively

Additional Derived Inference Rules

The following rules can be mathematically proven using IR1, IR2, IR3:

Table 3.8 Derived Inference Rules

Rule	Name	Statement	Meaning
IR4	Decomposition	If $X \rightarrow YZ$, then $X \rightarrow Y$ and $X \rightarrow Z$	Split up the right-hand side
IR5	Union	If $X \rightarrow Y$ and $X \rightarrow Z$, then $X \rightarrow YZ$	Combine dependencies with same left-hand side
IR6	Pseudotransitive	If $X \rightarrow Y$ and $WY \rightarrow Z$, then $WX \rightarrow Z$	Substitute left-hand side with another set that determines it

Critical Cautionary Notes

- $X \rightarrow YZ$ does **NOT** imply $X \rightarrow Y$ and $X \rightarrow Z$ separately **without the decomposition rule**.
- $X \rightarrow Y$ and $X \rightarrow Z$ does **NOT** imply $Y \rightarrow Z$ or $Z \rightarrow Y$.

Determining the Closure of a Set of Attributes (X^+)

Database designers usually only specify the semantically obvious FDs (set F). To find everything else that depends on a specific set of attributes X , we calculate its **Closure** (denoted X^+).

Definition

The closure X^+ is the **complete set of attributes** that are functionally determined by X , based on the given set of dependencies F .

Algorithm 15.1: Calculating X^+ **Algorithm**

Input: A set F of FDs on relation schema R , and a subset of attributes X .

1. Set $X^+ := X$ (Start by assuming X determines itself via the Reflexive Rule).
2. Repeat:
 - $oldX^+ := X^+$
 - For every functional dependency $Y \rightarrow Z$ in F :
 - If all attributes of Y are currently inside X^+ :
 - Add the attributes of Z to X^+ ($X^+ := X^+ \cup Z$)
3. Until X^+ stops changing (i.e., $X^+ == oldX^+$)

Example of Finding Closures

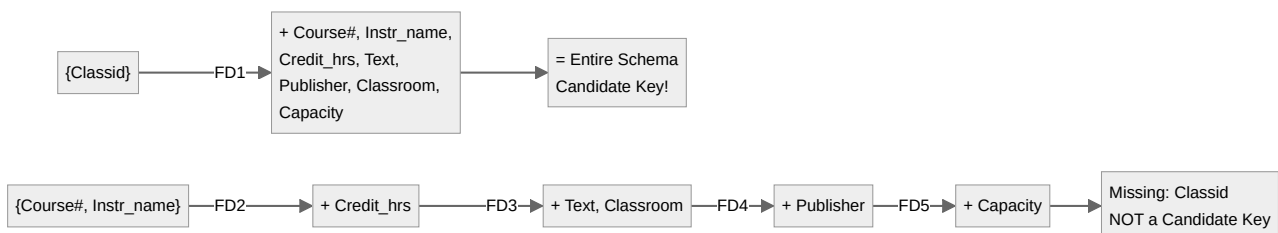
Relation: CLASS(Classid, Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity)

Given FDs (F):

- FD1: Classid → Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity
- FD2: Course# → Credit_hrs
- FD3: Course#, Instr_name → Text, Classroom
- FD4: Text → Publisher
- FD5: Classroom → Capacity

Table 3.9 Closure Calculation Examples

Closure	Steps	Result	Conclusion
{Classid}⁺	Starts as {Classid}; applying FD1 determines everything.	{Classid, Course#, Instr_name, Credit_hrs, Text, Publisher, Classroom, Capacity}	Candidate Key! (closure = entire schema)
{Course#}⁺	Starts as {Course#}; applying FD2 gives Credit_hrs.	{Course#, Credit_hrs}	NOT a candidate key (missing Instr_name, etc.)
{Course#, Instr_name}⁺	Apply FD2 → Credit_hrs; FD3 → Text, Classroom; FD4 → Publisher; FD5 → Capacity	{Course#, Instr_name, Credit_hrs, Text, Classroom, Publisher, Capacity}	NOT a candidate key (Classid is missing!) — proves a course taught by a specific instructor can have multiple class sections.

**Figure 3.3:** Closure Calculation Flow

Part 4: Exam Practice Q&A

This section contains practice questions with detailed answers to help you prepare for your exam.

ER-to-Relational Mapping Questions

Q1: When mapping a regular (strong) entity type to a relation, should you include foreign keys in Step 1?

A: No. Step 1 only maps the entity's own simple attributes. Foreign keys and relationship attributes are added in later steps (Steps 3-7 depending on the relationship type). This keeps entity relations clean and semantically clear.

Q2: A weak entity DEPENDENT has a partial key Dependent_name and owner EMPLOYEE with PK Ssn. What is the primary key of the DEPENDENT relation?

A: The PK is {Essn, Dependent_name}. Essn is the FK referencing EMPLOYEE.Ssn, and it is combined with the partial key Dependent_name to uniquely identify each dependent. You must also specify CASCADE for ON UPDATE and ON DELETE because a weak entity has existence dependency on its owner.

Q3: For a 1:1 relationship between EMPLOYEE and DEPARTMENT where every department must have a manager but not every employee manages a department, where should the foreign key go?

A: The FK (Mgr_ssn) should go into the DEPARTMENT table because DEPARTMENT has **total participation** (every department has a manager). Putting it in EMPLOYEE would create massive NULLs since most employees don't manage a department.

Q4: Why can't we use the foreign key approach for M:N relationships?

A: In an M:N relationship, one entity instance on either side can be related to **multiple** instances on the other side. A single FK column can only hold one value per tuple, so it cannot represent multiple relationships. Therefore, we **must** create a separate relationship relation (cross-reference table) with the PKs of both entities as FKs, and their combination becomes the PK.

Q5: How many join operations are needed to materialize an M:N relationship compared to a 1:N relationship?

A: A 1:N relationship mapped with foreign keys requires **1 join operation** (join the two tables on the FK). An M:N relationship requires **2 join operations** (first join one entity table with the relationship relation, then join the result with the other entity table).

Design Guidelines Questions

Q6: What are the four informal design guidelines for relation schemas?

A: (1) **Clear semantics** — each relation should represent one entity or relationship type; (2) **Reduce redundancy** — avoid update anomalies; (3) **Reduce NULL values** — don't store frequently-null attributes in base relations; (4) **Avoid spurious tuples** — decompose so joins on PK/FK pairs produce only valid data.

Q7: Explain the three types of update anomalies with examples.

A: **Insertion anomalies:** Can't insert a department without employees (would need NULL PK) or must duplicate department info for every new employee. **Deletion anomalies:** Deleting the last employee in a department accidentally deletes the department itself. **Modification anomalies:** Changing a department manager requires updating every employee in that department; partial updates cause inconsistency.

Q8: Why are "fat" relations like EMP_DEPT acceptable as Views but not as Base Relations?

A: As Views, they provide convenient read-only access to joined data without storing redundant data. As Base Relations, they actually store duplicated data, causing insertion, deletion, and modification anomalies. Views are virtual; Base Relations are physical storage.

Q9: What does the nonadditive (lossless) join property guarantee?

A: It guarantees that when you decompose a relation and then join the decomposed tables back together using equality conditions on PK/FK pairs, you get **exactly** the original tuples — no spurious (fake) tuples and no lost tuples.

Functional Dependencies & Normal Forms Questions

Q10: Can you determine a functional dependency just by looking at the current data in a table?

A: No. You can only **disprove** an FD with a counterexample from data. To **prove** an FD, you must understand the real-world semantics and business rules. A current snapshot might coincidentally satisfy an FD that doesn't actually hold in general.

Q11: What is the difference between a partial dependency and a transitive dependency?

A: A **partial dependency** occurs when a nonprime attribute depends on only **part** of a composite primary key (violates 2NF). A **transitive dependency** occurs when a nonprime attribute depends on another nonprime attribute, which in turn depends on the primary key (violates 3NF). Partial = depends on part of key; Transitive = depends on something that depends on key.

Q12: A relation has a single-attribute Primary Key. What normal forms is it automatically in?

A: It is automatically in 2NF (assuming it's also in 1NF). Partial dependencies require a composite key to exist, so with a single-attribute PK, partial dependencies are impossible. However, it may still violate 3NF if there are transitive dependencies.

Q13: What is the difference between 3NF and BCNF?

A: In 3NF, for every nontrivial FD $X \rightarrow A$, either X is a superkey OR A is a prime attribute. In BCNF, the second condition is removed — X **must be a superkey**, period. BCNF is stricter. Every BCNF relation is in 3NF, but not vice versa. BCNF may require sacrificing dependency preservation.

Q14: Why might we intentionally choose 3NF over BCNF?

A: Because achieving BCNF sometimes requires decomposing in a way that **loses functional dependencies** (dependency preservation is sacrificed). If enforcing a particular FD across multiple tables is critical for data integrity, designers may accept 3NF to keep that dependency enforceable within a single table.

Q15: State Armstrong's three axioms and what "sound" and "complete" mean.

A:IR1 Reflexive: If $Y \subseteq X$, then $X \rightarrow Y$. **IR2 Augmentation:** If $X \rightarrow Y$, then $XZ \rightarrow YZ$. **IR3 Transitive:** If $X \rightarrow Y$ and $Y \rightarrow Z$, then $X \rightarrow Z$. **Sound** means any derived FD is guaranteed true. **Complete** means the axioms can derive every FD that logically follows from F .

Q16: How do you use closure (X^+) to find candidate keys?

A: Calculate X^+ for a set of attributes X . If X^+ contains **all attributes** of the relation schema, then X is a superkey. If no proper subset of X also determines all attributes, then X is a **candidate key** (minimal superkey).

Q17: Given FDs: $A \rightarrow BC$, $B \rightarrow D$, $D \rightarrow E$. Find A^+ .

A: Start with $A^+ = \{A\}$. Apply $A \rightarrow BC$: $A^+ = \{A, B, C\}$. Apply $B \rightarrow D$: $A^+ = \{A, B, C, D\}$. Apply $D \rightarrow E$: $A^+ = \{A, B, C, D, E\}$. If these are all attributes, A is a candidate key.

Q18: A relation $R(A, B, C, D)$ has FDs: $AB \rightarrow C$, $B \rightarrow D$, and candidate keys AB and BC . Is R in 3NF? Is it in BCNF?

A: Check $B \rightarrow D$: B is not a superkey ($B^+ = \{B, D\} \neq$ all attributes). D is not a prime attribute (D is not part of any candidate key: AB or BC). So $B \rightarrow D$ violates 3NF. Therefore, R is **NOT in 3NF** and consequently **NOT in BCNF**. Decompose into $R_1(B, D)$ with PK B , and $R_2(A, B, C)$ with PK AB (or BC).

Q19: Decompose $EMP_DEPT(ENAME, SSN, BDATE, ADDRESS, DNUMBER, DNAME, DMGR_SSN)$ into 3NF.

A: PK is SSN . FDs: $SSN \rightarrow \{ENAME, BDATE, ADDRESS, DNUMBER\}$; $DNUMBER \rightarrow \{DNAME, DMGR_SSN\}$. The violation: $DNAME$ and $DMGR_SSN$ are transitively dependent on SSN through $DNUMBER$. Decompose: (1) $EMPLOYEE(SSN, ENAME, BDATE, ADDRESS, DNUMBER)$ where $DNUMBER$ is FK; (2) $DEPARTMENT(DNUMBER, DNAME, DMGR_SSN)$.

Q20: What are the two critical properties that must be maintained when decomposing relations?

A: (1) **Nonadditive (Lossless) Join Property** — mandatory; guarantees no spurious tuples when rejoining. (2) **Dependency Preservation Property** — highly desirable; ensures every original FD can be checked within at least one decomposed table without needing joins.

Part 5: Quick Reference Cheat Sheet

Normal Forms at a Glance

Table 5.1 Normal Forms Summary

Normal Form	Rule	What It Prevents	How to Achieve
1NF	All attributes are atomic (indivisible)	Multivalued attributes, nested relations	Split into separate relations or use atomic domains
2NF	1NF + no partial dependencies of nonprime attributes on any candidate key	Partial dependencies (nonprime attribute depends on part of a composite key)	Decompose to separate the partial dependency
3NF	2NF + no transitive dependencies of nonprime attributes on any candidate key; OR for every FD $X \rightarrow A$: X is superkey OR A is prime	Transitive dependencies (nonprime depends on nonprime)	Decompose to remove the transitive chain
BCNF	For every nontrivial FD $X \rightarrow A$: X is a superkey	All remaining anomalies from prime-attribute loophole in 3NF	Decompose; may lose dependency preservation

ER-to-Relational Mapping Cheat Sheet

Table 5.2 Seven-Step Mapping Quick Reference

Step	ER Construct	Relational Result	Key Notes
1	Regular Entity	Entity relation with simple attributes; choose PK	No FKs yet
2	Weak Entity	Relation with owner's PK as FK + partial key	CASCADE triggers; PK = owner PK + partial key
3	Binary 1:1	FK in entity with total participation (or merge if both total)	Avoid NULLs; include relationship attributes
4	Binary 1:N	FK on the N-side	Add relationship attributes to N-side
5	Binary M:N	New relationship relation with both PKs as FKs	PK = composite of both FKs; CASCADE triggers
6	Multivalued Attribute	New relation with owner PK as FK	PK = FK + multivalued attribute
7	N-ary Relationship	New relation with all participant PKs as FKs	If any entity has cardinality 1, exclude its FK from PK

Armstrong's Axioms Quick Reference

Table 5.3 Inference Rules Summary

Rule	Statement	Use
Reflexive (IR1)	$Y \subseteq X \Rightarrow X \rightarrow Y$	Trivial FDs; starting point for closures
Augmentation (IR2)	$X \rightarrow Y \Rightarrow XZ \rightarrow YZ$	Expanding both sides equally
Transitive (IR3)	$X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$	Chaining dependencies
Decomposition (IR4)	$X \rightarrow YZ \Rightarrow X \rightarrow Y, X \rightarrow Z$	Splitting RHS
Union (IR5)	$X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$	Combining RHS
Pseudotransitive (IR6)	$X \rightarrow Y, WY \rightarrow Z \Rightarrow WX \rightarrow Z$	Substituting LHS

Closure Algorithm Summary

Algorithm: Finding X^+

1. Initialize: $X^+ = X$
2. Repeat until no change:
 - For each FD $Y \rightarrow Z$ in F :
 - If $Y \subseteq X^+$ then $X^+ = X^+ \cup Z$
3. If $X^+ =$ all attributes $\rightarrow X$ is a superkey
4. If no proper subset of X also determines all $\rightarrow X$ is a candidate key

Key Formulas & Definitions

Table 5.4 Essential Formulas

Concept	Formula / Definition
Full Dependency	$X \rightarrow Y$ is full if removing any attr from X destroys it
Partial Dependency	$X \rightarrow Y$ is partial if some proper subset $X' \subset X$ also determines Y
Transitive Dependency	$X \rightarrow Z \rightarrow Y$ where Z is not a candidate key and Y is nonprime
Lossless Join Test	$R_1 \bowtie R_2$ is lossless if $R_1 \cap R_2$ is a superkey of R_1 or R_2
Dependency Preservation	Each FD in F must be enforceable in at least one decomposed relation

Final Exam Checklist

- Know all 7 mapping steps in order
- Know the 4 informal guidelines and their violations
- Be able to identify insertion, deletion, and modification anomalies
- Understand why FDs are semantic, not based on data snapshots
- Know the definitions of 1NF, 2NF, 3NF, and BCNF precisely
- Be able to decompose relations to achieve each normal form
- Know Armstrong's 6 inference rules
- Be able to compute attribute closures and find candidate keys
- Understand the lossless join vs. dependency preservation trade-off

DBMS Detailed Study Notes

File Operations, Indexes, & Transactions

Comprehensive Exam Preparation Material

May 2026

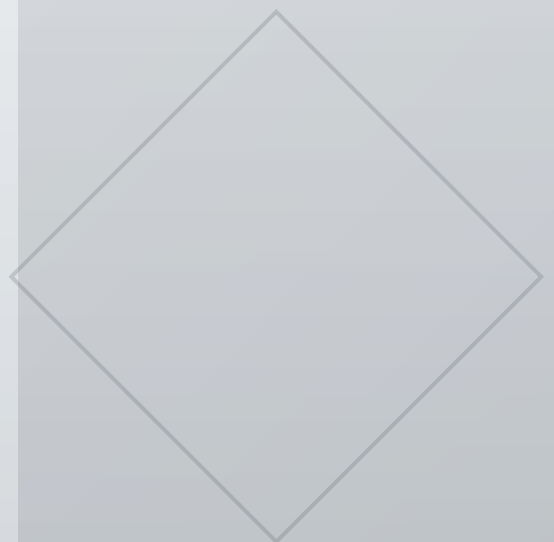


Table of Contents

1. Operations on Files

- 1.1 Categories of File Operations
- 1.2 Selection Conditions
- 1.3 Standard File Operations
- 1.4 File Organization vs. Access Method
- 1.5 Design Choices in File Organization

2. Primary Indexes

- 2.1 What is a Primary Index?
- 2.2 Index Structure
- 2.3 Dense vs. Sparse Indexes
- 2.4 Efficiency Analysis
- 2.5 Problems with Primary Indexes

3. Multilevel Indexes

- 3.1 Concept and Fan-out
- 3.2 B+ Trees

4. Secondary and Clustering Indexes

- 4.1 Clustering Index
- 4.2 Secondary Index

5. Transaction Processing & Concurrency

- 5.1 Single-User vs. Multiuser Systems
- 5.2 Transactions and Operations
- 5.3 Concurrency Control Problems

6. ACID Properties

- 6.1 The ACID Properties
- 6.2 Isolation Levels

7. Concurrency Control Techniques

- 7.1 Two-Phase Locking

7.2 Timestamp Ordering

8. Recovery Techniques

9. Exam Questions & Answers

1. Operations on Files (Section 16.5)

File operations in a Database Management System (DBMS) are the fundamental commands used to access, manage, and modify data stored on disk. These operations are broadly grouped into two categories: retrieval (read-only) and update (modify data).

1.1 Two Main Categories of File Operations

Retrieval Operations

Purpose: To locate specific records so their field values can be examined or processed.
Effect: They do not change any data in the file (Read-only).

Update Operations

Purpose: To modify the file's contents. Effect: They change the file by inserting new records, deleting existing records, or modifying specific field values within a record.

1.2 Selection Conditions (Filtering Conditions)

Whenever we retrieve, delete, or modify a record, we usually do not want to affect the entire database. We use **selection conditions** to specify criteria that the desired records must satisfy.

Table 1 *Types of Selection Conditions*

Type	Description	Example
Simple	Equality comparison on a single field	(Ssn = '123456789') or (Department = 'Research')
Complex	Other comparison operators (>, <, ≥, ≤) or Boolean expressions with AND/OR	(Salary ≥ 30000) AND (Department = 'Research')

How DBMS Handles Complex Conditions: Disk searches are generally optimized for simple conditions. If you give the DBMS a complex condition, it decomposes it to extract a

simple condition first.

Example Process: If the condition is `((Salary ≥ 30000) AND (Department = 'Research'))`, the DBMS might use the simple condition `(Department = 'Research')` to physically locate those specific records on the disk. Once located and brought into memory, each record is then checked against the second part of the condition `(Salary ≥ 30000)` to see if it is a full match.

The "Current Record" Concept

When a search condition matches multiple records, the DBMS locates the first record (based on its physical sequence on the disk). This record is designated as the **current record**. Any subsequent search operations (like finding the next match) will commence from this exact position.

1.3 Standard File Operations

Different systems use different commands, but here is the representative set of standard operations used by high-level programs (like DBMS software) to access records.

Record-at-a-Time Operations

These operations apply to a single record at a time.

Table 2 Record-at-a-Time File Operations

Operation	Description
Open	Prepares the file for reading/writing. Allocates main memory buffers (typically at least two) to hold data blocks from disk. Retrieves the file header. Sets the file pointer to the beginning.
Reset	Sets the file pointer of an already open file back to the very beginning.
Find (Locate)	Searches for the first record satisfying the search condition. Transfers the disk block into a buffer (if not already there). The file pointer now points to this record, making it the current record.
Read (Get)	Copies the current record from the memory buffer into a program variable. Usually advances the current record pointer to the next record, which might trigger reading the next disk block.
FindNext	Searches for the next record satisfying the condition. Transfers its block into the buffer (if needed). The newly found record becomes the new current record.
Delete	Deletes the current record and eventually updates the file on physical disk.
Modify	Changes some field values of the current record and eventually updates the physical disk.
Insert	Locates the correct block for the new record, transfers that block to a buffer, writes the new record, and eventually writes the buffer back to disk.
Close	Completes file access by releasing allocated memory buffers and doing necessary cleanup.

Streamlined Operation: Scan

A hybrid operation that combines Find, FindNext, and Read. If the file is just opened/reset, Scan returns the first record. Otherwise, it returns the next record. It can also accept a search condition to only return matching records.

Set-at-a-Time Operations

These are higher-level operations that manipulate multiple records at once:

- **FindAll:** Locates all records satisfying a condition.
- **Find n (Locate n):** Searches for the first record satisfying a condition, then continues to locate the next n records. Transfers blocks containing all located records into memory.
- **FindOrdered:** Retrieves all records in a specifically requested order.

- **Reorganize:** Starts a reorganization process. Some file setups require maintenance, like physically re-sorting all file records based on a specific field.

1.4 File Organization vs. Access Method

It is crucial to understand the difference between these two terms:

Table 3 *File Organization vs. Access Method*

Concept	Definition	Example
File Organization	The physical structure . How data is organized into records, blocks, and access structures on the storage medium.	Ordering records sequentially by Name.
Access Method	The logical software actions . A group of operations (like Find, Read, Insert) applied to a file.	Indexed access method.

Rule of Thumb: You can apply several access methods to one file organization, but some access methods require a specific organization. For example, you cannot use an "indexed access method" if the file organization does not physically include an "index".

1.5 Design Choices in File Organization

Database Administrators (DBAs) must choose file organizations based on how the file is used.

Static vs. Dynamic Files

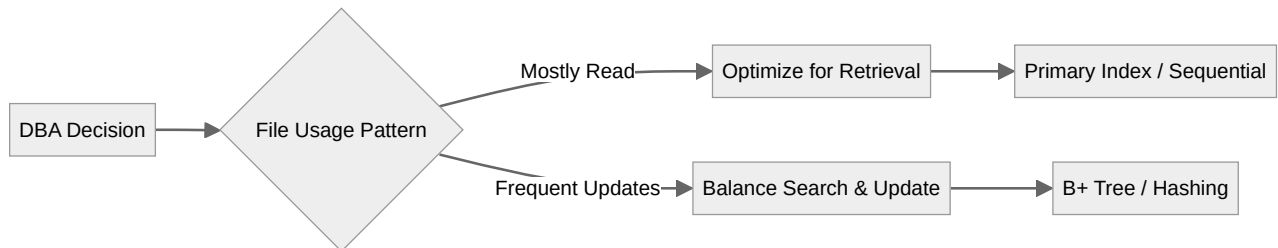
- **Static / Read-only files:** Rarely updated. Optimization focuses entirely on fast retrieval. (e.g., Data Warehouses predominantly use read-only files).
- **Dynamic files:** Frequently updated (insertions/deletions). Optimization must balance fast searching with efficient updating.

Conflicting Requirements (The DBA's Dilemma)

Imagine an EMPLOYEE file:

- **Application 1** constantly searches for employees by Ssn. The best organization is to physically order the file by Ssn or create an index on Ssn.

- **Application 2** needs to print paychecks grouped by Department. The best organization is to cluster records by Department.
- **The Conflict:** You can only physically sort a file one way on the disk. DBAs must make difficult design choices based on the expected mix and importance of different operations.



2. Primary Indexes (Section 17.1.1)

2.1 What is a Primary Index?

A primary index is a supplementary, ordered file that acts as an "access structure" to help you quickly search for and access data records in the main data file.

Key Characteristics of a Primary Index

- It is an **ordered file**.
- Its records are of **fixed length**.
- It consists of exactly two fields:
 1. **Primary Key:** Has the same data type as the ordering key field of the main data file.
 2. **Block Pointer:** A pointer to a physical disk block (block address).

2.2 How the Primary Index is Structured

In the data file shown in [Figure 1](#), the main Data File is physically ordered by the Name field (making Name the primary key). The data file is divided into disk blocks.

Figure 17.1

Primary index on the ordering key field of the file shown in Figure 16.7.

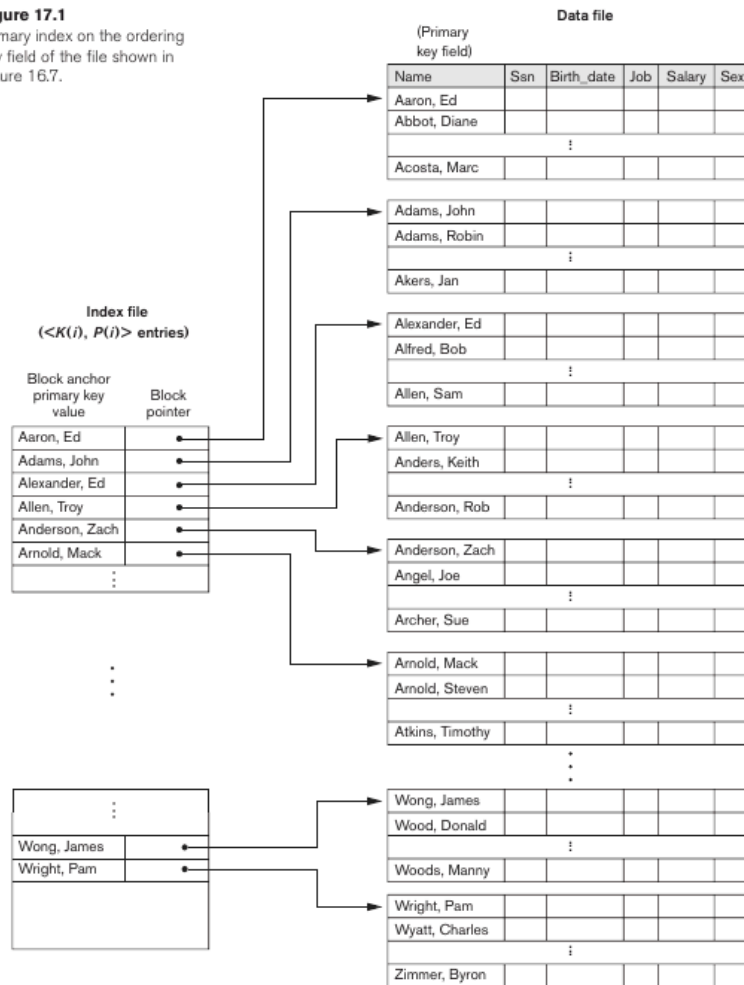


Figure 1 Primary index on the ordering key field of the data file. The index file contains $\langle K(i), P(i) \rangle$ entries where $K(i)$ is the block anchor and $P(i)$ points to the data block.

The **Index Entry format** is $\langle K(i), P(i) \rangle$:

- There is exactly **one index entry per block** in the data file.
- **K(i)**: The value of the primary key for the **first record in block i**. This first record is critically called the **Block Anchor** (or anchor record).
- **P(i)**: A pointer to block i .

Data extraction from Figure 1:

- **Block 1** of Data File: The first record (Block Anchor) is "Aaron, Ed".
Index Entry 1: $\langle K(1) = (\text{Aaron, Ed}), P(1) = \text{address of block 1} \rangle$
- **Block 2** of Data File: The first record (Block Anchor) is "Adams, John".
Index Entry 2: $\langle K(2) = (\text{Adams, John}), P(2) = \text{address of block 2} \rangle$

Note on Pointers: The pointer $P(i)$ in an entry can be a physical address, a record address (relative address within a file), or a logical address mapped to a physical one.

2.3 Dense vs. Sparse Indexes

Table 4 Dense vs. Sparse Index Comparison

Property	Dense Index	Sparse (Nondense) Index
Index entries	One for every single search key value (every record)	One for some search values only
Space required	More space	Less space
Search speed	Directly points to record	Must search within block after locating it
Primary index?	No	Yes -- Primary index is ALWAYS sparse

Important Concept

A Primary Index is **ALWAYS** a Sparse Index. Why? Because it only contains an entry for the block anchors (one per block), not an entry for every single employee in the file.

2.4 Why are Primary Indexes Highly Efficient?

Because the index is small and ordered, we can perform a **Binary Search** on the index file, which requires drastically fewer disk accesses than searching the main data file.

Mathematical Efficiency Example

The Setup (Main Data File):

- Total records (r) = 30,000
- Block size (B) = 1024 bytes
- Record length (R) = 100 bytes
- Blocking factor (bfr) = $\lfloor B/R \rfloor = 10$ records per block
- Total data blocks (b) = $\lceil r/bfr \rceil = 3000$ blocks
- **Cost without Index:** Binary search on data file requires $\lceil \log_2 b \rceil = \lceil \log_2 3000 \rceil = 12$ block accesses.

The Improvement (With Primary Index):

- Key field size (V) = 9 bytes
- Pointer size (P) = 6 bytes
- Index entry size (R_i) = $V + P = 15$ bytes
- Index blocking factor (bfr_i) = $\lfloor B/R_i \rfloor = 68$ entries per block
- Total index entries (r_i) = $b = 3000$ (one for each data block)
- Total index blocks (b_i) = $\lceil r_i/bfr_i \rceil = 45$ blocks
- **Cost with Index:** Binary search on the index requires $\lceil \log_2 b_i \rceil = 6$ accesses. Plus 1 final access to grab the actual data block = **7 block accesses total**.

Summary

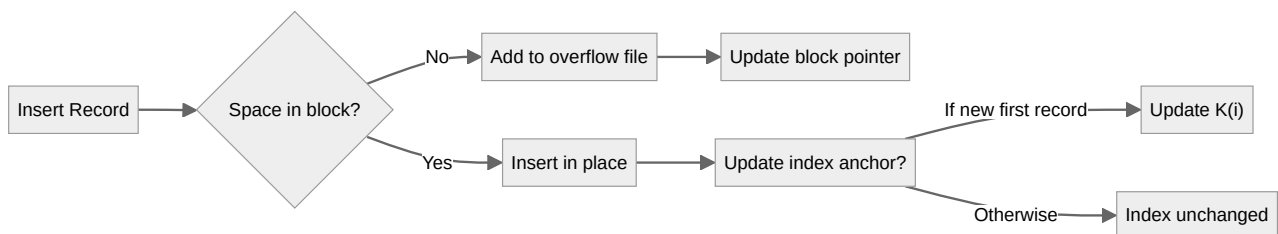
Without index: 12 block accesses. With primary index: 7 block accesses. The primary index nearly **doubles** search efficiency, and the gains are even more dramatic for larger files.

2.5 The Major Problem with Primary Indexes

Because the main file is **physically ordered**, insertions and deletions are very expensive. If you insert a new record, you have to physically move existing records down to make space. Moving records changes the Block Anchors, forcing continuous updates to the Primary Index.

Solutions:

- Use an **unordered overflow file** for new insertions.
- Use **linked lists** of overflow records per block.
- Use **deletion markers (tombstones)** instead of physically removing records.



3. Multilevel Indexes (Section 17.2)

3.1 Concept and Fan-out

Standard indexing schemes (like the Primary Index) use a binary search on the index file.

- A binary search reduces the search space by a factor of 2 at each step.
- Number of accesses required: $\lceil \log_2 b_i \rceil$ for b_i index blocks.

A **multilevel index** is designed to reduce the search space significantly faster than a binary search. Instead of dividing the search space by 2, it divides it by the index's blocking factor (bfr_i).

Fan-out (fo)

The blocking factor of the index (bfr_i) is referred to as the **fan-out** (fo). This represents how many n-ways the search space is divided at each step.

Math Comparison: Searching a multilevel index requires approximately $\lceil \log_{fo} b_i \rceil$ block accesses, as opposed to $\lceil \log_2 b_i \rceil$. Since the fan-out is usually much larger than 2, the number of accesses plummets.

Real-world Example of Fan-out Power

Assume a standard block size of **4096 bytes**.

- If a block pointer is **7 bytes** (which can accommodate 2^{56} blocks).
- If the search key (like an SSN) is **9 bytes**.
- The size of one index entry = $9 + 7 = \mathbf{16 \text{ bytes}}$.
- The Fan-out (fo) = $\lfloor 4096/16 \rfloor = \mathbf{256}$ entries per block.
- **Result:** Instead of dividing the search space in half (2) at each step, a multilevel index divides it by **256** at each step!

3.2 B+ Trees

Multilevel indexes naturally lead to a structure called the **B+ Tree**, which is the most widely used index structure in modern DBMSs.

B+ Tree Properties

- All paths from root to leaf have the same length (balanced tree).
- Each internal node has between $\lceil fo/2 \rceil$ and fo children.
- Leaf nodes contain actual data pointers (or the data records themselves in a clustered index).
- Leaf nodes are linked together for efficient range queries.

Table 5 B+ Tree Order (m) vs. Fan-out

Parameter	Definition	Typical Value
Order (m)	Maximum number of pointers per node	100 - 500
Fan-out	Actual average number of pointers used	50 - 250
Height	Number of levels from root to leaf	2 - 4 for millions of records

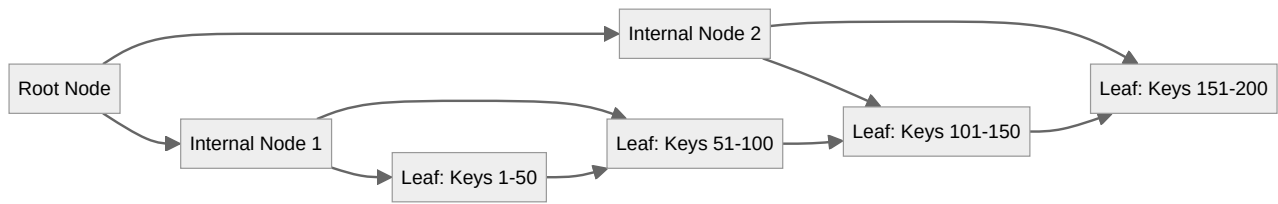
Insertion in B+ Tree: Insertions may cause node splitting (if a node exceeds fo entries), which can propagate up to the root, increasing tree height. Because splitting is localized, insertions are much cheaper than reorganizing an entire sorted file.

Deletion in B+ Tree: Deletions may cause node underflow (below $\lceil fo/2 \rceil$ entries). Remedies: redistribution from a sibling or merging with a sibling.

Why B+ Trees Dominate

1. They maintain automatic balance without expensive reorganization.
2. They support both exact-match and range queries efficiently.
3. Insertions and deletions are logarithmic time.
4. Leaf-level linking enables fast sequential access after a search.

3. Multilevel Indexes (Section 17.2)



4. Secondary and Clustering Indexes (Section 17.1.2, 17.1.3)

These index types solve problems that primary indexes cannot. They allow efficient access by fields other than the primary ordering key.

4.1 Clustering Index

If records are **physically ordered** on a non-key field (a field that does not have a unique value for each record), the index on that field is called a **clustering index**.

Clustering Index Characteristics

- There is **one entry per distinct value** of the clustering field, not one per block.
- Each entry points to the **first block** that contains a record with that value.
- All records with the same clustering field value are stored **consecutively**.
- It is a **sparse index** because there is one entry per distinct value, not per record.

Example: An EMPLOYEE file physically ordered by Department. The clustering index has one entry for "Research", pointing to the first block with Research employees. All Research employees are stored contiguously.

Table 6 *Primary vs. Clustering Index*

Property	Primary Index	Clustering Index
Ordering field	Primary key (unique)	Non-key field (may have duplicates)
Entries	One per data block	One per distinct value
Data file order	Ordered by primary key	Ordered by clustering field
Type	Sparse	Sparse

4.2 Secondary Index

A **secondary index** provides an alternative way to access records when the primary or clustering index does not match the desired search field. The data file is **not physically ordered** by the secondary index field.

Secondary Index Types

Type A: Secondary index on a key field (unique values)

- One index entry per record.
- This is a **dense index**.
- Entry format: <KeyValue, RecordPointer>

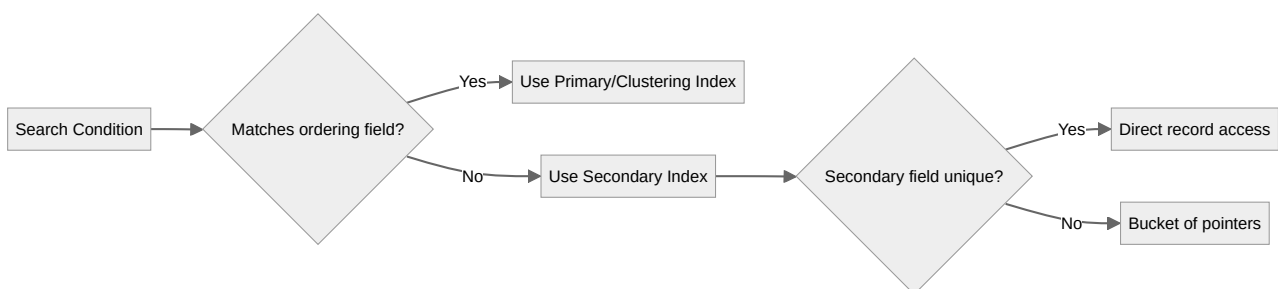
Type B: Secondary index on a non-key field (duplicate values allowed)

- Option 1: One index entry per record (dense) with multiple entries having the same key value.
- Option 2: One index entry per distinct key value, where the pointer points to a bucket of record pointers.

Table 7 Comparison of All Index Types

Index Type	Data File Ordered?	Dense/Sparse	Entry Points To
Primary	Yes (by primary key)	Sparse	Data block (block anchor)
Clustering	Yes (by clustering field)	Sparse	First block with that value
Secondary (key)	No	Dense	Individual record
Secondary (non-key)	No	Dense or bucket	Individual record or bucket

Important: A file can have **at most one** physical ordering, therefore at most one primary or clustering index. But it can have **many secondary indexes**.



5. Transaction Processing & Concurrency (Section 20.1)

5.1 Single-User versus Multiuser Systems

Table 8 *Single-User vs. Multiuser DBMS*

Feature	Single-User DBMS	Multiuser DBMS
Users	At most one user at a time	Many users concurrently
Examples	Personal computer databases	Airline reservations, banks, stock exchanges
Concurrency	None	Hundreds or thousands of simultaneous transactions

How Concurrent Execution Works

Multiuser access is possible due to **multiprogramming**, allowing the Operating System to execute multiple programs (processes) at the same time.

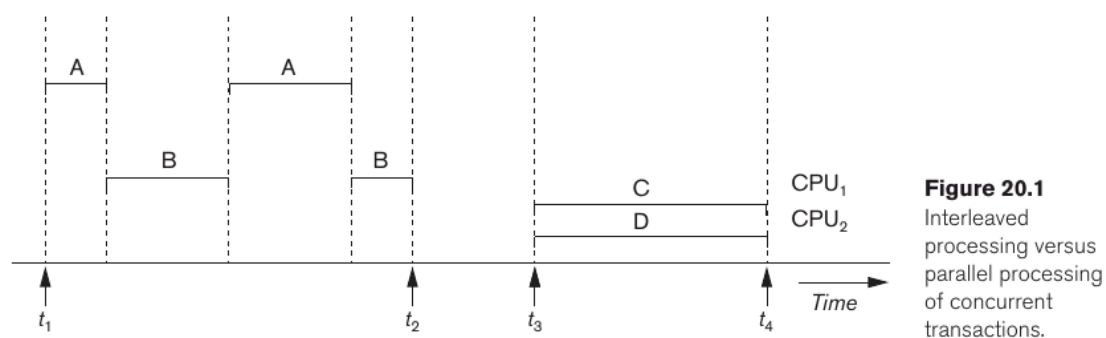


Figure 20.1
Interleaved
processing versus
parallel processing
of concurrent
transactions.

Figure 2 *Interleaved processing versus parallel processing of concurrent transactions. Left: Processes A and B take turns on CPU1. Right: Processes C and D run simultaneously on CPU1 and CPU2.*

1. Interleaved Processing (Single CPU): A single CPU can only execute one process at an exact moment. However, it executes commands from Process A, suspends A, executes Process B, suspends B, and resumes A. Interleaving keeps the CPU busy when a process is

waiting for I/O (like reading from a disk) and prevents long processes from hogging the system.

2. Parallel Processing (Multiple CPUs): If the hardware has multiple CPUs, processes can truly run at the exact same time. Process C runs on CPU1 while Process D simultaneously runs on CPU2.

Note

Most concurrency control theory in databases is based on the **interleaved model**.

5.2 Transactions, DB Items, Operations, and Buffers

Transaction

An executing program that forms a logical unit of database processing. It includes one or more database access operations (insert, delete, update, read). Transaction boundaries are defined by explicit begin transaction and end transaction statements in an application.

Types: Read-only transaction (only retrieves data) vs. Read-write transaction (updates data).

Database Items and Granularity

A database is treated as a collection of named data items. **Granularity** is the size of a data item. It could be an entire disk block, a database record, or just an individual field value (like a specific salary attribute).

Basic Access Operations

Transactions interact with these items using two basic commands:

Table 9 Basic Transaction Operations

Operation	Steps
read_item(X)	1. Find the disk block address containing X. 2. Copy that disk block into a main memory buffer (if not already there). 3. Copy item X from the buffer to the program variable.
write_item(X)	1. Find the disk block address containing X. 2. Copy that disk block into a main memory buffer. 3. Copy item X from the program variable into its correct location in the buffer. 4. Store the updated disk block from the buffer back to disk. (This physically updates the database).

DBMS Buffers & Memory Management

- Step 4 of a write operation is not always immediate. The DBMS Recovery Manager and the OS maintain a cache of data buffers in main memory.
- When buffers are full and a new block needs to be loaded, the system uses a buffer replacement policy, commonly **LRU (Least Recently Used)**.
- If a buffer being replaced has been modified, it must be written back to the physical disk before the space is reused.

Figure 20.2

Two sample transactions.

- (a) Transaction T_1 .
(b) Transaction T_2 .

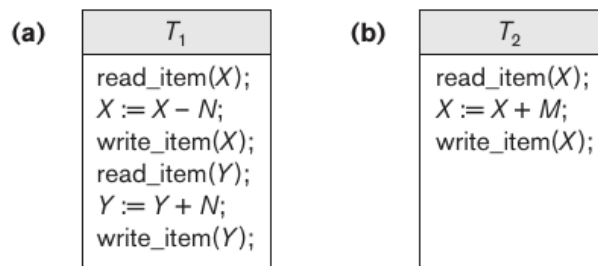


Figure 3 Two sample transactions. (a) Transaction T_1 reads X , modifies X , writes X , reads Y , modifies Y , writes Y . (b) Transaction T_2 reads X , modifies X , writes X .

Read-Set and Write-Set

- **Read-set:** The set of all items a transaction reads.
- **Write-set:** The set of all items a transaction writes.

Example from Figure 3:

- Transaction T1 reads X, modifies X, writes X, reads Y, modifies Y, writes Y.
Read-set = {X, Y}, Write-set = {X, Y}.
- Transaction T2 reads X, modifies X, writes X.
Read-set = {X}, Write-set = {X}.

5.3 Why Concurrency Control Is Needed

When multiple transactions execute concurrently without strict control, disaster strikes. Let us look at four major problems using a simplified airline reservation example:

- Item X: Number of reserved seats on Flight 1.
- Item Y: Number of reserved seats on Flight 2.
- Transaction T1: Transfers N reservations from Flight X to Flight Y.
- Transaction T2: Reserves M seats on Flight X.

Figure 20.3

Some problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update problem. (c) The incorrect summary problem.

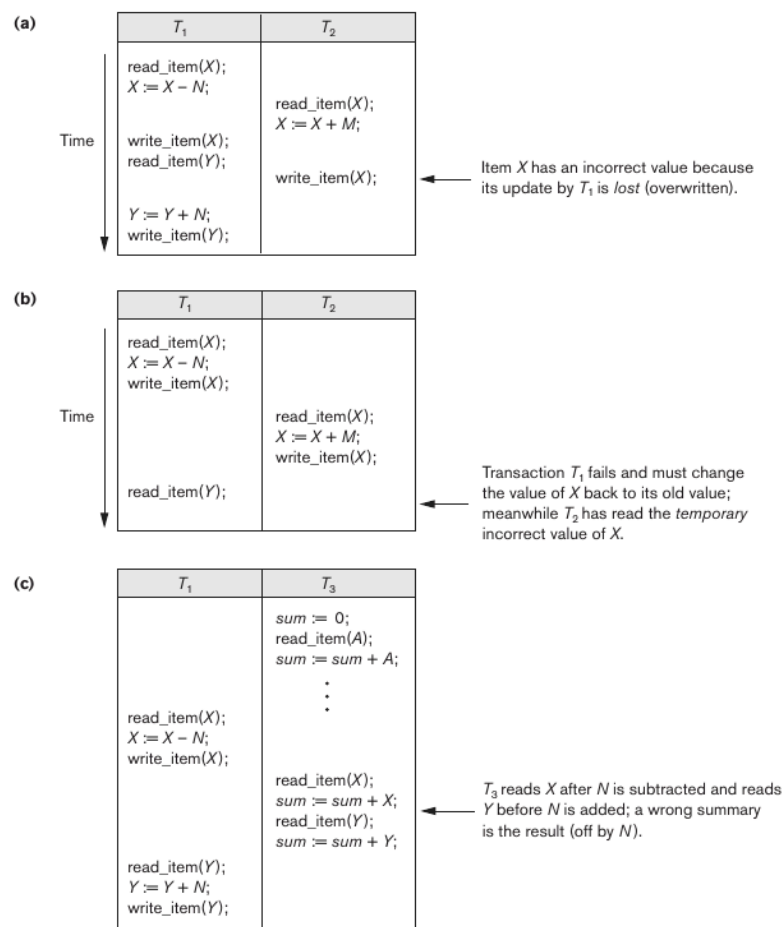


Figure 4 Problems that occur when concurrent execution is uncontrolled. (a) The lost update problem. (b) The temporary update (dirty read) problem. (c) The incorrect summary problem.

The Scenario

- T1 and T2 are submitted at the same time.
- T1 reads X.
- T2 reads X.
- T1 updates X ($X := X - N$) and writes X.
- T2 updates X ($X := X + M$) and writes X.

The Error: Because T2 read the value of X before T1 wrote its changes, T2's final write completely overwrites (loses) T1's update.

Mathematical Proof: If initial $X = 100$, $N = 5$, $M = 10$. The final result should be $100 - 5 + 10 = 105$. However, because T2 read X as 100 and added 10, it writes 110 to the database. T1's subtraction of 5 is lost!

2. The Temporary Update (Dirty Read) Problem (Figure 4b)

Occurs when one transaction updates an item, fails, and must be rolled back -- but another transaction reads that temporary, incorrect value before the rollback happens.

The Scenario

- T1 reads X, updates X ($X := X - N$), and writes X.
- T2 reads X. (This is Dirty Data because T1 hasn't finished yet).
- T1 fails for some reason! The system must roll X back to its original value.

The Error: T2 has already read the temporary, incorrect value of X and is proceeding with its calculations based on data that technically never existed permanently in the database.

3. The Incorrect Summary Problem (Figure 4c)

Occurs when one transaction calculates an aggregate summary (like a sum) while other transactions are actively updating the items being summarized.

The Scenario

- Transaction T3 is calculating the total number of reservations across all flights.
- T3 reads X and adds it to sum.
- Interleave: T1 executes completely. It subtracts N from X and adds N to Y.
- T3 continues its loop and eventually reads Y, adding it to sum.

The Error: T3 read X after N seats were subtracted, but it read Y before those N seats were added. The final summary result will be short by exactly N seats.

4. The Unrepeatable Read Problem

Occurs when a transaction T1 reads the exact same database item twice during its execution, but another transaction T2 modifies that item in between the two reads.

The Scenario

A customer checks seat availability on several flights. They decide on Flight X. Before finalizing, the transaction reads Flight X's availability a second time just to be sure. Transaction T2 booked a seat on Flight X between the first and second read. The original transaction gets two completely different values for the exact same read operation.

6. Desirable Properties of Transactions (ACID) (Section 20.3)

For a database to be reliable, transactions must possess specific properties. These are commonly known as the **ACID properties**. They are enforced by the concurrency control and recovery methods of the DBMS.

6.1 The ACID Properties Explained

1. Atomicity (The "All or Nothing" Rule)

Definition: A transaction is an indivisible, atomic unit of processing. It must be performed entirely, or not performed at all.

Responsibility: Enforced by the transaction recovery subsystem of the DBMS.

Mechanism: If a transaction fails to complete (e.g., due to a system crash midway), the recovery technique must undo any partial effects it had on the database. If it successfully commits, its write operations must be written to disk.

Example: You are transferring 100 from Account A to Account B. The system deducts 100 from A, but crashes before adding it to B. Because of atomicity, the system will undo the deduction from A so no money vanishes into thin air.

2. Consistency Preservation

Definition: A transaction must take the database from one valid, consistent state to another valid, consistent state. (A consistent state means the database satisfies all rules and schema constraints).

Responsibility: Enforced primarily by the programmers (who write logical database programs) and the DBMS integrity constraints module.

Example: In a bank database, a constraint states that the sum of all money across all accounts must not randomly change. If a transaction transfers money, the total sum of money in the bank must be identical before and after the transaction runs.

3. Isolation

Definition: A transaction should appear as if it is executing in complete isolation from all other transactions, even if hundreds are executing concurrently. It should experience zero interference.

Responsibility: Enforced by the concurrency control subsystem.

Mechanism: One common way to enforce this is to ensure a transaction's updates (writes) remain invisible to other transactions until it fully commits. (This entirely solves the "Dirty Read" problem mentioned earlier).

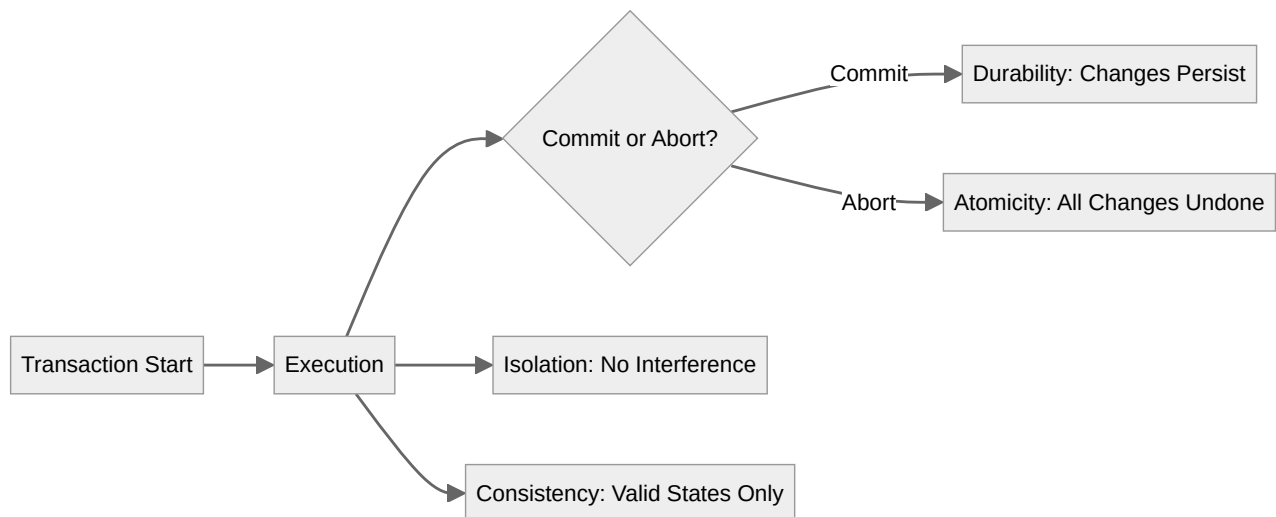
Example: If two travel agents try to book the last seat on a flight at the exact same millisecond, isolation ensures they don't see a jumbled, half-updated database. One transaction completes its full booking first, and the second one sees the seat as fully booked.

4. Durability (Permanency)

Definition: Once a transaction is "committed" (successfully completed), its changes to the database must persist permanently. They must never be lost, even in the event of a catastrophic system failure.

Responsibility: Enforced by the recovery subsystem.

Example: If you get a confirmation screen that your concert ticket is booked, and the server loses power two seconds later, your ticket is still reserved when the server boots back up.



6.2 Levels of Isolation

DBMS systems can enforce varying degrees of isolation depending on the performance vs. strictness needed. These levels build upon each other:

Table 10 *Isolation Levels and Problems Solved*

Level	Properties	Problems Prevented
Level 0	Does not overwrite dirty reads of higher-level transactions	Basic protection
Level 1	Level 0 + no lost updates	Lost Update Problem (Figure 4a)
Level 2	Level 1 + no dirty reads	Temporary Update / Dirty Read (Figure 4b)
Level 3 (True Isolation)	Level 2 + repeatable reads	Unrepeatable Read + all above

Note: Another practical method often used in modern DBMS is called **snapshot isolation**, which is an alternative to standard locking mechanisms. In snapshot isolation, each transaction sees a consistent snapshot of the database as of the time it started.

7. Concurrency Control Techniques

This section supplements the core notes with additional concurrency control mechanisms that are essential for a complete understanding of transaction management.

7.1 Two-Phase Locking (2PL)

Two-Phase Locking is the most common concurrency control protocol. It guarantees serializability by regulating when transactions can acquire and release locks.

Lock Types

- **Shared Lock (S-lock / Read Lock):** A transaction can read but not write the item. Multiple transactions can hold S-locks on the same item simultaneously.
- **Exclusive Lock (X-lock / Write Lock):** A transaction can both read and write the item. Only one transaction can hold an X-lock on an item at any time.

The Two-Phase Locking Protocol

Every transaction must follow these two phases:

1. **Growing Phase (Expanding Phase):** The transaction may obtain locks (S or X), but may not release any lock.
2. **Shrinking Phase (Contracting Phase):** The transaction may release locks, but may not obtain any new locks.

2PL Variants

- **Basic 2PL:** Locks are released as soon as possible after the peak (when the transaction starts shrinking).
- **Strict 2PL:** All exclusive locks are held until the transaction commits or aborts. This prevents cascading rollbacks.
- **Rigorous 2PL:** All locks (shared and exclusive) are held until commit/abort. Simplifies recovery but reduces concurrency.

Table 11 2PL Lock Compatibility Matrix

	S-lock requested	X-lock requested
S-lock held	Compatible	Not compatible
X-lock held	Not compatible	Not compatible

7.2 Timestamp Ordering

An alternative to locking that uses transaction timestamps to enforce serializability without locks or deadlock detection.

Timestamp Ordering Rules

Each transaction T is assigned a unique timestamp $TS(T)$ when it starts. For each data item X , the system maintains:

- **read_TS(X):** The timestamp of the youngest transaction that read X .
- **write_TS(X):** The timestamp of the youngest transaction that wrote X .

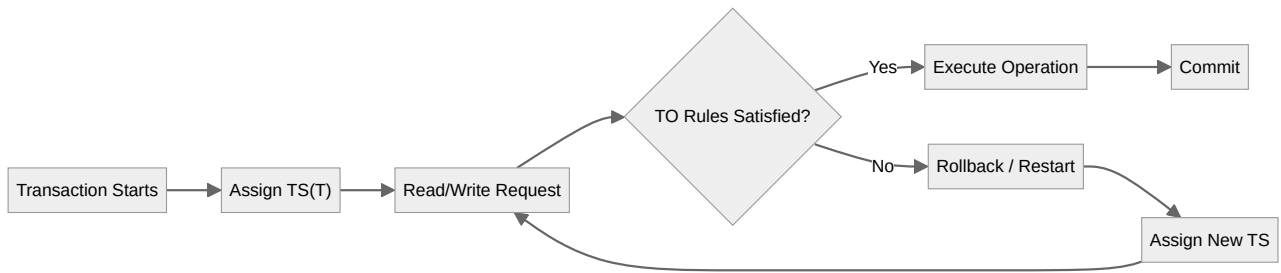
Basic Timestamp Ordering (TO) Protocol:

- **Transaction T issues read(X):**
 - If $TS(T) < \text{write_TS}(X)$: T is trying to read a value that was overwritten after T started. Reject the operation and roll back T (T is "too late").
 - If $TS(T) \geq \text{write_TS}(X)$: Allow the read. Update $\text{read_TS}(X) = \max(\text{read_TS}(X), TS(T))$.
- **Transaction T issues write(X):**
 - If $TS(T) < \text{read_TS}(X)$: The item was already read by a younger transaction. Reject and roll back T .
 - If $TS(T) < \text{write_TS}(X)$: T is trying to write an obsolete value. Reject and roll back T .
 - Otherwise: Allow the write. Update $\text{write_TS}(X) = TS(T)$.

Thomas Write Rule (Optimization)

If $TS(T) < \text{write_TS}(X)$ when T tries to write X , instead of rejecting T , simply **ignore the write** (it would be obsolete anyway). This reduces unnecessary rollbacks while still guaranteeing serializability.

7. Concurrency Control Techniques



8. Recovery Techniques

Recovery ensures **atomicity** and **durability** when failures occur. The recovery manager must bring the database back to a consistent state.

8.1 Types of Failures

Table 12 *Types of System Failures*

Failure Type	Cause	Effect
Transaction failure	Logical error, system error, deadlock	Transaction must abort
System crash	Power failure, software bug	Volatile memory lost; non-volatile storage intact
Disk failure	Head crash, hardware defect	Non-volatile storage partially lost

8.2 Log-Based Recovery

The most common recovery technique maintains a **log** (or journal) on stable storage that records all modifications to the database.

Log Record Types

- `[T, X, old_value, new_value]` -- Update record for transaction T on item X.
- `[T, start]` -- Transaction T has started.
- `[T, commit]` -- Transaction T has committed.
- `[T, abort]` -- Transaction T has aborted.

Deferred Update (NO-UNDO/REDO)

- Updates are not written to the database until the transaction commits.
- If transaction aborts: nothing needs to be undone (NO-UNDO).

- If system crashes after commit but before disk write: REDO all committed transactions using the log.

Immediate Update (UNDO/REDO)

- Updates can be written to the database before commit.
- If transaction aborts: UNDO its changes using the log's old values.
- If system crashes: UNDO uncommitted transactions, REDO committed ones.

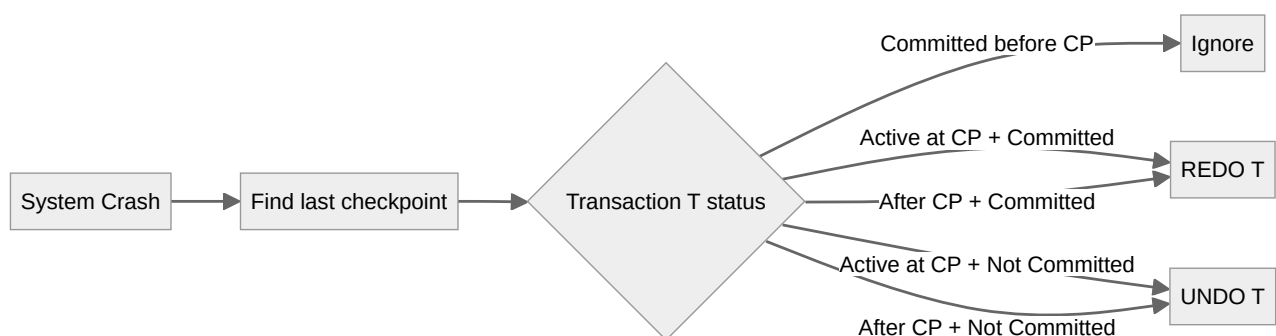
8.3 Checkpoints

To reduce recovery time, the system periodically establishes a **checkpoint**:

1. Suspend all transactions temporarily.
2. Force all modified buffers to disk.
3. Write a [checkpoint, L] record to the log, where L is the list of active transactions.
4. Resume transactions.

Checkpoint Recovery Rules

- Transactions that committed **before** the checkpoint: Already on disk, nothing to do.
- Transactions in list L (active at checkpoint): Must be examined. If committed, REDO; if not, UNDO.
- Transactions that started **after** the checkpoint: If committed, REDO; if not, UNDO.



9. Exam Questions & Answers

This section contains practice questions and detailed answers to help you prepare for your DBMS examination.

Chapter 1: File Operations

Q1. What is the difference between retrieval and update operations on files?

A: Retrieval operations (like Find, Read) locate specific records for examination but do not change any data in the file -- they are read-only. Update operations (like Insert, Delete, Modify) change the file's contents by adding, removing, or altering records.

Q2. Explain the concept of a "current record" in file operations.

A: When a search condition matches multiple records, the DBMS locates the first record based on its physical sequence on the disk. This record is designated as the current record. Subsequent search operations (like FindNext) commence from this exact position.

Q3. What is the difference between File Organization and Access Method?

A: File Organization is the physical structure of how data is stored on disk (e.g., sequential ordering by Name). Access Method is the logical set of operations applied to a file (e.g., Find, Read, Insert). Several access methods can be applied to one organization, but some methods require specific organizations.

Q4. Describe the DBA's dilemma in file organization design.

A: Different applications may need the same file organized differently. For example, one application searches by SSN (best order: SSN), while another groups by Department (best order: Department). Since a file can only be physically sorted one way, the DBA must choose based on the expected mix and importance of operations.

Chapter 2: Primary Indexes

Q5. Define a primary index and list its key characteristics.

A: A primary index is a supplementary, ordered file that acts as an access structure to help quickly search data records. It is an ordered file with fixed-length records. Each record has two fields: (1) a primary key matching the data file's ordering key, and (2) a block pointer to a physical disk block.

Q6. Why is a primary index always a sparse (nondense) index?

A: A primary index is always sparse because it only contains one entry per data block (the block anchor), not one entry for every single record in the file. Since a block contains multiple records, the index has fewer entries than the data file has records.

Q7. Calculate the search cost with and without a primary index given: $r=30,000$ records, $B=1024$ bytes, $R=100$ bytes, $V=9$ bytes, $P=6$ bytes.

A:

- Blocking factor $bfr = \text{floor}(1024/100) = 10$ records/block.
- Data blocks $b = \text{ceil}(30000/10) = 3000$ blocks.
- Without index: $\text{ceil}(\log_2 3000) = 12$ block accesses.
- Index entry size $= 9 + 6 = 15$ bytes.
- Index blocking factor $= \text{floor}(1024/15) = 68$ entries/block.
- Index blocks $= \text{ceil}(3000/68) = 45$ blocks.
- With index: $\text{ceil}(\log_2 45) + 1 = 6 + 1 = 7$ block accesses.

Q8. What is the major problem with primary indexes, and what are the solutions?

A: Because the main file is physically ordered, insertions and deletions are expensive. Inserting a record may require moving existing records, which changes block anchors and forces index updates. Solutions: use an unordered overflow file, use linked lists of overflow records per block, or use deletion markers (tombstones).

Chapter 3: Multilevel Indexes

Q9. What is fan-out, and why is it more efficient than binary search for multilevel indexes?

A: Fan-out (fo) is the blocking factor of the index -- how many entries fit in one index block. Binary search divides the search space by 2 at each step ($\log_2 b$ accesses). A multilevel index divides the search space by fo at each step ($\log_{fo} b$ accesses). Since fo is typically 100-500, the number of accesses is drastically reduced.

Q10. List four reasons why B+ trees are the dominant index structure in modern DBMSs.

A:

1. They maintain automatic balance without expensive reorganization.
2. They support both exact-match and range queries efficiently.
3. Insertions and deletions are logarithmic time.
4. Leaf-level linking enables fast sequential access after a search.

Chapter 4: Secondary and Clustering Indexes

Q11. What is the difference between a clustering index and a secondary index?

A: A clustering index is built on a non-key field where the data file is physically ordered by that field. It is sparse (one entry per distinct value). A secondary index is built on a field where the data file is NOT physically ordered by that field. It can be dense (one entry per record) or use buckets for non-key fields.

Q12. Can a file have multiple primary indexes? Multiple secondary indexes? Why?

A: A file can have at most ONE primary or clustering index because the data file can only be physically ordered one way. However, it can have MANY secondary indexes because secondary indexes are separate ordered files that do not affect the physical ordering of the data file.

Chapter 5: Concurrency Problems

Q13. Describe the Lost Update Problem and how it occurs.

A: The Lost Update Problem occurs when two transactions read the same item, then one writes before the other reads, and the second write overwrites the first. Example: T1 and T2 both read X=100. T1 sets X=95 and writes. T2 (still using X=100) sets X=110 and writes. T1's update to 95 is lost.

Q14. What is a Dirty Read, and why is it dangerous?

A: A Dirty Read occurs when transaction T2 reads an item that has been modified by transaction T1, but T1 has not yet committed. If T1 later aborts and rolls back its changes, T2 has used data that never permanently existed in the database, leading to incorrect decisions or writes.

Q15. Explain the Incorrect Summary Problem.

A: This occurs when a transaction calculating an aggregate (like a sum) reads some items before another transaction updates them and other items after the update. The aggregate reflects a partial, inconsistent state. Example: T3 sums X+Y. It reads X before T1 subtracts N, but reads Y after T1 adds N. The sum is off by N.

Chapter 6: ACID Properties

Q16. Name and define the four ACID properties of transactions.

A:

- **Atomicity:** All operations complete successfully, or none do. Enforced by recovery.
- **Consistency:** Database moves from one valid state to another. Enforced by integrity constraints and correct programs.
- **Isolation:** Each transaction executes as if alone, despite concurrency. Enforced by concurrency control.
- **Durability:** Committed changes survive any failure. Enforced by recovery.

Q17. Which isolation level solves which concurrency problem?

A:

- Level 1: Prevents lost updates.
- Level 2: Prevents lost updates and dirty reads.
- Level 3: Prevents lost updates, dirty reads, and unrepeatable reads (true isolation).

Chapter 7: Concurrency Control Techniques

Q18. Explain the Two-Phase Locking protocol and its phases.

A: In 2PL, every transaction has two phases: (1) Growing Phase -- the transaction may obtain locks but may not release any. (2) Shrinking Phase -- the transaction may release locks but may not obtain any new locks. This protocol guarantees serializability.

Q19. What is the difference between Strict 2PL and Basic 2PL?

A: In Basic 2PL, locks can be released as soon as the shrinking phase begins. In Strict 2PL, all exclusive (write) locks are held until the transaction commits or aborts. Strict 2PL prevents cascading rollbacks but reduces concurrency compared to Basic 2PL.

Q20. How does Timestamp Ordering prevent serializability violations without using locks?

A: Each transaction gets a unique timestamp. For each data item, the system tracks read_TS and write_TS. If a transaction tries to read an item written by a younger transaction, or write an item already read/written by a younger transaction, it is rolled back and restarted with a new timestamp. This enforces an order equivalent to serial execution by timestamp.

Chapter 8: Recovery

Q21. What is the purpose of a checkpoint in log-based recovery?

A: A checkpoint reduces the amount of log that must be examined during recovery. At a checkpoint, the system forces all modified buffers to disk and writes a checkpoint record listing active transactions. During recovery, transactions that committed before the checkpoint need no action, limiting REDO/UNDO work to recent transactions.

Q22. In immediate update (UNDO/REDO) recovery, what actions are taken after a crash?

A: After a crash with immediate update, the recovery manager: (1) REDOs all committed transactions to ensure their updates are on disk (durability). (2) UNDOs all uncommitted (active) transactions to erase any partial changes they wrote before the crash (atomicity).

Comprehensive / Long-Answer Questions

Q23. Compare and contrast Primary, Clustering, and Secondary indexes with respect to: (a) whether the data file is ordered, (b) dense vs. sparse, (c) what the pointer points to, and (d) maximum number allowed per file.

A:

Index	Data Ordered?	Type	Pointer	Max per File
Primary	Yes, by primary key	Sparse	Data block	1
Clustering	Yes, by clustering field	Sparse	First block with value	1
Secondary (key)	No	Dense	Individual record	Many
Secondary (non-key)	No	Dense or bucket	Record or bucket	Many

Q24. With a diagram or clear step-by-step explanation, show how Two-Phase Locking prevents the Lost Update Problem.

A: Consider T1 and T2 both wanting to write X.

1. T1 requests and acquires X-lock on X (growing phase).
2. T2 requests X-lock on X but must wait because T1 holds it.
3. T1 reads X, modifies X, writes X, then commits and releases X-lock (shrinking phase).
4. T2 now acquires X-lock, reads the updated X, modifies it, and writes.

Because T2 was blocked until T1 finished, T2 reads the value after T1's update. No update is lost.

Q25. Explain how the ACID properties map to the concurrency and recovery subsystems of a DBMS.

A:

- **Atomicity:** Enforced by the recovery subsystem. Uses logs to UNDO partial changes of failed transactions.
- **Consistency:** Enforced by programmers writing correct logic and by the DBMS integrity constraints module (keys, checks, triggers).
- **Isolation:** Enforced by the concurrency control subsystem. Uses locking (2PL) or timestamp ordering to prevent interleaving conflicts.
- **Durability:** Enforced by the recovery subsystem. Uses force-writing logs and checkpoints to ensure committed data survives crashes.